

The Platform Engineer's Supplement Interview Book

100 questions on Linux, automation, Go, containers, bare metal and HPC — the topics senior platform roles ask about beyond OpenShift

RHEL 9 · Ansible AAP · Python · Go · Podman/Buildah · Satellite · GPU Operator

Edition V6 - supplement to the OpenShift V5 book · quick-learning highlighted copy

Reading route: seventeen parts, three gears

Parts are ordered so each one only depends on the ones before it. Inside every part the questions climb from Foundation to Architect, so you can stop at your level and come back later.

Gear	Parts	What you are building	Stop when you can
Basics	1-4	Linux and container primitives, the control plane, workload objects, scheduling and resources	Explain what happens between oc apply and a running pod without notes
Intermediate	5-11	Networking, storage, security, operators, cluster lifecycle and observability	Trace a request end to end and read a failure from the owning controller's conditions
Senior	12-17	Troubleshooting under pressure, HA and DR, HPC, automation, fleet operations and architecture	Run an incident, defend a design trade-off and describe prevention, not just a fix

Every question carries a spoken answer, an analogy, production context, a step-by-step procedure, the evidence you would quote, the red flag that costs you the offer, and the follow-up question the interviewer is already holding.

Contents

100 questions across 8 parts. Parts are ordered so that nothing depends on a concept you have not met yet, and inside each part the questions climb from Foundation to Architect.

Part	Topic	Questions
1	Senior Linux administration systemd, storage, SELinux, and the failure modes that show up before Kubernetes ever gets blamed	Q1-Q14
2	Bash and shell scripting for SRE set -euo pipefail, traps, and the glue code that outlives every dashboard	Q15-Q24
3	Python for SRE and platform engineers The language platform teams actually write production automation in	Q25-Q36
4	Advanced Ansible for senior platform engineers Variable precedence, testing, execution environments and enterprise controller patterns	Q37-Q50
5	Go for platform engineers The language behind client-go, operators and most of the control plane — and what interviewers actually probe	Q51-Q64
6	Podman, Buildah, Skopeo and container images Building and moving images without Docker — the toolchain RHEL and OpenShift CI actually standardise on	Q65-Q76
7	Bare metal provisioning From BMC power-on to a node joining the cluster — Satellite, PXE, Kickstart and IroniC	Q77-Q88
8	HPC and AI on OpenShift GPUs, RDMA, MPI and the operators that turn a Kubernetes cluster into an AI factory	Q89-Q100

Read this page first

This book exists because most interview prep fails in the same way: you can recite the definition, the interviewer asks "and how would you prove that?", and the conversation stops. Definitions are table stakes. What gets scored is whether you can name the owning controller, quote the evidence, choose the smallest safe action, and say what you would change afterwards so it never happens again.

So every one of the 250 questions here is answered in six layers. Read the first layer if you are short on time. Read all six if this is a topic you would rather not be surprised by.

ABOUT THE YELLOW HIGHLIGHTING

This is the quick-learning copy. In every question the opening claim of the spoken answer and the closing consequence of the production context are **marked in yellow like this**. Read only the highlighted sentences for a fast revision pass through all 250 questions, then come back to the full text for the topics where the highlight did not feel like enough. A clean unhighlighted copy of the same book is published alongside this one.

How every answer in this book is built

Six layers per question. Under pressure you speak layers 1-2; when the interviewer digs, you already have layers 3-6 loaded.

1. Say this	The 45-90 second spoken answer. Plain words, no padding, leads with the point.
2. Think of it like	One analogy that makes the mechanism obvious and buys you thinking time.
3. Why it matters	The production context: what breaks, who notices, what it costs.
4. Step by step	The ordered procedure or diagnostic path you would actually follow.
5. Evidence	The commands and signals that prove your claim instead of asserting it.
6. Red flag	The answer that quietly loses the offer, plus the follow-up you should expect.

Levels, and why they are marked

Each question is tagged Foundation, Intermediate, Senior or Architect. The tag is not about how hard the topic is - it is about how deep the expected answer goes. A Foundation question about etcd quorum and an Architect question about etcd quorum are the same subject scored on completely different scales.

The answer ladder: what each level actually sounds like

Same question - "What is a PodDisruptionBudget?" - answered four ways. Interviewers score the rung you land on, not the number of facts you list.

Foundation

Names the object

"A PDB controls how many pods can be disrupted."
Correct, forgettable, scores one point.

Intermediate

Explains the mechanism

"It caps voluntary disruptions so a minimum number of replicas stays available during a drain."

Senior

Adds failure modes and evidence

"It only covers voluntary eviction, not node crashes. I check the selector and healthy replica count before maintenance, because a badly sized PDB blocks drains and stalls MCO rollout."

Architect

Adds trade-off, standard and prevention

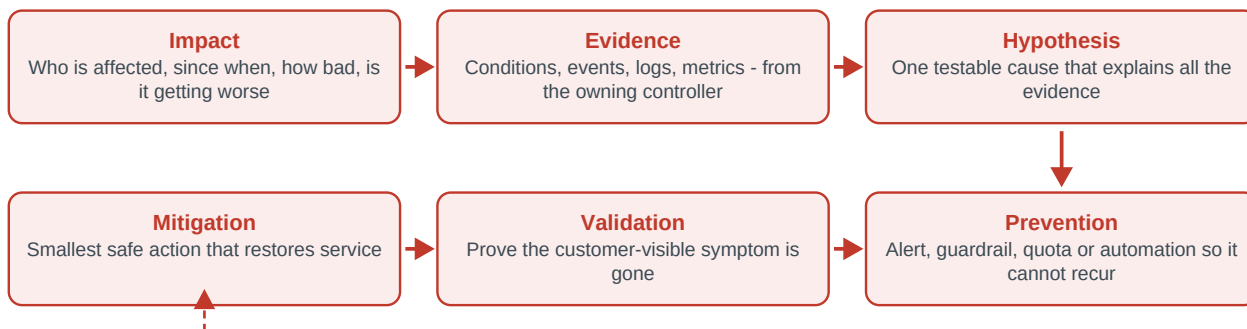
"I pair PDBs with replica minimums, topology spread and tested maintenance headroom, and I validate them in a game day so the first real drain is not the first test."

Two frameworks that carry most of the interview

Roughly a third of a senior platform interview is troubleshooting and another third is design. If you only memorise two things from this book, memorise these two loops. They give you a structure to speak into when a question catches you cold, which is worth more than any individual fact.

The evidence-first loop - use this for every troubleshooting question

Say these six words out loud before you say anything technical. Candidates who jump straight to "I would restart the pod" lose the point even when restarting works.



For design and architecture questions, swap the loop for: Requirement, Options, Trade-off, Decision, Risk, Verification. Say the requirement back before you propose anything - it is the single fastest way to sound like someone who has shipped a platform rather than read about one.

The scorecard your interviewer is probably using

Score yourself 0-3 on each row after every mock answer. Anything below 2 is where your next hour of study should go.

Dimension	What a 3 sounds like
Correctness	Names the right object, the owning controller and the control loop involved
Evidence	Quotes conditions, events, metrics or a specific command instead of asserting
Safety	Controls blast radius, uses supported procedures, says what could go wrong
Validation	Checks the customer-visible symptom, not just that a resource is green
Prevention	Adds an alert, guardrail or automation so the incident cannot repeat
Communication	States impact and uncertainty plainly, and flags trade-offs without hedging

A three-day, twenty-hour run

If you have three days, do not read this book front to back at a constant speed. Spend day one on Parts 1 to 6 for breadth and to find your weak spots. Spend day two on Parts 7 to 13, which is where most senior interviews actually live: storage, security, lifecycle, observability, troubleshooting and recovery. Spend day three on Parts 14 to 17 plus spoken rehearsal - answer thirty questions out loud, timed, without notes.

In the final hour, stop reading new material. Re-read only the red flags and the analogies. The red flags stop you from losing points you have already earned, and the analogies are what you will reach for when you are asked something you did not prepare.

Honest scope note

The healthcare and enterprise examples in this book are illustrative. They are not descriptions of any specific customer environment or confidential architecture. Product behaviour is described against current OpenShift 4.x and ACM 2.x documentation; a real project may run an earlier supported or EUS release. In the interview, state the principle first, then ask which version and infrastructure provider the project runs on. That question reads as experience, not ignorance.

PART 1 · Q1-Q14 · 14 QUESTIONS

Senior Linux administration

systemd, storage, SELinux, and the failure modes that show up before Kubernetes ever gets blamed

Platform interviews still open on Linux because every abstraction leaks. A pod that will not mount a volume is often an SELinux label problem. A node that goes NotReady is frequently disk, OOM, or a systemd unit in a restart loop. Senior candidates are expected to work from symptoms to evidence without guessing — journalctl before reboot, ss before tcpdump, and an AVC denial before chmod 777. **This part covers the administration depth that separates someone who knows kubectl from someone who can own the node underneath.**

Level mix - Foundation: 3 · Intermediate: 5 · Senior: 4 · Architect: 2

A container is a Linux process wearing five costumes

There is no "container" object in the kernel. Every isolation feature you see is a separate Linux mechanism, which is exactly why container problems are usually Linux problems.

Namespaces	What the process can see: PID, network, mount, UTS, IPC, user
cgroups v2	What the process can use: CPU shares and quota, memory limit, IO and PID caps
Union filesystem	What the process reads: stacked read-only image layers plus a thin writable layer
SELinux + seccomp	What the process may do: MCS labels on files, syscall filtering, capability set
Image + registry	What the process is: a manifest of layers addressed by an immutable digest
Pod (Kubernetes)	The shared unit: one network namespace and shared volumes for one or more containers

Q1**FOUNDATION**

What are the main Linux process states, and which ones should worry you in production?

SAY THIS

Running means on CPU or runnable. Sleeping in an interruptible wait is normal for I/O. Uninterruptible sleep — state D — means the process is stuck in a kernel call, usually blocked disk or NFS, and cannot be killed until the kernel returns. Zombie means the process exited but the parent has not reaped it; one zombie is harmless, thousands mean a broken parent. Stopped is a job-control pause. In interviews and incidents, D-state processes and zombie storms are the ones that escalate because they do not respond to SIGKILL and they often point at storage or a buggy supervisor.

THINK OF IT LIKE

Think of process states as people in a queue. Running is at the counter. Sleeping is waiting politely and can leave if called. D-state is glued to the counter mid-transaction — security cannot move them. Zombies are receipts nobody filed away.

WHY IT MATTERS IN PRODUCTION

On OpenShift nodes you see D-state during hung NFS or a failing SAN LUN; kubelet and container processes pile up and the node looks busy while nothing completes. Zombie leaks from a wrapper script that never wait on children can exhaust the PID table even when free memory looks fine. **Interviewers use this question to see whether you reach for kill -9 or read /proc and the parent chain.**

STEP BY STEP

- 1 Name the states interviewers expect: R, S, D, Z, T — and what each means in one line.
- 2 Highlight D-state as unkillable and almost always storage or kernel I/O.
- 3 Explain zombies as a parent bug, not a child problem — find who is not reaping.
- 4 Close with the operational move: ps, /proc/<pid>/stack or wchan, and fix storage or the parent.

EVIDENCE YOU WOULD QUOTE

```
ps aux | awk '$8 ~ /D/ {print}'

cat /proc/<pid>/wchan ; cat /proc/<pid>/stack

ps -eo pid,ppid,stat,cmd | awk '$3 ~ /Z/' | head -20
```

RED FLAG - DO NOT SAY THIS

Do not say you would SIGKILL a D-state process. It does not work and tells the interviewer you have not lived through a storage outage.

EXPECT THIS FOLLOW-UP

You see fifty D-state processes on one node. What do you check before draining the node?

Q2

FOUNDATION

How do you use journalctl to troubleshoot a failed systemd service?

SAY THIS

journalctl is the unified log for systemd and anything that logs to the journal. I start with the unit since boot — journalctl -u nginx.service -b — then widen the window if needed. -f follows live, --since narrows time, -p err filters priority, and -o json-pretty helps when I need structured fields. If the service fails on start I also check systemctl status for the exit code and the last few log lines inline. On RHEL the journal persists under /var/log/journal unless someone disabled it, so reboot does not automatically erase the evidence you need for the interview story.

THINK OF IT LIKE

journalctl is the building's CCTV system with timestamps and camera IDs. systemctl status is the alarm panel summary; the journal is where you actually see who opened the door.

WHY IT MATTERS IN PRODUCTION

This comes up constantly for kubelet, crio, chronyd, and node-exporter on platform nodes. Candidates who only tail /var/log/messages miss stdout from Type=simple units and lose the ordering systemd already captured. In a 2025/2026 SRE loop, correlating journal timestamps with Prometheus alerts is the expected workflow — not grepping random flat files.

STEP BY STEP

- 1 State that systemd aggregates service, kernel, and structured logs in one place.
- 2 Give the first command: journalctl -u <unit> -b and systemctl status <unit>.
- 3 Add filters: --since, -p err, -n, and -o json-pretty for automation.
- 4 Mention persistence and rotation — verify Storage= and not assuming logs vanish on restart.

EVIDENCE YOU WOULD QUOTE

```
systemctl status kubelet --no-pager -l

journalctl -u kubelet -b --since '30 min ago' -p err --no-pager
```



```
journalctl -u crio -n 100 -o json-pretty | head -40
```

RED FLAG - DO NOT SAY THIS

Do not say you always edit `rsyslog.conf` first. On modern RHEL, `systemd journal` is the primary source and `rsyslog` is optional forwarding.

EXPECT THIS FOLLOW-UP

Logs show the service started cleanly but clients still fail. Where do you look next?

Q3**FOUNDATION****What is systemd responsible for, and how do you manage a service day to day?****SAY THIS**

`systemd` is PID 1: it starts the boot target, brings up units in dependency order, supervises daemons, mounts filesystems, and applies `cgroups` to service processes. A unit file describes what to run — Service section — when — `Install` and `WantedBy` — and how to restart. Day to day I use `systemctl start`, `stop`, `enable`, and `daemon-reload` after unit edits. `status` shows Active state and the last logs; `show` tells me the effective unit after drop-ins. The interview point is that `systemd` is not just `init` — it is the process supervisor and dependency graph the whole node relies on.

THINK OF IT LIKE

`systemd` is an air-traffic controller, not a single runway worker. It decides which planes take off in what order, watches them in flight, and clears a gate when one crashes.

WHY IT MATTERS IN PRODUCTION

On RHCOS many changes go through `MachineConfig` rather than hand-edited units, but debug pods still use `systemctl` and `journalctl` on the host. Platform candidates confuse `systemd` with Kubernetes controllers; the parallel is real — desired state in unit files, observed state in `systemctl` — but the failure modes are local: a bad `After=` dependency, a missing `EnvironmentFile`, `Restart=` always masking a config error.

STEP BY STEP

- 1 Define `systemd` as PID 1 plus unit supervision, dependencies, mounts, and `cgroups`.
- 2 Explain unit file sections: Unit, Service, Install — and drop-in overrides.
- 3 List daily commands: `enable`, `start`, `status`, `daemon-reload`, `cat/show`.
- 4 Connect to troubleshooting: failed state, restart loops, and ordering with `After/Wants/Requires`.

EVIDENCE YOU WOULD QUOTE

```
systemctl list-units --failed --no-pager
```

```
systemctl cat kubelet.service
```

```
systemctl show kubelet -p ActiveState,SubState,RestartCount,FragmentPath
```

RED FLAG - DO NOT SAY THIS

Do not describe `systemd` as "just replacing `init` scripts". Miss the dependency graph and supervision story and senior follow-ups will expose it immediately.

EXPECT THIS FOLLOW-UP

A unit is in a restart loop with exit code 1. How do you stop the loop without losing logs?

Q4

INTERMEDIATE

How do you read and fix an SELinux AVC denial on RHEL?

SAY THIS

An AVC denial means SELinux blocked an action that UNIX permissions allowed — different domain, wrong file context, or a capability the policy denies. I reproduce or wait for the failure, then search audit logs: `ausearch -m avc -ts recent` or `journalctl` with SETroubleshoot hints. The denial names source context, target context, object class, and permission. The fix hierarchy is: apply the correct file context with `semanage fcontext` plus `restorecon`, use a supported boolean if it is a known pattern, or adjust the service domain through policy modules — never `setenforce 0` in production. On OpenShift, pod MCS labels and volume contexts are the same problem wearing a different hat.

THINK OF IT LIKE

UNIX permissions are a door lock. SELinux is the badge reader next to it. You can have the key and still be denied because your badge is not cleared for that room.

WHY IT MATTERS IN PRODUCTION

This is still a top-ten real interview question for RHEL and OpenShift roles in 2025/2026. Classic cases: a custom app writing to `/var/log`, NFS mounts without the right context, a backup agent reading container volumes.

Candidates who reach for `chmod 777` fail the security bar; candidates who `grep AVC` and fix labels pass.

STEP BY STEP

- 1 Define AVC as mandatory access control denying despite `chmod` being open.
- 2 Collect evidence: `ausearch`, `sealert`, or `journalctl` AVC lines with timestamp correlation.
- 3 Read the denial fields: `scontext`, `tcontext`, `tclass`, `permission`.
- 4 Apply the fix ladder: `fcontext/restorecon`, boolean, policy module — and verify with a test run.

EVIDENCE YOU WOULD QUOTE

```
ausearch -m avc -ts recent | tail -20

sealert -a /var/log/audit/audit.log 2>/dev/null | less

ls -Z /path/to/file ; semanage fcontext -l | grep myapp
```

RED FLAG - DO NOT SAY THIS

Do not recommend `setenforce 0` or disabling SELinux as a normal fix. That is an instant fail in regulated and platform interviews.

EXPECT THIS FOLLOW-UP

The denial mentions `container_file_t` but the app needs a host log directory. What are your options?

Q5

INTERMEDIATE

How does the Linux OOM killer work, and how do you tell it from a cgroup memory limit?

SAY THIS

When the kernel cannot satisfy a memory allocation and reclaim is exhausted, the OOM killer selects a process using an `oom_score` derived from RSS, age, and tunables like `oom_score_adj`. Containers and systemd services often set `oom_score_adj` so critical daemons survive longer. Exit code 137 in a container usually means cgroup memory.max was hit — the killer ran inside that cgroup. Host-level OOM shows in dmesg as "Out of memory: Killed process" and can take down sshd or kubelet if the node is truly exhausted. The interview skill is splitting those two stories with evidence before blaming the application.

THINK OF IT LIKE

Host OOM is the building manager evicting someone when the whole block is out of space. Cgroup OOM is your flat's smart meter cutting power because you exceeded your prepaid cap — the building still has electricity.

WHY IT MATTERS IN PRODUCTION

Platform engineers see both daily. A pod OOMKilled with node memory free is almost always limits. A node flapping NotReady with ssh hangs is often host OOM or memory pressure evictions. In 2026 interviews expect a follow-up on overcommit, swap absence on RHCOS, and why requests matter for scheduling but limits enforce death.

STEP BY STEP

- 1 Explain the trigger: allocation failure after reclaim, then `oom_score` selection.
- 2 Differentiate cgroup limit enforcement from global host OOM via dmesg versus pod status.
- 3 Mention `oom_score_adj` — kubelet and systemd adjust who dies first.
- 4 Land on mitigation: right-size limits, fix leaks, protect critical daemons, alert on memory pressure.

EVIDENCE YOU WOULD QUOTE

```
dmesg -T | grep -i 'out of memory'
```

```
oc describe pod <pod> | grep -A5 'Last State' # or crictl inspect for OOM
```

```
cat /proc/<pid>/oom_score_adj ; systemctl show kubelet -p OOMScoreAdjust
```

RED FLAG - DO NOT SAY THIS

Do not say "Kubernetes killed the pod for using too much memory" without mentioning the kernel OOM killer and cgroup limits — the distinction is the whole question.

EXPECT THIS FOLLOW-UP

Node memory looks 40% free but the OOM killer still ran. What mechanisms could explain that?

Q6

INTERMEDIATE

df reports 100% full but du shows plenty of free space. How do you troubleshoot?**SAY THIS**

I work through the usual suspects in order. Deleted-but-open files: `lsuf +L1` or `lsuf | grep deleted` — space returns only when the process closes the descriptor. Inodes exhausted: `df -i` shows 100% while blocks remain; find tiny files or runaway socket files. Reserved blocks on ext4: `tune2fs -l` shows five percent root-only reserve — `df` counts it as used. A mount overlay: `df` on the wrong mountpoint hides that `/var/log` is on a full partition. Sparse or deleted snapshots can confuse if you mix LVM and bind mounts. Each hypothesis has a thirty-second test; the interview win is a ordered method, not guessing reboot.

THINK OF IT LIKE

It is like a warehouse that looks empty on paper but every pallet is still booked because someone forgot to close the shipping manifest — the space exists but the ledger says full.

WHY IT MATTERS IN PRODUCTION

This is a classic senior Linux phone-screen question and a real kubelet failure mode: `/var` fills because container logs were deleted but still held open by a long-running process, image layers cannot unpack, and the node enters DiskPressure. Candidates who only run `du` on `/` and give up lose credibility fast.

STEP BY STEP

- 1 Confirm which filesystem with `df -h` and `df -i` on the exact mountpoint.
- 2 Check deleted open files with `lsuf +L1` sorted by size.
- 3 Inspect reserved blocks and mount topology with `tune2fs -l` and `findmnt`.
- 4 Fix the holder process or inode consumer, then verify with `df` before closing the incident.

EVIDENCE YOU WOULD QUOTE

```
df -h /var ; df -i /var

lsuf +L1 2>/dev/null | sort -k7 -n | tail -20

findmnt /var ; tune2fs -l /dev/sda2 | grep -i block
```

RED FLAG - DO NOT SAY THIS

Do not answer "just reboot the server" as step one. You lose the chance to name the process holding deleted files and you destroy evidence.

EXPECT THIS FOLLOW-UP

`lsuf` shows a multi-gigabyte deleted log held by `journald`. What safe cleanup path do you take?

Q7

INTERMEDIATE

How do you debug basic network connectivity from a Linux host?**SAY THIS**

I split "cannot connect" into layers. DNS: `getent hosts` or `dig` shows name resolution. Routing: `ip route` get tells me which interface and source IP the kernel would use. Local listen state: `ss -tlnp` shows whether anything is bound on the port. Path and firewall: `ping` or `tracert` for L3, `curl -v` for application handshake, `tcpdump` on the interface for silent drops. "Connection refused" means I reached a host that actively rejected the port — nothing listening or firewall REJECT. "Timed out" means packets vanished — wrong route, ACL, or asymmetric return path. That distinction is what interviewers listen for.

THINK OF IT LIKE

Debugging the network is like delivering a letter: first check the address book, then the route on the map, then whether anyone is home to sign for it, then whether the post office threw it away without telling you.

WHY IT MATTERS IN PRODUCTION

SRE interviews use this to filter out candidates who only know `kubectl exec curl`. On nodes you debug CNI, DNS CoreDNS upstream, MTU mismatches on overlay networks, and conntrack exhaustion — all of which start with `ip`, `ss`, and `tcpdump` on the host. **In 2025/2026 expect IPv6 and nftables to appear in follow-ups.**

STEP BY STEP

- 1 Clarify the failure string: refused versus timeout versus name resolution failure.
- 2 Test DNS and routing before the application — `getent`, `ip route get`.
- 3 Verify local sockets with `ss -tlnp` and remote reachability with `curl -v` or `nc -vz`.
- 4 Capture packets with `tcpdump` when the syscalls lie — counters increment but clients hang.

EVIDENCE YOU WOULD QUOTE

```
getent hosts api.example.com ; dig +short api.example.com

ip route get 10.0.0.5 ; ss -tlnp | grep :8443

curl -v --connect-timeout 3 https://api.example.com:8443/health
```

RED FLAG - DO NOT SAY THIS

Do not skip straight to `tcpdump` without stating what refused versus timeout implies. That misdiagnosis wastes minutes in real incidents.

EXPECT THIS FOLLOW-UP

`curl` times out but `tcpdump` shows SYN leaving and SYN-ACK returning. What do you check next?

Q8**INTERMEDIATE****Explain LVM building blocks and how you extend a filesystem online.****SAY THIS**

Physical volumes are disk partitions or LUNs. Volume groups pool PV capacity. Logical volumes are sliced from the VG and presented as block devices like `/dev/mapper/rhel-root`. To grow online I extend the LV with `lvextend -r` if the filesystem supports it, or `lvextend` then `xfs_growfs` or `resize2fs` separately. I always verify `pvdisplay`, `vgdisplay`, and `lvdisplay` before typing — extending the wrong LV on a shared VG is a resume event. Snapshots and thin pools are advanced topics but the interview baseline is PV/VG/LV vocabulary and a safe grow workflow.

THINK OF IT LIKE

LVM is a flexible wallet system: PVs are cash in your pocket, the VG is the wallet, LVs are labelled envelopes inside it — you can move space between envelopes without buying a new wallet.

WHY IT MATTERS IN PRODUCTION

Bare-metal and VM platform roles still manage root and data volumes outside Kubernetes. Satellite hosts, registry nodes, and database VMs on the cluster edge all hit "disk eighty-five percent" tickets. **Interviewers want to hear that XFS grow is online on RHEL, that you check VG free space first, and that you update kickstart or Terraform after the manual fix so the next rebuild matches.**

STEP BY STEP

- 1 Define PV, VG, LV and how device mapper names map to mounts.
- 2 Check free space: `vgs`, `lvs`, `df` on the mountpoint.

- 3 Extend: `lvextend -L +50G /dev/vg/lv` then `xfs_growfs` or `resize2fs`.
- 4 Verify `df`, update infrastructure-as-code or kickstart so the change is not a one-off snowflake.

EVIDENCE YOU WOULD QUOTE

```
pvs ; vgs ; lvs -o lv_name,vg_name,lv_size,data_percent

df -h /var/lib ; lsblk -f

lvextend -r -L +20G /dev/rhel/var && xfs_growfs /dev/rhel/var
```

RED FLAG - DO NOT SAY THIS

Do not run `lvextend` without confirming filesystem type and VG free space. Growing is easy; shrinking XFS is not, and interviewers will ask.

EXPECT THIS FOLLOW-UP

The VG is out of free extents but the SAN LUN has unused space. What are your next steps?

Q9**SENIOR**

How do you write a production-grade systemd unit with dependencies and restart policy?

SAY THIS

I start from a vendor unit or `systemd-analyze cat systemd-example.service` and override with drop-ins under `/etc/systemd/system/foo.service.d/` rather than editing vendor files. `After=` and `Wants=` express ordering versus hard requirement — `Requires=` fails the unit if the dependency fails. `Type=notify` for daemons that signal readiness; `Restart=on-failure` with `RestartSec=` avoids tight loops. `LimitNOFILE=` and `MemoryMax=` belong in the unit for services I own. `ExecStartPre=` runs validation before the main process. I always `systemctl daemon-reload`, `systemctl start`, then `systemd-analyze verify` and `systemd-analyze critical-chain` to prove ordering. On fleet-managed nodes the same content becomes a MachineConfig fragment.

THINK OF IT LIKE

A unit file is a recipe card with prep steps, oven temperature, and what must finish before this dish goes in — skip the prep line and the whole kitchen backs up.

WHY IT MATTERS IN PRODUCTION

Senior platform roles wrap agents, exporters, and bootstrap scripts in systemd. The interview tests whether you understand that `Restart=` always hides misconfiguration, that ordering races cause "works on reboot sometimes", and that `Type=simple` marks started before the daemon is actually ready.

STEP BY STEP

- 1 Prefer drop-in overrides; list Unit, Service, Install sections you actually set.
- 2 Explain `After` versus `Requires` — ordering versus hard dependency.
- 3 Choose Type and Restart policy deliberately; add resource limits where needed.
- 4 Validate with `daemon-reload`, `critical-chain`, journal on failure, and document in IaC or MachineConfig.

EVIDENCE YOU WOULD QUOTE

```
systemd-analyze verify /etc/systemd/system/myapp.service

systemd-analyze critical-chain myapp.service
```

```
systemctl cat myapp.service ; journalctl -u myapp -b --no-pager
```

RED FLAG - DO NOT SAY THIS

Do not set `Restart=always` on a service that fails its config check every time — you get a log flood and a missed root cause.

EXPECT THIS FOLLOW-UP

The service must start only after an NFS mount and a specific TLS file exist. How do you model that?

Q10**SENIOR****How do you diagnose and fix file descriptor exhaustion?****SAY THIS**

Symptoms include "too many open files", accept failures on busy sockets, and mysterious 502s under load. I compare the process soft limit from `/proc/<pid>/limits` with how many descriptors it holds — `ls /proc/<pid>/fd | wc -l`. System-wide, `fs.file-nr` in `/proc/sys/fs/file-nr` shows allocated versus maximum. `lsof -p` or `lsof -c` names the leak — often log pipelines, database pools, or forgotten sockets in long-lived workers. Fix is raise `LimitNOFILE` in `systemd` or `/etc/security/limits.d`, tune `fs.file-max` at the kernel, and fix the application leak. For Kubernetes nodes, kubelet and CRI-O defaults matter when pod density is high.

THINK OF IT LIKE

File descriptors are coat-check tickets. You can enlarge the cloakroom — `ulimit` — but if one guest hoards tickets and never returns them, expanding the room only delays the same collapse.

WHY IT MATTERS IN PRODUCTION

This appears in 2025/2026 SRE loops for API gateways, log aggregators, and etcd-adjacent proxies. Interviewers want both the `sysctl` and `systemd` knobs and the insistence on finding the leak. **Saying only "increase `ulimit`" without `/proc/<pid>/fd` is junior.**

STEP BY STEP

- 1 Confirm the error — application log, `errno 24`, or `ss` showing listen drops.
- 2 Compare per-process fd count to limits via `/proc/<pid>/limits` and fd count.
- 3 Check system totals with `fs.file-nr` and `lsof` summaries.
- 4 Raise limits appropriately and fix the leak; add alerting on fd usage percentage.

EVIDENCE YOU WOULD QUOTE

```
cat /proc/<pid>/limits | grep 'open files'

ls /proc/<pid>/fd | wc -l ; lsof -p <pid> | wc -l

cat /proc/sys/fs/file-nr ; sysctl fs.file-max
```

RED FLAG - DO NOT SAY THIS

Do not crank `fs.file-max` to millions without finding which process consumes them. You mask a leak until the node runs out of memory tracking metadata.

EXPECT THIS FOLLOW-UP

Only one pod on the node hits the limit while others are fine. Where is the boundary?

Q11

SENIOR

When would you use an LVM snapshot, and what are the risks?

SAY THIS

An LVM snapshot is a copy-on-write view of a logical volume at a point in time — useful for consistent backups before a risky migration or for short-lived rollback windows. I create it with `lvcreate -L size -s -n snap /dev/vg/lv`, mount read-only if possible, back up with `xfsdump` or `pg_dump` from the snapshot not the live LV, then `lvremove` the snapshot promptly. Risks: the snapshot COW table fills if the source LV changes heavily while the snapshot exists — writes fail or the snapshot becomes invalid. Size snapshots generously or keep them minutes not days. They are not a replacement for replicas; they are a crash-consistent or application-quieted backup aid.

THINK OF IT LIKE

A snapshot is a photocopy of your ledger at noon. Every edit after noon is tracked on a sticky note stack — if the stack overflows, the copy is useless and the books may lock.

WHY IT MATTERS IN PRODUCTION

Database and registry interview loops still mention LVM snapshots alongside `pg_start_backup` and Restic. Platform candidates managing StatefulSets on bare metal need to articulate why snapshots without quiesce are crash-consistent only, and why full thin pools differ from classic snapshots.

STEP BY STEP

- 1 Define COW snapshot behaviour and the point-in-time use case.
- 2 Describe create, mount read-only, backup, remove — short lifetime.
- 3 Warn about snapshot fill under write churn and sizing the COW pool.
- 4 Contrast with application-consistent backup and storage replication for DR.

EVIDENCE YOU WOULD QUOTE

```
lvcreate -L 10G -s -n db_snap /dev/vg/db

lvs -o lv_name,origin,data_percent,snap_percent

mount -o ro /dev/vg/db_snap /mnt && xfsdump -l 0 - /mnt
```

RED FLAG - DO NOT SAY THIS

Do not treat LVM snapshots as free infinite backups. An filled snapshot pool takes down writes on the origin LV.

EXPECT THIS FOLLOW-UP

You need application-consistent backup for PostgreSQL on LVM. Walk through the sequence.

Q12

SENIOR

Which kernel tunables do you adjust for a high-connection Linux server, and how do you persist them on RHEL?

SAY THIS

For connection-heavy workloads I review `net.core.somaxconn` and the application backlog, `net.ipv4.ip_local_port_range` for ephemeral ports, `net.ipv4.tcp_tw_reuse` carefully on modern kernels, `fs.file-max` and per-service `LimitNOFILE`, `vm.swappiness` on databases often tuned toward 1 or 10, and `vm.overcommit_memory` when the workload justifies it — for example Redis documentation. `Conntrack` `nf_conntrack_max` matters on nodes doing NAT. I set values in `/etc/sysctl.d/99-app.conf`, apply with `sysctl --system`, and verify with `sysctl -a` and `/proc`. On OpenShift fleet nodes the same keys belong in `MachineConfig` `kubeletConfig` or tuned profiles — not manual `sysctl` on one box.

THINK OF IT LIKE

Kernel tunables are city traffic rules — speed limits, lane counts, and parking spaces. You can widen the road, but if every driver triple-parks you still gridlock unless the rules match reality.

WHY IT MATTERS IN PRODUCTION

2025/2026 platform interviews tie this to ingress controllers, service meshes, and etcd latency. Candidates should mention tuned-adm profile for throughput versus latency, and that blind copy-paste from blog posts without measuring `ss -s` and `conntrack -S` is how you break TCP behaviour.

STEP BY STEP

- 1 Start from the symptom: port exhaustion, accept queue drops, conntrack full, swap thrash.
- 2 Name relevant sysctl keys and what each actually controls — not magic numbers.
- 3 Persist via `/etc/sysctl.d` and `sysctl --system`; on OpenShift via `MachineConfig`.
- 4 Validate under load and document rollback — tunables are changes with blast radius.

EVIDENCE YOU WOULD QUOTE

```
sysctl net.core.somaxconn net.ipv4.ip_local_port_range fs.file-max
```

```
ss -s ; cat /proc/sys/net/netfilter/nf_conntrack_count
```

```
sysctl --system ; tuned-adm active
```

RED FLAG - DO NOT SAY THIS

Do not recite deprecated tunables like raising `tcp_fin_timeout` without context. Interviewers flag cargo-cult `sysctl` lists from 2012 blog posts.

EXPECT THIS FOLLOW-UP

After raising `somaxconn`, clients still see connection timeouts under load. What do you measure next?

Q13

ARCHITECT

How would you design disk layout and LVM strategy for a stateful database node?**SAY THIS**

I separate concerns onto different LVs or disks: OS root small and boring, data LV for the database on the fastest durable tier, WAL or redo on separate low-latency devices when the engine benefits, logs on their own mount so a log storm cannot fill data, and backups landing on a different VG or object storage — not the same full VG. I align filesystem choice with workload — XFS default on RHEL for most data volumes — size LVs with headroom but snapshot COW in mind, and document extent growth in Terraform or kickstart. RAID level, multipath, and LUKS for data-at-rest sit underneath LVM; the interview is about blast-radius isolation and operational headroom, not one giant /.

THINK OF IT LIKE

Designing disk layout is zoning a factory: raw materials, assembly line, and shipping docks are separate bays so a spill in shipping does not shut production.

WHY IT MATTERS IN PRODUCTION

Architect-level loops ask this for PostgreSQL on bare metal, OpenShift Data Foundation edge cases, and migration from "one volume" VMS. They listen for separating logs from data, monitoring inode and block usage independently, and having a grow path that does not require downtime.

STEP BY STEP

- 1 List workloads and I/O patterns — random versus sequential, latency sensitivity for WAL.
- 2 Map mounts to LVs with failure isolation: data, logs, temp, backup target.
- 3 Choose RAID/multipath/encryption under the PVs; leave VG free space for growth and snapshots.
- 4 Encode in IaC, define alerts at seventy and eighty-five percent, and test restore not just backup.

EVIDENCE YOU WOULD QUOTE

```
lsblk -f ; lvs -o+devices ; df -h

iostat -xz 1 5 # validate device separation under load

cat /etc/fstab ; grep -r lvcreate kickstart/ or terraform/
```

RED FLAG - DO NOT SAY THIS

Do not put database data, logs, and backups on one LV because "LVM can grow later". One full mount stops the database even if another area has space.

EXPECT THIS FOLLOW-UP

You inherit a monolithic /var with PostgreSQL and logs. How do you split without a full rebuild?

Q14

ARCHITECT

How do you manage kernel and sysctl configuration across a fleet without creating snowflakes?

SAY THIS

Single source of truth wins: tuned profiles plus `/etc/sysctl.d` fragments rendered from Git, applied by Ansible or MachineConfig on OpenShift, with node labels selecting which profile applies — GPU nodes versus ingress versus generic workers. I avoid ssh sysctl on individual hosts; drift shows up in compliance scans and surprise reboot behaviour. Changes go through PR, canary nodes, before fleet rollout, with sysctl `--system` idempotency and automated validation comparing sysctl `-a` to expected JSON. Document why each tunable exists — owner, rollback, blast radius — because sysctl debt is harder to audit than package debt. On immutable OSes, if it is not in the rendered config, it did not happen.

THINK OF IT LIKE

Fleet sysctl management is air-traffic rules for a country, not each pilot choosing their own altitude. Local exceptions need a filed flight plan and an expiry date.

WHY IT MATTERS IN PRODUCTION

This separates staff SRE from consultants who fix one node. 2026 interviews connect it to MCO pool semantics, PerformanceProfile for low latency, and compliance tools flagging unexpected `/proc/sys` values. The architect answer mentions observability — track conntrack usage and OOM events after fleet-wide changes.

STEP BY STEP

- 1 Declare principle: declarative, versioned, node-class scoped configuration.
- 2 Pick mechanisms: tuned, sysctl.d, MachineConfig kubeletConfig or custom MC.
- 3 Describe change flow: PR, canary pool, metrics watch, fleet promote, rollback path.
- 4 Add drift detection and documentation — no unnamed magic numbers in production.

EVIDENCE YOU WOULD QUOTE

```
oc get mc,pp -o wide # OpenShift path

ansible-playbook sysctl.yml --check --diff

sysctl -a | sha256sum # compare canary versus baseline
```

RED FLAG - DO NOT SAY THIS

Do not describe "we have a wiki page of sysctl settings" as fleet management. Wikis do not survive the first hire-and-forget emergency ssh session.

EXPECT THIS FOLLOW-UP

One node class needs a vendor-required kernel module and tunable the others must not have. How do you model that?

PART 2 · Q15-Q24 · 10 QUESTIONS

Bash and shell scripting for SRE

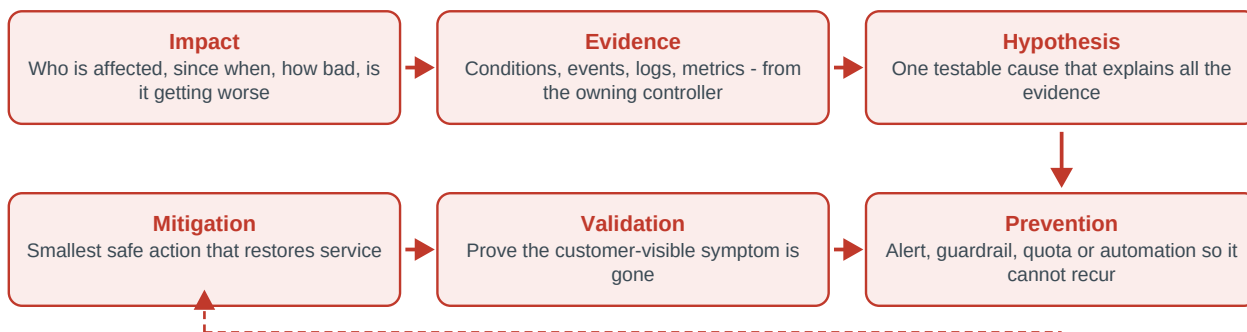
set -euo pipefail, traps, and the glue code that outlives every dashboard

Every platform team owns a graveyard of bash scripts — backup wrappers, release helpers, on-call runbooks frozen as code. Interviews test whether yours are safe under failure or whether they double-charge customers at 3 a.m. because pipefail was missing. Senior SRE scripting is not clever one-liners; it is strict mode, explicit error handling, structured logging, and choosing systemd timers over cron when ordering and journal integration matter. This part covers the bash depth hiring managers still probe because Ansible and Python do not replace the five-line fix at the shell.

Level mix - Foundation: 3 · Intermediate: 3 · Senior: 3 · Architect: 1

The evidence-first loop - use this for every troubleshooting question

Say these six words out loud before you say anything technical. Candidates who jump straight to "I would restart the pod" lose the point even when restarting works.

**Q15****FOUNDATION**

What does set -euo pipefail do, and why should production scripts use it?

SAY THIS

set -e exits when a command returns non-zero — no silent fall-through. set -u treats unset variables as errors, catching typos like \$SERVR instead of \$SERVER. set -o pipefail makes a pipeline fail if any stage fails, not just the last one — so curl ... | jq ... fails when curl fails. Together they turn bash from permissive glue into something that stops on the first lie. I put them at the top of every script after the shebang, sometimes with a short comment so the next editor does not remove them. Interviewers know most outages from shell scripts come from one missing flag.

THINK OF IT LIKE

Without strict mode, bash is a car with the check-engine light taped over. With it, the car pulls over at the first odd noise instead of driving until the engine falls out.

WHY IT MATTERS IN PRODUCTION

This is the most common bash screening question for SRE roles in 2025/2026. Real incidents: a migration script continued after rsync failed and deleted the wrong tree; a deploy helper with unset VERSION pushed latest everywhere. Pipelines without pipefail hide curl 404s when jq happily parses an error HTML page.

STEP BY STEP

- 1 Define each flag: -e command failure, -u unset vars, pipefail pipeline failure.
- 2 Explain why pipelines without pipefail lie — only the last exit status counts by default.
- 3 Show placement: `#!/usr/bin/env bash` then `set -euo pipefail` near the top.
- 4 Mention sensible exceptions — some commands expect non-zero and need `|| true` or `if ! cmd`.

EVIDENCE YOU WOULD QUOTE

```
bash -c 'set -e; false; echo unreachable'

bash -c 'set -u; echo $UNDEFINED'

bash -c 'set -o pipefail; false | true; echo exit:$?'
```

RED FLAG - DO NOT SAY THIS

Do not say "I never use set -e because it breaks my script". That signals you have not learned the conditional patterns to handle expected failures safely.

EXPECT THIS FOLLOW-UP

You need a command that may legitimately fail. How do you handle it without disabling -e globally?

Q16

FOUNDATION

Explain useful bash parameter expansion for defensive scripting.

SAY THIS

Parameter expansion lets scripts handle missing or messy input without sprawling if blocks. `${VAR:-default}` uses default when unset or empty; `${VAR:?message}` aborts with an error — great after `set -u`. `${VAR#pattern}` and `${VAR##pattern}` strip prefixes; `${VAR%/}` removes a trailing slash safely. `${#VAR}` is length. For paths I use `${VAR%/}` before joining so I do not double slashes. In production I combine `${VAR:?}` for required env vars with `:-` for optional knobs. Interviewers listen for whether you know the difference between `:-` and `:=` — the latter assigns the default.

THINK OF IT LIKE

Parameter expansion is form validation on a web field — default placeholders for optional boxes, hard stops on required ones — before the data reaches the database.

WHY IT MATTERS IN PRODUCTION

SRE scripts consume environment variables from CI, Vault exports, and Kubernetes downward API. Typos in `NAMESPACE` or `ENDPOINT_URL` cause wide blast radius. Defensive expansion is cheaper than debugging a loop that ran against production because `$ENV` was empty and `:- prod` was wrong.

STEP BY STEP

- 1 Cover defaults and required vars: `:-` versus `?:` with a concrete env var example.
- 2 Show string cleanup: `# ## % %%` for paths and suffixes.
- 3 Mention `:=` when you want to assign a default once and reuse it.
- 4 Tie to `set -u` — expansion patterns complement strict mode rather than fight it.

EVIDENCE YOU WOULD QUOTE

```
NAMESPACE=${NAMESPACE:?set NAMESPACE}; echo $NAMESPACE

BASE=${BASE:-https://localhost:8080}; echo ${BASE%/}/health

FILE=/var/log/app.log; echo ${FILE##*/}
```

RED FLAG - DO NOT SAY THIS

Do not use unquoted \$VAR everywhere. Expansion does not fix word-splitting; "\${VAR}" still matters.

EXPECT THIS FOLLOW-UP

How do you validate that ENDPOINT_URL is HTTPS and non-empty before curl uses it?

Q17**FOUNDATION****How do you write and use bash functions in maintainable scripts?****SAY THIS**

Functions group reusable logic with local variables and explicit return codes. I declare locals with local inside the function so I do not clobber globals. Return 0 or 1 for success checks — or echo structured output and capture with "\$()" when the caller needs data. Naming is verb-noun: log_info, require_root, wait_for_port. For SRE scripts I keep functions small enough to test in isolation by sourcing the file with bash -c 'source lib.sh; wait_for_port localhost 8080'. Passing arrays to functions needs nameref — declare -n — in bash 4.4 plus. The interview point is structure: main at the bottom, library functions above, no three-hundred-line flat scripts.

THINK OF IT LIKE

Functions are tools on a pegboard — each one does one job, returns to the same place, and does not leave shavings all over the workbench.

WHY IT MATTERS IN PRODUCTION

Platform repos mix shell and Python; bash functions wrap kubectl, oc, and aws with consistent logging and retry. Interviewers dislike copy-pasted blocks; they like a log() function that timestamps to stderr while results go to stdout for piping.

STEP BY STEP

- 1 Show basic shape: myfunc() { local x=\$1; ...; return 0; }.
- 2 Explain local, return codes, and capturing output with "\$()".
- 3 Separate logging on stderr from machine-readable stdout.
- 4 Mention sourcing a shared lib.sh and guarding with if [[\${BASH_SOURCE[0]} != "\$0"]].

EVIDENCE YOU WOULD QUOTE

```
log() { printf "[%s] %s\n" "$(date -Is)" "$*" >&2; }

wait_for_port() { local host=$1 port=$2; ...; }

bash -c 'source ./lib.sh; wait_for_port 127.0.0.1 22'
```

RED FLAG - DO NOT SAY THIS

Do not use the function keyword unless you need it for historical compatibility — myfunc() is portable and idiomatic in modern bash.

EXPECT THIS FOLLOW-UP

How do you pass an array of hostnames into a function and iterate safely?

Q18

INTERMEDIATE

How do traps help you write scripts that clean up on exit or signals?

SAY THIS

trap registers a handler for signals or EXIT. I almost always trap cleanup on EXIT so temp dirs vanish whether the script succeeds or set -e aborts — `trap 'rm -rf "$tmpdir"' EXIT`. For INT and TERM I trap to a function that logs, kills child processes, and exits 130 so orchestrators know it was interrupted. Quote the handler carefully — expand variables at trap time versus execution time trips people up; use single quotes on EXIT with variables defined before trap, or a function. In long-running SRE jobs, trap prevents leaving stale lock files or half-finished database exports when someone hits Ctrl-C.

THINK OF IT LIKE

A trap is the "close the door behind you" sign — whether you leave happily or the fire alarm goes off, the door still gets closed.

WHY IT MATTERS IN PRODUCTION

Cron and CI jobs that create `/tmp/work.$$` directories without traps fill disks. Kubernetes preStop hooks are different, but host-level backup scripts that trap TERM behave correctly under systemd stop timeout. This is a favourite live-coding prompt: add a trap to an existing messy script.

STEP BY STEP

- 1 Explain trap COMMAND SIGNAL and that EXIT fires on any shell exit path.
- 2 Show tmpdir pattern: `tmpdir=$(mktemp -d); trap 'rm -rf "$tmpdir"' EXIT`.
- 3 Handle INT/TERM with a cleanup function that reaps children.
- 4 Warn about quoting and overwriting traps — use `trap - EXIT` to remove when done.

EVIDENCE YOU WOULD QUOTE

```
tmpdir=$(mktemp -d); trap 'rm -rf "$tmpdir"' EXIT

cleanup() { kill "$(jobs -p)" 2>/dev/null; rm -f /var/lock/myjob.lock; }; trap cleanup EXIT INT TERM

bash -x ./script.sh # verify trap runs on failure path
```

RED FLAG - DO NOT SAY THIS

Do not trap without quoting paths — `rm -rf $tmpdir` without quotes is how you `rm -rf /` when tmpdir is empty.

EXPECT THIS FOLLOW-UP

Your script acquires a flock lock file. How do traps interact with the lock on failure?

Q19

INTERMEDIATE

How do you debug a bash script that fails only in production?

SAY THIS

I reproduce locally with the same env vars and bash version — bash --version matters on RHEL 8 versus 9. bash -n checks syntax without running. set -x enables trace to stderr; I wrap it with PS4='+\${BASH_SOURCE}:\${LINENO}: ' for readable lines. shellcheck catches quoting and portability issues in CI. For prod-only failures I log set -x output to a file temporarily, compare PATH, umask, and cwd, and check whether cron or systemd runs a minimal environment without variables CI sets. strace -f -e trace=file,process on a canary host shows missing files or permission denials bash errors swallow. The interview arc is systematic narrowing, not sprinkling echo debug forever.

THINK OF IT LIKE

Debugging bash is diagnosing a car that only stalls on cold mornings — you need the same temperature and fuel, not just a test drive on a sunny afternoon.

WHY IT MATTERS IN PRODUCTION

2025/2026 SRE interviews often give a broken deploy.sh. Production-only bugs usually mean environment differences, race conditions, or parallel execution — flock and set -e interactions. Candidates who mention shellcheck and PS4 stand out from those who only say "add echo statements".

STEP BY STEP

- 1 Reproduce: match bash version, env, cwd, and invoking user or unit file.
- 2 Static check: bash -n and shellcheck in CI.
- 3 Dynamic trace: PS4 and set -x scoped to the failing section.
- 4 System-level: strace or auditd when the script insists the file does not exist.

EVIDENCE YOU WOULD QUOTE

```
shellcheck -x deploy.sh
```

```
PS4='+${BASH_SOURCE}:${LINENO}: '; bash -x deploy.sh 2>trace.log
```

```
bash -n deploy.sh ; strace -f -e trace=openat,execve ./deploy.sh 2>&1 | tail -30
```

RED FLAG - DO NOT SAY THIS

Do not leave set -x enabled in production logs long term — credentials in arguments leak into journald and Splunk.

EXPECT THIS FOLLOW-UP

The script succeeds when run manually but fails under systemd. What differs in the environment?

Q20

INTERMEDIATE

When do you choose cron versus a systemd timer for scheduled work?

SAY THIS

cron is fine for simple fixed schedules on a single host — log rotation reminders, weekly reports. systemd timers win when I need dependency ordering — `After=network-online.target` — persistent catch-up if the host was down, journal integration, resource limits, and `RandomizedDelaySec` to desynchronize fleet thundering herds. Timers pair with a service unit so `TimeoutStartSec`, `User=`, and `EnvironmentFile` are first-class. On RHEL 9 I default new platform automation to timer plus service unless the job is trivial. In interviews I also mention that cron's minimal environment causes half of "works manually, fails at night" tickets.

THINK OF IT LIKE

cron is an alarm clock — it rings at seven whether or not you have power. A systemd timer is a smart alarm tied to the house alarm system — it waits until the network is up and logs why it skipped.

WHY IT MATTERS IN PRODUCTION

Fleet SRE teams replace crontab sprawl with packaged timer units in RPM or Ansible. Kubernetes CronJob is a separate layer; the question targets nodes and jump hosts. Interviewers want `OnCalendar` versus `Monotonic`, `Persistent=true`, and why you `systemctl list-timers --all` to audit.

STEP BY STEP

- 1 State cron strength: simple, universal, quick one-liners.
- 2 State timer strengths: deps, journaling, limits, missed-run catch-up, fleet jitter.
- 3 Show unit pair: `foo.timer` triggers `foo.service` with `[Timer]` settings.
- 4 Mention auditing: `list-timers`, enable timer not just service, avoid duplicate cron and timer.

EVIDENCE YOU WOULD QUOTE

```
systemctl list-timers --all

systemctl cat backup.timer backup.service

journalctl -u backup.service -b --since today
```

RED FLAG - DO NOT SAY THIS

Do not say cron is "deprecated". It is not — but pretending cron and systemd timers are identical shows you have not operated both in production.

EXPECT THIS FOLLOW-UP

You need a job to run at most once across three redundant hosts. How would you design that in shell/systemd?

Q21

SENIOR

What patterns make bash error handling production-safe beyond set -e?

SAY THIS

Strict mode is the floor, not the ceiling. I check critical commands explicitly: `if ! cp "$src" "$dst"; then log and exit 1; fi`. For pipelines I use `if ! output=$(curl -fsS ... | jq -e .); then ...`. For retries I wrap with a small loop and exponential backoff rather than ignoring failure. `errexit` and `pipefail` still allow `if cmd; then and cmd || true` when intentional. I define a `die()` function that logs to `stderr` and exits with a chosen code. Scripts return distinct exit codes — 2 for usage, 3 for dependency missing — so CI and on-call playbooks branch correctly. Finally I avoid subshell surprises: `( set +e; ... )` does not affect parent `errexit` but can hide failures if mishandled.

THINK OF IT LIKE

set -e is the seatbelt. Explicit if checks, retries, and die() are airbags and crumple zones — different failures need different responses.

WHY IT MATTERS IN PRODUCTION

Senior SRE loops ask you to harden a naive script that rsyncs then deletes source. The expected answer includes verifying destination, dry-run flags, and atomic rename patterns. Financial and healthcare clients audit exit codes in runbooks.

STEP BY STEP

- 1 Combine `set -euo pipefail` with explicit `if !` for high-stakes commands.
- 2 Use `die/log` helpers and meaningful exit codes documented in the script header.
- 3 Handle retries without disabling `errexit` globally.
- 4 Watch subshells, pipelines in `if`, and temporary `set +e` blocks with comments why.

EVIDENCE YOU WOULD QUOTE

```
die() { log "ERROR: $" ">&2; exit "${2:-1}"; }

if ! rsync -a --dry-run "$src/" "$dst/"; then die "dry-run failed"; fi

for i in 1 2 3; do curl -fsS --retry 0 "$url" && break; sleep $((i*i)); done
```

RED FLAG - DO NOT SAY THIS

Do not wrap entire scripts in `set +e ... set -e` to "handle errors". You end up with a script that exits zero after partial failure.

EXPECT THIS FOLLOW-UP

How do you structure a script that should fail the CI job but still upload logs on failure?

Q22

SENIOR

How do you write bash scripts that are safe for automation and idempotent operations?

SAY THIS

Idempotent scripts produce the same end state when run twice — critical for Ansible-style loops and retrying CI. I check before mutate: `[[ -f "$marker" ]] && exit 0`, or test whether the user exists before `useradd`. `mkdir -p`, `install -d`, and `rsync --delete-after` with care are idempotent primitives. I use `flock` or atomic `mkdir` for `lockdir` so two cron invocations do not race. Logging includes run id, hostname, and who invoked it. Dry-run flags — `--dry-run`, echo before `rm` — are mandatory for destructive steps. Secrets come from `env` or files with `0600`, never `argv`. For automation I document inputs, outputs, side effects, and exit codes in a header comment so the next engineer and the orchestrator treat it like a small API.

THINK OF IT LIKE

An idempotent script is a light switch — flipping it twice leaves the room in the same state, not brighter and brighter until the bulb explodes.

WHY IT MATTERS IN PRODUCTION

Platform teams invoke bash from Tekton, GitHub Actions, and AWS. Non-idempotent scripts cause double provisioning, duplicate firewall rules, and torn partial state. 2026 interviews connect this to Terraform apply semantics — scripts should behave the same way.

STEP BY STEP

- 1 Define desired end state and check if already true before destructive actions.
- 2 Use locking — `flock` — and temp files with `mktemp` for safe concurrent runs.
- 3 Provide `--dry-run` or `VERBOSE=1` paths for operators to preview changes.
- 4 Document contract: env vars, exit codes, idempotency guarantees in script header.

EVIDENCE YOU WOULD QUOTE

```
flock -n /var/lock/myjob.lock -c './myjob.sh' || exit 0

id myuser &>/dev/null || useradd myuser

install -d -m 0750 /var/lib/myapp
```

RED FLAG - DO NOT SAY THIS

Do not append to config files on every run without deduplication — you get seventeen identical `Include` lines and a parser failure months later.

EXPECT THIS FOLLOW-UP

The script must update a line in `/etc/sysctl.d` only if the value differs. How do you implement that?

Q23

SENIOR

What advanced parameter expansion and bash features help SRE automation at scale?

SAY THIS

Arrays beat repeated strings: `hosts=(web1 web2 web3); for h in "${hosts[@]}"; do ...` `nameref declare -n ref=$1` passes array names by reference in bash 4.4 plus. `${var/pattern/replacement}` does inline replace; `${!prefix*}` lists indirect variable names for config profiles. `readarray -t mapfile` reads lines into an array without word-splitting pain. Brace expansion `{1..10}` and `printf` formatting beat `seq` in subshells. For parallel SSH I use `xargs -P` or GNU `parallel` with a quoted export of helper functions. Process substitution `diff <(sort a) <(sort b)` helps config drift checks. The senior bar is knowing when bash should stop and Python or Go should start — bash for glue, not for fifty-host orchestration logic.

THINK OF IT LIKE

Advanced bash is power tools in a workshop — faster for the right job, but you still should not build the whole house with a angle grinder.

WHY IT MATTERS IN PRODUCTION

Interviewers test array quoting because for `i` in `${arr[@]}` without quotes breaks on hostnames with spaces — rare but fatal. Fleet scripts parsing `oc get -o json` sometimes belong in `jq` or Python; knowing `readarray` and `jq -r @sh` shows mature judgment.

STEP BY STEP

- 1 Demonstrate arrays: `"${arr[@]}"` quoting and `mapfile` for line-safe input.
- 2 Show `nameref` or passing arrays to functions without global pollution.
- 3 Use parameter expansion for string ops before reaching for `sed`.
- 4 State the ceiling: complex JSON, API clients, and unit tests → Python or Go.

EVIDENCE YOU WOULD QUOTE

```
readarray -t lines < <(kubectl get nodes -o jsonpath='{.items[*].metadata.name}')

declare -n ref=$1; for item in "${ref[@]}"; do echo "$item"; done

diff <(sort /etc/hosts) <(sort hosts.golden)
```

RED FLAG - DO NOT SAY THIS

Do not parse JSON with `grep` and `awk` in new automation — interviewers file that under "will maintain poorly" unless you immediately justify `jq`.

EXPECT THIS FOLLOW-UP

You need to run a function against two hundred hosts in parallel with a concurrency cap. Sketch the approach.

Q24

ARCHITECT

How would you structure a reusable shell library for a platform team's automation?**SAY THIS**

I treat it like a tiny SDK: lib/log.sh, lib/http.sh, lib/k8s.sh sourced by thin entrypoint scripts. Each lib file is idempotent on source — guard with `[[ -n ${_LIB_LOG_LOADED:-} ]] && return; export _LIB_LOG_LOADED=1`. Public functions are verb-noun; internals prefixed with underscore. Strict mode lives in lib/init.sh. Version pin in git tags; consumers source by absolute path from repo root discovered via `git rev-parse --show-toplevel`. Tests use bats-core or shunit2 in CI with shellcheck and bash -n. Documentation lists env contract, exit codes, and examples. Packaging as an RPM or installing to /opt/platform/lib makes jump hosts consistent. The architect point: bash libraries scale until they do not — the library boundary is where Python modules or a CLI binary take over for anything needing structured config and typed errors.

THINK OF IT LIKE

A shell library is the shared toolbox in a fire truck — every crew grabs the same wrench; nobody carries a custom one that only fits their hand.

WHY IT MATTERS IN PRODUCTION

Staff-level platform interviews ask how ten engineers avoid ten slightly different oc wrapper scripts. The answer includes semver, code review, CI tests, and deprecation policy. Teams that failed this have /usr/local/bin chaos and scripts that source random GitHub gists.

STEP BY STEP

- 1 Split by concern: logging, retries, cloud or oc wrappers, lock helpers — thin scripts on top.
- 2 Enforce strict mode, shellcheck, bats tests, and semver tags in CI.
- 3 Install via RPM, git submodule, or /opt with a single PLATFORM_LIB root env var.
- 4 Define when to graduate a module to Python or Go — complexity threshold and testability.

EVIDENCE YOU WOULD QUOTE

```
shellcheck -x lib/*.sh ; bats tests/

source "$(git rev-parse --show-toplevel)/lib/init.sh"

rpm -ql platform-bash-lib | head
```

RED FLAG - DO NOT SAY THIS

Do not advocate `curl | bash` for internal libraries. That pattern is how supply-chain nightmares and undebuggable drift enter platform teams.

EXPECT THIS FOLLOW-UP

Two teams need conflicting behaviour in the same library function. How do you version and migrate safely?

PART 3 · Q25-Q36 · 12 QUESTIONS

Python for SRE and platform engineers

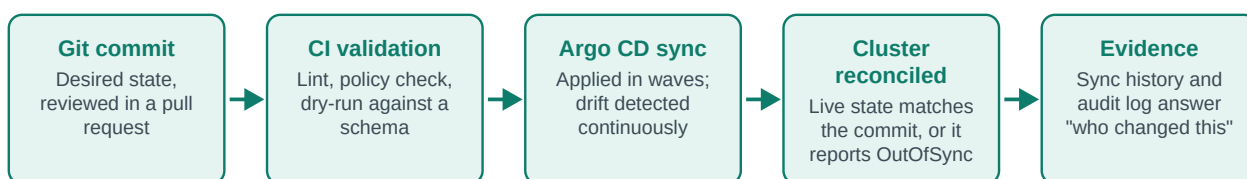
The language platform teams actually write production automation in

Python questions for platform roles are not about memorising syntax. Interviewers want to know whether you treat automation as production code: how you call APIs safely, handle failures, test behaviour, manage dependencies and build tools other engineers can run without reading your laptop's mind. **The best answers connect Python patterns to operational concerns - retries, idempotency, observability and least privilege - rather than listing libraries.**

Level mix - Foundation: 4 · Intermediate: 5 · Senior: 2 · Architect: 1

Change control that survives an audit

The point of GitOps is not the tool, it is that every cluster change has an author, a reviewer, a timestamp and a revert path.

**Q25****FOUNDATION**

How do you use pathlib instead of os.path for file operations?

SAY THIS

pathlib.Path represents filesystem paths as objects with methods for joining, reading, writing and iterating. Instead of `os.path.join` and string concatenation, you use the slash operator or `joinpath`, which handles separators correctly on every platform. It reads cleaner, fails less often on edge cases like trailing slashes, and makes recursive directory walks and globbing explicit rather than buried in nested `os` calls.

THINK OF IT LIKE

os.path is giving someone directions as a string of street names. pathlib is handing them a map with buttons for "go here" and "list what's inside".

WHY IT MATTERS IN PRODUCTION

Platform scripts constantly touch config files, certificate paths, inventory directories and log locations. `pathlib` reduces the class of bugs where a missing slash or a relative path resolves differently depending on where the script was launched. **It also composes well with `open()`, `shutil` and `subprocess` when you pass `.as_posix()` only where a string is required.**

STEP BY STEP

- 1 Prefer Path objects from the start rather than converting strings back and forth.
- 2 Use `/` or `joinpath` for composition; use `.exists()`, `.is_file()` and `.glob()` for inspection.
- 3 Read and write with `.read_text()`, `.write_text()` or `open(path)` for large files.
- 4 Resolve paths with `.resolve()` when you need an absolute, canonical location.

EVIDENCE YOU WOULD QUOTE

```
python3 -c "from pathlib import Path; p=Path('/etc')/'ansible'/'ansible.cfg'; print(p.exists(), p.resolve())"

grep -r 'from pathlib import Path' automation/ | head

python3 -m py_compile scripts/check_certs.py
```

RED FLAG - DO NOT SAY THIS

Do not build paths with f-strings and hard-coded slashes. It breaks the moment someone runs the script on Windows or from a different working directory.

EXPECT THIS FOLLOW-UP

Your script works when run from /opt/automation but fails from cron. What did pathlib miss?

Q26**FOUNDATION****How do you manage Python dependencies with virtual environments?****SAY THIS**

A virtual environment is an isolated directory containing a Python interpreter and its own site-packages, so project dependencies do not pollute the system Python or conflict with other projects. I create one per project or per automation repo, activate it for local development, pin exact versions in a requirements file or lockfile, and install from that file in CI and production images so every environment gets the same dependency set.

THINK OF IT LIKE

It is a labelled toolbox for one job site. The tools stay together, nobody borrows your specialist spanner for another project, and you can hand the whole box to someone else.

WHY IT MATTERS IN PRODUCTION

On RHEL and OpenShift control nodes, system Python is managed by the OS and should not be modified. Platform engineers who pip install globally create drift that is impossible to reproduce. Virtual environments are the minimum bar; in production automation the pattern graduates to container images or execution environments where the venv is baked in and versioned.

STEP BY STEP

- 1 Create an isolated environment with `python3 -m venv .venv` and activate it for local work.
- 2 Install dependencies and freeze exact versions: `pip freeze > requirements.txt`.
- 3 Install from the requirements file in CI and in container builds, never ad hoc.
- 4 Document the Python version alongside dependencies so upgrades are deliberate.

EVIDENCE YOU WOULD QUOTE

```
python3 -m venv .venv && . .venv/bin/activate && pip install -r requirements.txt

pip freeze | diff - requirements.txt

grep -E 'python|requirements' Dockerfile .gitlab-ci.yml
```

RED FLAG - DO NOT SAY THIS

Do not pip install packages as root on a shared control node. You have changed everyone's Python and nobody knows what version anything is.

EXPECT THIS FOLLOW-UP

Two engineers get different results from the same script. Where do you look first?

Q27

FOUNDATION

How should platform automation use the logging module instead of print?

SAY THIS

The logging module gives you levels, timestamps, structured context and configurable destinations without rewriting code. I configure a module-level logger, use DEBUG for verbose troubleshooting, INFO for normal progress, WARNING for recoverable issues and ERROR for failures that need attention. Print statements disappear into cron mail nobody reads; logs go to stdout for container platforms, to files with rotation on VMs, or to a central collector when integrated with the observability stack.

THINK OF IT LIKE

Print is shouting in a busy room. Logging is writing in a lab notebook with dated entries and severity labels that someone can search later.

WHY IT MATTERS IN PRODUCTION

Automation that runs unattended must be diagnosable from its output alone. When a nightly certificate check fails at 3am, the on-call engineer needs the cluster name, the API endpoint and the HTTP status - not a traceback with no context. Structured logging with extra fields, or JSON log format in containerised jobs, makes that correlation possible in Loki or Elasticsearch.

STEP BY STEP

- 1 Use logging.getLogger(__name__) in each module rather than configuring at import time.
- 2 Configure handlers and format once in main(), including level from an environment variable.
- 3 Log context with every message: cluster, namespace, resource name, correlation ID.
- 4 Reserve print for truly interactive CLI output that a human reads directly.

EVIDENCE YOU WOULD QUOTE

```
grep -r 'logging.getLogger' automation/ | head

oc logs job/cert-check-nightly --tail=50

LOG_LEVEL=DEBUG python3 scripts/sync_inventory.py 2>&1 | head
```

RED FLAG - DO NOT SAY THIS

Do not catch an exception and print(e) without logging the traceback. You have hidden the one line that explains the failure.

EXPECT THIS FOLLOW-UP

Your CronJob logs are empty but the script exits 1. What logging mistake did you make?

Q28

FOUNDATION

How do you build CLI tools with argparse or click?

SAY THIS

argparse, in the standard library, turns a script into a tool with named arguments, defaults, help text and validation - sufficient for most internal tools with a handful of flags. click adds nested subcommands, shell completion, environment variable fallbacks and CliRunner for tests, which pays off when multiple teams extend the tool over time. Both should exit with meaningful codes, expose --help, and validate input before touching production systems.

THINK OF IT LIKE

argparse is a control panel with labelled switches. click is the same panel mounted in a reception desk when strangers need to use it daily without training.

WHY IT MATTERS IN PRODUCTION

Platform teams ship inventory syncs, certificate checks and report generators. Start with argparse to avoid extra dependencies; migrate to click when subcommands multiply. Expose the finished tool via console_scripts in pyproject.toml so pip install puts it on PATH consistently across engineer laptops and execution environments.

STEP BY STEP

- 1 Use argparse ArgumentParser or click @group decorators with typed options and --help.
- 2 Add subcommands when the tool performs distinct operations like check, sync and report.
- 3 Validate inputs early with type=, choices= or click callbacks before side effects.
- 4 Return non-zero exit codes on failure and test CLI behaviour with CliRunner or subprocess.

EVIDENCE YOU WOULD QUOTE

```
python3 scripts/cluster_report.py --help

grep -A5 'console_scripts' pyproject.toml

pytest tests/test_cli.py -v
```

RED FLAG - DO NOT SAY THIS

Do not parse sys.argv manually with string splits. The first new flag breaks it and nobody gets --help.

EXPECT THIS FOLLOW-UP

How would you package and distribute this CLI to fifty engineers?

Q29**INTERMEDIATE****How do you call HTTP APIs safely with the requests library?****SAY THIS**

I wrap calls in a session for connection reuse, set explicit timeouts on every request, verify TLS unless there is a documented exception, handle non-2xx responses with raise_for_status(), and retry transient failures with backoff rather than hammering the API. Authentication goes in headers or the session, never hard-coded in the URL. For platform work I also log the method, URL, status and latency so failures are traceable.

THINK OF IT LIKE

It is placing a phone call with a time limit and a retry policy, not shouting into a dead line until something breaks.

WHY IT MATTERS IN PRODUCTION

Platform automation talks to Kubernetes APIs, cloud control planes, ticketing systems and internal REST services. The requests library is fine for synchronous calls; the failure modes are timeouts, rate limits, certificate issues and ambiguous 5xx responses. Production code treats every call as unreliable and plans for partial failure when calling many endpoints in a loop.

STEP BY STEP

- 1 Create a requests.Session with default headers, auth and timeout tuple.
- 2 Call raise_for_status() and catch requests.HTTPError with useful logging.
- 3 Retry idempotent methods on connection errors and 429/503 with exponential backoff.
- 4 Never disable TLS verification in production without a named, reviewed exception.

EVIDENCE YOU WOULD QUOTE

```
python3 -c "import requests; r=requests.get('https://api.example.com/health', timeout=(3,10));
r.raise_for_status(); print(r.status_code)"
```

```
grep -r 'requests.Session' automation/ | head
```

```
pytest tests/test_api_client.py -v -k timeout
```

RED FLAG - DO NOT SAY THIS

Do not use `requests.get(url)` with no timeout. It hangs forever when the API is sick and blocks your entire pipeline.

EXPECT THIS FOLLOW-UP

The API returns 429 with a `Retry-After` header. How do you handle that?

Q30

INTERMEDIATE

How do you run external commands with subprocess safely?

SAY THIS

`subprocess.run` with a list of arguments, never `shell=True` unless you have no alternative, captures output and return codes explicitly. I check `returncode` or use `check=True` for fail-fast behaviour, pass a timeout to prevent hung processes, and set `cwd` and `env` deliberately rather than inheriting whatever the parent had. For long-running commands I stream stdout line by line instead of buffering everything into memory.

THINK OF IT LIKE

`shell=True` is asking a stranger to interpret your instructions. A list of arguments is handing them a numbered checklist they execute literally.

WHY IT MATTERS IN PRODUCTION

Platform scripts often wrap `oc`, `kubectl`, `aws`, `ansible-playbook` or `openssl`. The classic incident is shell injection through an unquoted variable, or a subprocess that inherits the wrong `KUBECONFIG` and mutates production. Wrapping CLI tools is acceptable when no API exists, but the wrapper must validate context before calling and parse output defensively.

STEP BY STEP

- 1 Build commands as lists: `['oc', 'get', 'pods', '-n', namespace]`, not formatted strings.
- 2 Use `subprocess.run(..., capture_output=True, text=True, timeout=300, check=True)`.
- 3 Validate `returncode` and `stderr`; log the exact command at INFO for auditability.
- 4 For streaming output, use `Popen` with line iteration and explicit wait with timeout.

EVIDENCE YOU WOULD QUOTE

```
python3 -c "import subprocess; subprocess.run(['oc', 'whoami', '--show-server'], check=True,
text=True, capture_output=True)"
```

```
grep -r 'shell=True' automation/ # should be empty or heavily justified
```

```
bandit -r automation/ -ll | grep subprocess
```

RED FLAG - DO NOT SAY THIS

Do not use `shell=True` with user-supplied input. You have built a remote code execution hole in your automation.

EXPECT THIS FOLLOW-UP

You need to run `oc` with a specific kubeconfig. How do you pass that safely?

Q31**INTERMEDIATE****How do you parse YAML and JSON configuration in automation scripts?****SAY THIS**

Use the `json` module for JSON and PyYAML's `safe_load` for YAML - never `yaml.load` without `Loader=SafeLoader`, which can execute arbitrary code. I validate the parsed structure against expected keys and types before using it, fail with a clear error naming the file and field, and treat config as data separate from code so the same script works across environments by swapping input files or environment-specific paths.

THINK OF IT LIKE

Parsing config is reading the recipe, not guessing ingredients. Safe loading means the recipe cannot rewrite the kitchen.

WHY IT MATTERS IN PRODUCTION

Platform automation reads Ansible inventories, Kubernetes manifests, CI variables files and cloud API responses. YAML's flexibility is a footgun: implicit typing, merge keys and anchors make two files that look equivalent parse differently. For critical config, schema validation with `pydantic` or `jsonschema` catches mistakes at startup rather than mid-incident when a missing key becomes a `NoneType` error deep in the call stack.

STEP BY STEP

- 1 Load YAML with `yaml.safe_load` and JSON with `json.load`; specify encoding explicitly.
- 2 Validate required keys and types immediately after parsing.
- 3 Separate config file paths from logic via CLI flags or environment variables.
- 4 Round-trip test: dump and reload should preserve semantics for generated config.

EVIDENCE YOU WOULD QUOTE

```
python3 -c "import yaml; print(yaml.safe_load(open('inventory/group_vars/all.yml')))"

python3 -c "import json; json.load(open('config/clusters.json'))"

pytest tests/test_config_loader.py -v
```

RED FLAG - DO NOT SAY THIS

Do not use `yaml.load` on files from Git without `SafeLoader`. A malicious merge request can execute code when your CI parses it.

EXPECT THIS FOLLOW-UP

Your YAML parses but `replica_count` is the string `'3'` instead of an integer. How do you catch that?

Q32

INTERMEDIATE

How do you test platform automation with pytest?

SAY THIS

pytest discovers test functions by naming convention, runs them with readable failure output, and supports fixtures for shared setup like mock API responses or temporary directories. I unit-test pure functions with parametrised cases, mock external calls with unittest.mock or pytest-mock so tests do not need a live cluster, and keep integration tests separate and clearly marked for CI jobs that have credentials. The bar is that a pull request cannot merge if tests fail.

THINK OF IT LIKE

It is a rehearsal with stunt doubles for the expensive scenes. You verify the script's decisions without renting the whole theatre every time.

WHY IT MATTERS IN PRODUCTION

Untested automation is a liability: it runs when you are asleep and breaks in ways reviews do not catch. pytest's fixture model makes it practical to test error paths - what happens when the API returns 503, when a file is missing, when JSON is malformed. For platform code, the highest-value tests are often around parsing, retry logic and idempotency checks rather than full end-to-end cluster operations.

STEP BY STEP

- 1 Structure code so business logic lives in importable functions, not only in main().
- 2 Use @pytest.mark.parametrize for table-driven cases covering edge inputs.
- 3 Mock requests, subprocess and boto3 clients in unit tests; reserve live calls for integration.
- 4 Run pytest in CI on every commit with coverage thresholds on critical modules.

EVIDENCE YOU WOULD QUOTE

```
pytest tests/ -v --tb=short

pytest tests/ --cov=automation --cov-report=term-missing

grep -r '@pytest.fixture' tests/ | head
```

RED FLAG - DO NOT SAY THIS

Do not skip tests because the script is small. Small scripts cause large incidents when they touch production at 2am.

EXPECT THIS FOLLOW-UP

How do you test code that wraps oc without a real cluster?

Q33

INTERMEDIATE

How do you use concurrent.futures for parallel platform tasks?

SAY THIS

concurrent.futures provides ThreadPoolExecutor for I/O-bound work like HTTP calls and ProcessPoolExecutor for CPU-bound work. I submit tasks with executor.submit or map, set max_workers based on API rate limits rather than CPU count, collect results with as_completed and handle exceptions per future so one failure does not silently drop the rest. For many cluster operations, bounded parallelism dramatically cuts wall-clock time while staying under API quotas.

THINK OF IT LIKE

It is opening several checkout lanes instead of one, but only as many as the store manager allows so the shelves are not emptied at once.

WHY IT MATTERS IN PRODUCTION

Checking certificate expiry across five hundred namespaces, polling health endpoints for a fleet, or fetching metadata from multiple cloud accounts are natural parallel problems. The mistakes are unbounded parallelism that triggers rate limiting, sharing mutable state between threads without locks, and losing error context when futures fail quietly. `asyncio` is the alternative for high-concurrency I/O, but `ThreadPoolExecutor` is often enough and easier to integrate with synchronous requests code.

STEP BY STEP

- 1 Choose `ThreadPoolExecutor` for I/O-bound API calls; `ProcessPoolExecutor` for CPU-heavy parsing.
- 2 Set `max_workers` conservatively and respect API rate limits with semaphores if needed.
- 3 Use `as_completed` to process results as they arrive and log failures with context.
- 4 Shut down the executor explicitly or use a context manager to avoid dangling threads.

EVIDENCE YOU WOULD QUOTE

```
python3 scripts/check_all_clusters.py --workers 8 --dry-run

grep -r 'ThreadPoolExecutor' automation/ | head

pytest tests/test_parallel_checks.py -v -k rate_limit
```

RED FLAG - DO NOT SAY THIS

Do not fire five hundred threads at an API with no rate limiting. You will DoS your own control plane and blame the script.

EXPECT THIS FOLLOW-UP

When would you choose `asyncio` over `ThreadPoolExecutor`?

Q34**SENIOR**

How do you handle errors and retries in production Python automation?

SAY THIS

I classify errors: programming bugs get fixed, transient infrastructure errors get retried with exponential backoff and jitter, and permanent failures fail fast with a clear message and non-zero exit code. Retries wrap only idempotent operations or use idempotency keys where the API supports them. I never catch bare `Exception` and continue silently; each handler logs context, preserves the traceback, and decides whether to retry, abort or degrade gracefully.

THINK OF IT LIKE

Retries are redialling a busy phone line, not repeatedly pressing the wrong number and hoping it becomes right.

WHY IT MATTERS IN PRODUCTION

Platform automation runs against unreliable networks, throttled APIs and partially upgraded clusters. A script that retries forever masks an outage; one that fails on the first timeout creates alert noise. Libraries like `tenacity` or `urllib3`'s `Retry` help, but the design decision is yours: which exceptions retry, how many attempts, what backoff ceiling, and what happens to already-completed steps in a multi-step workflow.

STEP BY STEP

- 1 Distinguish transient errors (timeouts, 503, connection reset) from permanent ones (404, 403).
- 2 Apply retry with exponential backoff and jitter only to idempotent operations.

- 3 Use context managers and try/finally to release resources even when retries exhaust.
- 4 Exit with distinct codes or structured error output so callers and monitors can triage.

EVIDENCE YOU WOULD QUOTE

```
grep -r 'tenacity\|Retry\|backoff' automation/ | head

pytest tests/test_retry_logic.py -v

oc logs job/platform-sync --tail=100 | grep -i 'retry\|error'
```

RED FLAG - DO NOT SAY THIS

Do not wrap everything in except Exception: pass. The script exits 0, the monitor is green, and production is wrong.

EXPECT THIS FOLLOW-UP

Step three of five fails after steps one and two succeeded. How do you make the rerun safe?

Q35**SENIOR****How do you use boto3 and cloud SDK patterns in platform automation?****SAY THIS**

boto3 uses a session built from environment variables, instance profiles or named profiles, and clients or resources for each service. I prefer clients for explicit API control, paginate list operations instead of assuming a single page, handle ClientError by error code rather than message text, and use waiters for eventual consistency rather than sleep loops. Credentials never live in source code; IAM roles with least privilege replace long-lived keys wherever the job runs in AWS.

THINK OF IT LIKE

It is using the vendor's official courier with a tracked account, not forging shipping labels by guessing the form layout.

WHY IT MATTERS IN PRODUCTION

Multi-cloud platform teams use boto3 for AWS, but the patterns transfer: azure-identity with DefaultAzureCredential, google-cloud with application default credentials. The interview is really about credential chains, pagination, idempotent resource creation and tagging for cost allocation. Wrapping SDK calls in small functions with typed inputs makes them testable with botocore stubbers.

STEP BY STEP

- 1 Create a boto3.Session from the appropriate credential chain for the runtime environment.
- 2 Use client methods with explicit pagination via get_paginator or manual NextToken loops.
- 3 Catch botocore.exceptions.ClientError and branch on response['Error']['Code'].
- 4 Tag resources consistently and use idempotent create patterns: describe first, create if missing.

EVIDENCE YOU WOULD QUOTE

```
python3 -c "import boto3; print(boto3.Session().client('sts').get_caller_identity())"

aws sts get-caller-identity

pytest tests/test_aws_cleanup.py -v --mock-aws
```

RED FLAG - DO NOT SAY THIS

Do not embed access keys in the repository. If they leak, every resource your role can touch is exposed, and scanners find them in minutes.

EXPECT THIS FOLLOW-UP

How do you test boto3 code without calling real AWS APIs?

Q36**ARCHITECT****How would you structure a Python automation project for production use?****SAY THIS**

Separate library code from CLI entry points, pin dependencies in a lockfile or requirements file, run linting and tests in CI, package as a container image for anything that runs in-cluster or on shared infrastructure, and document how to configure credentials per environment. Secrets come from the environment or a vault, never from the repo. Observability is built in: structured logs, metrics for success and failure counts, and trace context when calling distributed APIs.

THINK OF IT LIKE

It is the difference between a script on someone's laptop and a product the platform team stands behind with a version number and a support channel.

WHY IT MATTERS IN PRODUCTION

The architect question is whether your Python is maintainable when the original author leaves. That means src layout, typed interfaces for external systems, contract tests for API responses, semver tagging, and a deliberate decision about what runs as a CronJob versus what stays on a controller. It also means knowing when Python should not be the answer - declarative GitOps beats a reconciliation loop written over a weekend.

STEP BY STEP

- 1 Lay out src/package with modules by domain, tests/, and pyproject.toml with pinned deps.
- 2 Run ruff or flake8, mypy where valuable, and pytest in CI on every pull request.
- 3 Build a container image with a non-root user for in-cluster or shared execution.
- 4 Document configuration, credentials, exit codes and operational runbooks alongside the code.

EVIDENCE YOU WOULD QUOTE

```
tree automation/ -L 2

cat .gitlab-ci.yml | grep -E 'pytest|ruff|build'

oc get cronjob,job -n platform-automation -l app=inventory-sync
```

RED FLAG - DO NOT SAY THIS

Do not treat production automation as a single 800-line script with no tests because we will refactor it later. Later is after the incident.

EXPECT THIS FOLLOW-UP

At what point should this Python project become a Kubernetes operator instead?

PART 4 · Q37-Q50 · 14 QUESTIONS

Advanced Ansible for senior platform engineers

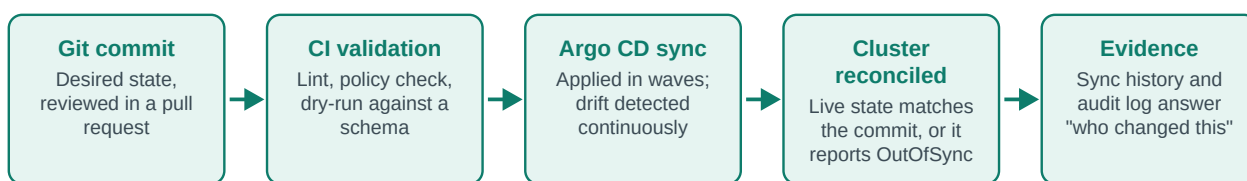
Variable precedence, testing, execution environments and enterprise controller patterns

Senior Ansible questions assume you have run playbooks in anger and now own the framework other teams depend on. Interviewers probe whether you understand precedence rules well enough to debug surprising values, whether you test roles before production, whether automation runs from reproducible execution environments, and whether your controller setup gives auditability without turning every change into a ticket. **Strong answers tie Ansible mechanics to change control and operational safety.**

Level mix - Foundation: 4 · Intermediate: 6 · Senior: 3 · Architect: 1

Change control that survives an audit

The point of GitOps is not the tool, it is that every cluster change has an author, a reviewer, a timestamp and a revert path.



Q37

FOUNDATION

What is Ansible variable precedence and why does it confuse people?

SAY THIS

Variable precedence defines which value wins when the same variable is defined in multiple places - extra vars on the command line beat most other sources, then task vars, block vars, role and include vars, host and group vars, inventory file vars, role defaults, and finally facts. People get confused because group_vars/all.yml and host_vars/myhost.yml feel authoritative until someone passes -e on the command line or a role default silently loses to a higher layer they forgot existed.

THINK OF IT LIKE

It is a stack of memos on a desk. The top one wins, and the memo you wrote last week is still there underneath even if you forgot about it.

WHY IT MATTERS IN PRODUCTION

Most real debugging sessions start with "why did this host get that value". Without precedence in muscle memory, engineers add another definition somewhere higher rather than finding the conflicting one. `ansible-inventory --host` and `--graph`, plus `ansible-playbook` with `-e ansible_debug=true` or `community.general.var_dump`, make the effective value visible. **Documenting where environment-specific data lives prevents the next person from fighting the same battle.**

STEP BY STEP

- 1 State the precedence order from highest (extra vars) to lowest (role defaults and facts).
- 2 Use `ansible-inventory --host hostname` to dump merged variables for one host.
- 3 Search group_vars, host_vars, role defaults and play vars when values surprise you.
- 4 Document the intended source of truth per variable class in the repository README.

EVIDENCE YOU WOULD QUOTE

```
ansible-inventory -i inventories/prod --host web01.example.com
```

```
ansible-playbook site.yml -e @extra.yml --check -v | grep -i myvar
```



```
grep -r 'myvar:' inventories/ roles/ | head
```

RED FLAG - DO NOT SAY THIS

Do not fix surprising values by adding yet another definition at a random layer. Find the conflict or you will inherit it forever.

EXPECT THIS FOLLOW-UP

A host_var and a group_var disagree. Which wins and how do you prove it?

Q38**FOUNDATION****How do you use Ansible Vault to protect secrets?****SAY THIS**

Ansible Vault encrypts sensitive strings or entire files with a password or key file. Encrypted content lives in Git safely; at runtime you provide the vault password via prompt, a permissions-restricted file, or a controller credential injected by AAP. I encrypt group_vars or dedicated vault files rather than sprinkling vault blocks through roles, rotate the vault password on a schedule, and never commit the password alongside the ciphertext.

THINK OF IT LIKE

The recipe is in the shared cookbook, but the safe combination is in a separate envelope only the head chef opens at service time.

WHY IT MATTERS IN PRODUCTION

Vault solves storage in Git, not runtime secrecy on the target host. After decryption, secrets appear in task arguments and can leak into logs if no_log is missing. Enterprise setups often combine vault for static secrets with an external lookup plugin pulling from HashiCorp Vault or AWS Secrets Manager at runtime, so rotation does not require re-encrypting files.

STEP BY STEP

- 1 Create encrypted files with ansible-vault create or encrypt existing files with encrypt.
- 2 Store vault IDs and passwords in a controller credential or a restricted file outside Git.
- 3 Mark tasks handling secrets with no_log: true to keep them out of job output.
- 4 Document rotation: new secret, re-encrypt, deploy, revoke old credential.

EVIDENCE YOU WOULD QUOTE

```
ansible-vault view inventories/prod/group_vars/all/vault.yml

ansible-playbook site.yml --ask-vault-pass --check

grep -r 'no_log: true' roles/ | head
```

RED FLAG - DO NOT SAY THIS

Do not commit vault_password_file to the repository. You have shipped the key with the lock.

EXPECT THIS FOLLOW-UP

How would you rotate a database password stored in vault without downtime?

Q39

FOUNDATION

What are handlers and how do they differ from regular tasks?

SAY THIS

Handlers are tasks that run once at the end of the play, and only if notified by another task that reported a change. They are the right tool for actions that should happen after configuration changes - restarting a service when a config file changes, reloading systemd when a unit file changes. Regular tasks run every time unless a when condition stops them; handlers coalesce multiple notifications into a single run.

THINK OF IT LIKE

Regular tasks are the chef cooking every course. Handlers are "only preheat the oven if at least one dish actually needed baking, and do it once at the end".

WHY IT MATTERS IN PRODUCTION

The classic mistake is using a handler to restart a service when a task uses `changed_when: false` or when copy reports ok instead of `changed` - the handler never fires and production runs stale config until someone notices. The other mistake is notifying from every task when a single meta: `flush_handlers` mid-play is needed for ordering. Handlers belong in roles, named consistently, and documented so application teams know what triggers a restart.

STEP BY STEP

- 1 Define handlers in `handlers/main.yml` with clear names matching their action.
- 2 Notify from tasks only when a meaningful change occurred; verify `changed_when` if needed.
- 3 Use `listen` to alias multiple notify strings to one handler.
- 4 Use meta: `flush_handlers` when later tasks depend on the handler having run.

EVIDENCE YOU WOULD QUOTE

```
grep -r 'notify:' roles/myapp/ | head

ansible-playbook site.yml --check --diff -v | grep -i 'RUNNING HANDLER'

molecule converge --scenario-name default
```

RED FLAG - DO NOT SAY THIS

Do not restart services with a regular task on every playbook run. You cause unnecessary outages and hide whether config actually changed.

EXPECT THIS FOLLOW-UP

Your handler never runs but the config file did change. What do you check?

Q40

FOUNDATION

What are Ansible tags and when do you use them?

SAY THIS

Tags label tasks and roles so you can run a subset of a playbook with `--tags` or skip parts with `--skip-tags`. I tag by concern - packages, config, monitoring, never - and use `never` on destructive or slow tasks that should run only when explicitly requested. Tags make large `site.yml` playbooks usable for targeted maintenance without copying playbooks into fragments that drift apart.

THINK OF IT LIKE

Tags are coloured stickers on workshop drawers. You can pull just the electrical tools without emptying every drawer onto the floor.

WHY IT MATTERS IN PRODUCTION

Tags are powerful and easy to misuse. Untagged tasks always run, which surprises people who expect `--tags` to limit everything. Duplicate tag names across roles can pull in more than intended. Document the tag vocabulary in the README and test tagged runs in CI so a `--tags config` run still converges correctly.

STEP BY STEP

- 1 Define a consistent tag vocabulary: config, packages, services, monitoring, never.
- 2 Apply tags at role level where the whole role shares a concern, task level for exceptions.
- 3 Run `ansible-playbook site.yml --tags config --list-tasks` before executing.
- 4 Test tagged runs in Molecule or CI to ensure partial runs leave the host valid.

EVIDENCE YOU WOULD QUOTE

```
ansible-playbook site.yml --tags config --list-tasks

ansible-playbook site.yml --tags never,destructive --list-tasks

grep -r 'tags:' roles/ | sort | uniq -c | head
```

RED FLAG - DO NOT SAY THIS

Do not rely on tags as your only safety gate for destructive tasks. A forgotten tag still runs if the task is untagged.

EXPECT THIS FOLLOW-UP

Someone runs `--tags config` and a database task still executes. Why?

Q41

INTERMEDIATE

What is Molecule and how do you use it to test Ansible roles?

SAY THIS

Molecule is a testing framework for Ansible roles. It creates ephemeral test instances with a driver like Docker or libvirt, applies your role with `ansible-playbook`, runs verify steps with either Ansible assert tasks or Testinfra, and destroys the instance. The idempotence test runs converge twice and expects no changes on the second pass. It turns "works on my laptop" into evidence in CI.

THINK OF IT LIKE

It is a crash test for a car part before it goes on the production line, using a cheap dummy vehicle you can wreck and rebuild.

WHY IT MATTERS IN PRODUCTION

Roles without tests rot quietly as platforms upgrade underneath them. Molecule scenarios per OS version or major configuration variant catch module deprecations, wrong package names and broken templates before AAP job templates run against production inventory. The cost is maintaining Docker images or cloud templates for the driver; execution environments make the Ansible side reproducible.

STEP BY STEP

- 1 Init a scenario with `molecule init scenario -d docker -r myrole`.
- 2 Write `converge.yml` to apply the role and `verify.yml` with assertions on final state.
- 3 Run `molecule test` locally and in CI on every pull request touching the role.
- 4 Include the idempotence stage so the second converge reports zero changes.

EVIDENCE YOU WOULD QUOTE

```
cd roles/myrole && molecule test
```

```
molecule converge && molecule idempotence

grep -r 'molecule' .gitlab-ci.yml .github/workflows/
```

RED FLAG - DO NOT SAY THIS

Do not skip the idempotence test. It is the cheapest check that your role is safe to re-run after a partial failure.

EXPECT THIS FOLLOW-UP

Your role passes on Rocky 9 but fails on RHEL 8. How do you structure scenarios?

Q42**INTERMEDIATE****What is ansible-lint and what does it catch?****SAY THIS**

ansible-lint statically analyses playbooks, roles and collections against rules for correctness, style and security. It flags bare variables that need quoting, risky shell usage, missing name keys, deprecated modules, privilege escalation mistakes and filename violations. Running it in CI on every commit prevents low-quality content from reaching the controller and gives reviewers a baseline beyond opinion.

THINK OF IT LIKE

It is spell-check and grammar-check for automation, with a few rules that also stop you from leaving the stove on.

WHY IT MATTERS IN PRODUCTION

ansible-lint profiles let you ramp strictness: min for legacy repos, production for new work. Some rules are stylistic; others catch real bugs like command-instead-of-shell without creates/removes guards. Suppressing rules needs a comment explaining why, or the exception becomes permanent. Pair lint with yamllint for formatting and schema validation for inventory structure.

STEP BY STEP

- 1 Run ansible-lint on the repository root or per collection with a pinned version.
- 2 Enable a profile appropriate to the repo age and tighten it over time.
- 3 Fix or justify violations; use noqa comments sparingly with a reason.
- 4 Gate merges in CI on ansible-lint exit code zero.

EVIDENCE YOU WOULD QUOTE

```
ansible-lint roles/myrole

ansible-lint --profile production .

grep ansible-lint .gitlab-ci.yml
```

RED FLAG - DO NOT SAY THIS

Do not disable ansible-lint globally because the legacy repo is noisy. Fix or profile incrementally or quality never improves.

EXPECT THIS FOLLOW-UP

Which ansible-lint rule has caught a real bug for you, not just style?

Q43

INTERMEDIATE

What is dynamic inventory and when do you use it?

SAY THIS

Dynamic inventory generates the host list at runtime from a script or plugin querying a source of truth - cloud APIs, a CMDB, VMware, OpenShift - instead of maintaining static INI files that drift. Inventory plugins are the modern form: YAML config pointing at a plugin like `amazon.aws.aws_ec2` or constructed to build groups from host vars. The automation always targets what actually exists now.

THINK OF IT LIKE

Static inventory is a printed phone book. Dynamic inventory is directory enquiry: you get today's numbers, not the ones from three years ago.

WHY IT MATTERS IN PRODUCTION

Hybrid estates often combine constructed inventory: a dynamic plugin pulls cloud hosts, then inventory plugins add groups by tags, and `group_vars` layer environment config. Caching with `cache_plugin` helps rate limits but can serve stale data; tune timeout for your change velocity. For OpenShift, the `kubernetes.core.k8s` inventory targets pods or nodes when you need to automate inside the cluster boundary.

STEP BY STEP

- 1 Choose an inventory plugin matching the source of truth, not a hand-maintained export.
- 2 Configure authentication via environment or credential plugin, not keys in the repo.
- 3 Validate with `ansible-inventory -i config.yml --graph` and `--host` for sample hosts.
- 4 Set cache timeout consciously if using inventory caching.

EVIDENCE YOU WOULD QUOTE

```
ansible-inventory -i inventories/aws_ec2.yml --graph | head -30
```

```
ansible-inventory -i inventories/aws_ec2.yml --list --export | jq '.meta.hostvars | keys | length'
```

```
ansible-inventory --graph -i inventories/ | grep -c 'orphan'
```

RED FLAG - DO NOT SAY THIS

Do not export cloud inventory to a static file on a cron job without noticing when the export fails. You automate against ghosts.

EXPECT THIS FOLLOW-UP

Inventory cache shows terminated hosts. What failed and how do you detect it?

Q44

INTERMEDIATE

What are Ansible collections and how do you manage them?

SAY THIS

Collections are distributable packages of modules, plugins, roles and playbooks under a namespace, installed with `ansible-galaxy collection install` from Galaxy, Automation Hub or a private index. `requirements.yml` pins names and versions; execution environments install them at build time so runtime does not depend on the public internet. Vendoring collections into the repo is an option for disconnected sites but shifts upgrade responsibility to you.

THINK OF IT LIKE

Collections are app store packages for Ansible. The core engine stays slim; you install the AWS shelf, the Windows shelf and the network shelf separately.

WHY IT MATTERS IN PRODUCTION

Most production modules live in collections now, not ansible.builtin alone. Pinning versions matters because collection majors change module parameters and deprecate old names. CI should run `ansible-galaxy collection install -r requirements.yml --force` followed by tests so a broken pin fails before the controller job template does.

STEP BY STEP

- 1 Declare dependencies in requirements.yml with explicit versions or version ranges.
- 2 Install with `ansible-galaxy collection install -r requirements.yml -p ./collections`.
- 3 Build execution environments that bake the same requirements for controller jobs.
- 4 Review release notes before bumping collection majors in production.

EVIDENCE YOU WOULD QUOTE

```
cat requirements.yml

ansible-galaxy collection list | grep amazon.aws

grep collections requirements.yml execution-environment.yml
```

RED FLAG - DO NOT SAY THIS

Do not run `ansible-galaxy install latest` on every job without pins. You will upgrade into a breaking module change mid-incident.

EXPECT THIS FOLLOW-UP

A collection major upgrade renames a module. How do you migrate safely?

Q45**INTERMEDIATE****What is ansible-builder and how does it relate to execution environments?****SAY THIS**

ansible-builder reads an execution-environment definition - base image, Python packages, system packages, ansible-core version, collections - and produces a container image with everything installed. That image is what ansible-navigator, AAP job templates and CI pipelines run. It replaces ad hoc pip install on control nodes with a versioned artefact built once, scanned, signed and promoted.

THINK OF IT LIKE

ansible-builder is the factory that assembles the toolbox. The execution environment is the finished toolbox on the truck.

WHY IT MATTERS IN PRODUCTION

The definition file lives in Git and changes like application code: pull request, CI build, test playbooks against the new image, promote by tag or digest. Multi-stage builds keep images smaller. Disconnected environments pre-build on a connected builder, push to a private registry, and AAP pulls from there. Pin ansible-builder itself so image layout does not shift unexpectedly.

STEP BY STEP

- 1 Author execution-environment.yml with base image, deps and galaxy requirements.
- 2 Run `ansible-builder build -t ee-platform:1.2 -f execution-environment.yml`.
- 3 Test the image with ansible-navigator or a CI playbook before promoting.
- 4 Reference the image by digest in AAP, not only by floating tags.

EVIDENCE YOU WOULD QUOTE

```
ansible-builder build -t ee-platform:1.2 -f execution-environment.yml

podman inspect ee-platform:1.2 | jq '[0].Digest'

ansible-navigator run site.yml --execution-environment-image ee-platform:1.2
```

RED FLAG - DO NOT SAY THIS

Do not let every job template build its own EE on the fly. You have no idea what ran last Tuesday.

EXPECT THIS FOLLOW-UP

Two job templates use different EE images with different collection versions. What breaks?

Q46

INTERMEDIATE

What is an Ansible execution environment in practice?

SAY THIS

An execution environment is a container image containing `ansible-core`, collections, Python dependencies and optionally system binaries your modules need. AAP and `ansible-navigator` run playbooks inside this image so developer laptops, CI and production controllers execute identical automation. It eliminates "works on my control node" and is the standard delivery mechanism on Ansible Automation Platform 2.

THINK OF IT LIKE

It is the standard issue toolkit every engineer gets on day one, identical down to the serial number, instead of everyone bringing their own mismatched screwdrivers.

WHY IT MATTERS IN PRODUCTION

Legacy `virtualenv` on the control node still appears in older estates but does not scale when fifty job templates need different collection sets. Execution environments compose with private automation hub for validated content. Operational concerns include image size, pull secrets on disconnected controllers, and RBAC over who can publish new image versions.

STEP BY STEP

- 1 Define one EE per logical dependency profile, not one per playbook.
- 2 Build from `execution-environment.yml` with `ansible-builder` and store in a trusted registry.
- 3 Assign EE images to job templates and project sync jobs in AAP.
- 4 Rotate and scan images on the same schedule as other production container artefacts.

EVIDENCE YOU WOULD QUOTE

```
ansible-navigator images

awx --help 2>/dev/null; oc get automationcontroller -n aap 2>/dev/null | head

grep execution_environment inventories/controller/job_templates/
```

RED FLAG - DO NOT SAY THIS

Do not run production jobs from the controller's bundled EE if it drifted from what you tested in CI.

EXPECT THIS FOLLOW-UP

How do you debug a module missing from the execution environment?

Q47**SENIOR****How do block, rescue and always work in Ansible playbooks?****SAY THIS**

block groups tasks with optional error handling: tasks in block run normally; if one fails, rescue tasks run instead; always tasks run regardless of success or failure, like a finally block. Use it for transactional sections - attempt a change, run cleanup on failure, notify monitoring in always. It does not replace proper idempotency; it contains failure blast radius within a play.

THINK OF IT LIKE

block is the main procedure, rescue is the spill kit when something breaks, always is turning the lights off and signing the logbook before you leave.

WHY IT MATTERS IN PRODUCTION

Rescue runs only on task failures, not on unreachable hosts unless you handle that separately with max_fail_percentage or ignore_unreachable. Always runs even when rescue also runs, which is useful for slack notifications or metric emission. Overusing block for flow control when when conditions suffice makes playbooks harder to read than straight-line tasks with clear names.

STEP BY STEP

- 1 Wrap risky task groups in block with a descriptive name on the block itself.
- 2 Write rescue tasks that restore safe state or gather diagnostics, not hide errors.
- 3 Use always for notifications, log uploads and metric pushes.
- 4 Re-raise or fail explicitly in rescue when the play should stop.

EVIDENCE YOU WOULD QUOTE

```
grep -A20 'block:' playbooks/rolling_update.yml | head -25

ansible-playbook site.yml -v | grep -E 'RESCUE|ALWAYS'

molecule verify --scenario-name failure_path
```

RED FLAG - DO NOT SAY THIS

Do not use rescue to swallow errors and continue as if nothing happened. The playbook shows ok and production is broken.

EXPECT THIS FOLLOW-UP

Rescue runs but always reports success while the service is down. What went wrong?

Q48

SENIOR

How do you perform rolling updates with Ansible safely?

SAY THIS

Use `serial` or `max_fail_percentage` on the play to limit concurrent hosts - for example `serial: "10%"` or `serial: 1` for one-at-a-time. Combine with health checks after each batch, `delegate_to` monitoring for verification, and stop the play if a batch fails before touching the rest. Pre-tasks drain connections where applicable; handlers restart services only after config validates.

THINK OF IT LIKE

It is renovating one hotel floor at a time while checking guests can still reach the reception desk, not closing the entire building at once.

WHY IT MATTERS IN PRODUCTION

Rolling updates intersect with load balancers: remove from pool, patch, wait for health, return to pool. Ansible orchestrates the steps but the pattern belongs in a role with variables for batch size and health check URL. For Kubernetes workloads Ansible is often the wrong tool; for VMs and appliances it remains essential. Document rollback per batch because `serial forward` is only half the story.

STEP BY STEP

- 1 Set `serial` or `max_fail_percentage` appropriate to service tolerance and capacity headroom.
- 2 Remove host from load balancer or mark out of service before destructive tasks.
- 3 Run post-update validation tasks; fail the play if health checks do not pass.
- 4 Use `run_once` or `delegate_to` for monitoring integration and audit logging.

EVIDENCE YOU WOULD QUOTE

```
grep -E 'serial:|max_fail_percentage:' playbooks/rolling_update.yml

ansible-playbook rolling_update.yml --limit webservers --check -v

Controller job output showing batch boundaries and health check results
```

RED FLAG - DO NOT SAY THIS

Do not run the same play against all hosts with no `serial` on a stateful service. You will take the entire fleet offline simultaneously.

EXPECT THIS FOLLOW-UP

Batch two fails health checks. How do you roll back batch one hosts?

Q49

SENIOR

How does Ansible Automation Platform change how you run Ansible at scale?

SAY THIS

AAP adds a controller for job templates, schedules, surveys, RBAC, credential injection, execution environments, automation hub for curated collections, and an audit trail of who launched what against which inventory. Teams run vetted automation without SSH keys to every host or direct access to playbooks. It turns Ansible from individual scripts into a shared service with governance.

THINK OF IT LIKE

It is the difference between everyone driving their own car to a job site and a dispatch office that assigns certified drivers, tracked vehicles and signed work orders.

WHY IT MATTERS IN PRODUCTION

The interview is about governance, not feature lists. Job templates expose parameters through surveys while hiding inventory credentials. Workflow templates chain jobs with approval nodes for change windows. Event-driven ansible rulebooks react to webhooks. Integration with IdM maps LDAP groups to roles so application teams can restart a service without seeing database passwords.

STEP BY STEP

- 1 Publish roles as projects synced from Git with CI-tested commit SHAs.
- 2 Create job templates with scoped inventories, credentials and execution environments.
- 3 Use RBAC so teams run only what they need; audit logs capture user and outcome.
- 4 Chain high-risk operations through workflow templates with approval gates.

EVIDENCE YOU WOULD QUOTE

```
awx job_templates list 2>/dev/null || echo 'query controller API'
```

```
Controller job history: user, template, inventory, start time, status
```

```
grep -r 'survey' controller/job_templates/ | head
```

RED FLAG - DO NOT SAY THIS

Do not give everyone admin on the controller to save ticket time. You have centralised all credentials in one place with no gate.

EXPECT THIS FOLLOW-UP

How would you let an app team restart a service without exposing SSH or vault passwords?

Q50

ARCHITECT

How would you design an enterprise Ansible framework end to end?

SAY THIS

Collections and small roles in Git with semver tags; inventories separated from logic; dynamic inventory from the CMDB or cloud; secrets from vault or external lookups; execution environments built with ansible-builder and pinned in AAP; CI running ansible-lint, yamllint and Molecule; job templates as the only production entry point with RBAC and surveys; promotion through dev, staging and prod inventories mirroring application release trains.

THINK OF IT LIKE

It is building a municipal water system with treatment plants, tested pipes and metered connections - not asking each house to dig its own well.

WHY IT MATTERS IN PRODUCTION

The architect question is organisational as much as technical. A framework nobody uses because it is slower than SSH loses by default. Success means application teams get self-service within guardrails, platform teams own the EE and collection upgrades, and auditors get job history plus Git blame. The anti-pattern is a monolithic site.yml with environment if-statements copied into every role.

STEP BY STEP

- 1 Separate content (collections/roles), data (inventories/group_vars), and runtime (EE images).
- 2 Enforce quality gates in CI: lint, test, build EE, scan image.
- 3 Expose operations through controller job templates with tiered RBAC and documented surveys.
- 4 Measure adoption: which manual tasks remain and why teams bypass the framework.

EVIDENCE YOU WOULD QUOTE

Repository layout: collections/, inventories/, execution-environment.yml, .gitlab-ci.yml

CI pipeline stages: lint, molecule, ee-build, deploy-template

Controller metrics: job count by team, failure rate, template age

RED FLAG - DO NOT SAY THIS

Do not centralise automation without a migration path for existing playbooks. Teams will parallel-run shadow automation forever.

EXPECT THIS FOLLOW-UP

Teams bypass AAP and run ansible-playbook from laptops. What is missing from your design?

PART 5 · Q51-Q64 · 14 QUESTIONS

Go for platform engineers

The language behind client-go, operators and most of the control plane — and what interviewers actually probe

Go is not optional on a senior platform team anymore. The Kubernetes API machinery, client-go, controller-runtime, the Operator SDK and a large fraction of the cluster components you operate are written in it. Interviewers rarely ask you to implement a red-black tree; they ask whether you can reason about goroutine lifetimes, whether you know why a controller requeues, and whether you would trust your own operator in production. **This part covers the Go idioms and Kubernetes framework pieces that show up in those conversations.**

Level mix - Foundation: 3 · Intermediate: 6 · Senior: 5

Who owns what in an OpenShift cluster

When something is broken, the fastest question is not "what is wrong" but "who owns this". Read the conditions on the owner, not the symptom at the bottom.

CVO	Owns the release payload. Drives every Cluster Operator toward the version you asked for.
Cluster Operators	Own core capabilities: authentication, ingress, storage, monitoring, network, etcd.
OLM	Owns optional add-on operators through CatalogSource, Subscription, InstallPlan and CSV.
MCO	Owns node configuration. Renders MachineConfigs and rolls them out pool by pool.
Machine API	Owns machine existence: MachineSet, Machine, MachineHealthCheck, autoscaling.
kube-controller-manager	Owns workload reconciliation: ReplicaSets, endpoints, node lifecycle, PV binding.
kubelet + CRI-O	Owns what actually runs: pod sandboxes, containers, probes, image pulls, node status.

Q51**FOUNDATION**

What is a goroutine, and when would you not spawn one?

SAY THIS

A goroutine is a lightweight concurrent function started with the `go` keyword. The Go runtime multiplexes thousands of them onto a small pool of OS threads, so they are cheap to create and cheap to block on I/O. You use them when work can proceed independently — serving HTTP requests, watching an API stream, fanning out health checks. You do not spawn one for every tiny task: unbounded goroutines on a hot path are a memory leak with a polite name, and anything that needs strict ordering is usually clearer with a channel or a worker pool than with goroutines firing at random.

THINK OF IT LIKE

Goroutines are extra staff on a shift, not extra buildings. Hiring is fast, but if you hire everyone who walks past the door you will run out of desks.

WHY IT MATTERS IN PRODUCTION

This shows up constantly in operators. A reconcile function that launches a goroutine per object without a wait group or context cancellation will leak work on shutdown and can double-apply changes during leader election flaps. **Interviewers use this question to see whether you understand concurrency as a resource decision, not a**

syntax trick.**STEP BY STEP**

- 1 Define a goroutine as a concurrent function scheduled by the Go runtime, not an OS thread.
- 2 Explain why they are cheap: small stacks, M:N scheduling, blocking does not pin a thread.
- 3 Give a good use case — HTTP handler, watch loop, parallel I/O with a bounded fan-out.
- 4 Name the failure mode — unbounded creation, work that outlives process shutdown, races on shared state — and point to channels, errgroup or a worker pool as the fix.

EVIDENCE YOU WOULD QUOTE

```
go tool pprof http://localhost:6060/debug/pprof/goroutine # count stacks in a controller

GODEBUG=gctrace=1 ./operator --zap-devel # correlate goroutine growth with GC pressure

grep -R 'go func' internal/controller/ | wc -l # audit unbounded spawns in your codebase
```

RED FLAG - DO NOT SAY THIS

Do not say "just use a goroutine, they are free". They are cheap, not free, and the runtime will not save you from spawning one per reconcile forever.

EXPECT THIS FOLLOW-UP

Your operator's goroutine count climbs during a thundering herd. How do you cap it?

Q52**FOUNDATION****How do channels work, and what is the difference between buffered and unbuffered?****SAY THIS**

A channel is a typed conduit for sending values between goroutines. An unbuffered channel synchronises: the sender blocks until a receiver is ready, which gives you a hand-off point. A buffered channel decouples sender and receiver up to its capacity — useful for worker pools and rate smoothing, dangerous if nobody drains it. Send and receive directions are part of the type. Closing a channel signals no more values; receivers should range or use the comma-ok form to detect close. The idiomatic rule is "do not communicate by sharing memory; share memory by communicating" — but only when a channel actually models the problem.

THINK OF IT LIKE

Unbuffered is a direct hand-off at the door. Buffered is a tray on the counter — fine until the tray fills and the kitchen stops cooking.

WHY IT MATTERS IN PRODUCTION

In platform code you see channels in rate limiters, shutdown signals and test harnesses. Controllers in client-go rarely expose channels to you directly — they use workqueues instead — but understanding blocking and back-pressure explains why a saturated queue stalls reconciliation. Senior interviews often pivot from channels to "why doesn't controller-runtime use a channel for the informer cache?"

STEP BY STEP

- 1 State the purpose: safe communication and synchronisation between goroutines.
- 2 Contrast unbuffered (rendezvous, enforces pairing) with buffered (decouples up to N).
- 3 Explain close semantics — only the sender closes; receivers must handle zero values.
- 4 Land on when not to use one — shared mutable state with a Mutex is sometimes clearer.

EVIDENCE YOU WOULD QUOTE

```
go test -race ./... # channel misuse often surfaces as a data race first

dlv debug ./cmd/manager --headless # break on send/receive in a worker pool

grep -n 'make(chan' internal/worker/pool.go # review buffer sizes in your code
```

RED FLAG - DO NOT SAY THIS

Do not pick buffer size 1000 because it "feels safe". Every buffer is a latency and memory trade-off you should be able to justify.

EXPECT THIS FOLLOW-UP

When would you reach for a sync.Mutex instead of a channel?

Q53**FOUNDATION****How do Go interfaces work, and why does Kubernetes code use small ones?****SAY THIS**

An interface is a set of method signatures. A type satisfies an interface implicitly by implementing those methods — no implements keyword. The empty interface `any` holds anything but costs a type assertion at use. Kubernetes and controller-runtime favour small interfaces — `io.Reader`, `client.Reader`, `client.Writer` — so mocks and fakes are easy in tests. `client.Client` is deliberately narrow compared to the raw `clientset`. Accept interfaces, return concrete types is the local idiomatic rule, though controllers often depend on `client.Client` injected by the manager.

THINK OF IT LIKE

An interface is a job description. Anyone who can do the listed tasks gets hired; you do not care which university they attended.

WHY IT MATTERS IN PRODUCTION

Interviewers use interfaces to probe testability. If you cannot explain why `reconcile.Recorder` is an interface but your database handle is a concrete struct, you will struggle on "how do you unit test without a cluster". Fake client from controller-runtime's fake package exists because `Client` is an interface.

STEP BY STEP

- 1 Define implicit satisfaction and contrast with Java-style explicit implementation.
- 2 Explain small interfaces — one or two methods — and the mocking benefit.
- 3 Map to client-go: `Client`, `Reader`, `StatusClient`, the recorder pattern.
- 4 Warn on `interface{}` / any soup in API types — use generics or concrete structs where clear.

EVIDENCE YOU WOULD QUOTE

```
go doc sigs.k8s.io/controller-runtime/pkg/client Client

grep -R 'client.Client' controllers/ | head

go test ./controllers/ -run TestReconcile -v # fake client in unit tests
```

RED FLAG - DO NOT SAY THIS

Do not create a fifteen-method interface "for flexibility". Large interfaces are harder to mock and usually mean the abstraction is wrong.

EXPECT THIS FOLLOW-UP

How does the fake client differ from envtest for controller tests?

Q54**INTERMEDIATE****What does select do, and how do you avoid starving a case?****SAY THIS**

select waits on multiple channel operations and executes whichever case is ready first. If several are ready, Go chooses one at random — there is no priority. A default case makes the select non-blocking, which is useful for polling but easy to abuse into a busy loop. The classic pattern combines a work channel, a done channel from context cancellation, and sometimes a timer. Starvation happens when one case always wins — for example, a tight loop on default while a shutdown signal never runs — and the fix is to remove default, use context, or split responsibilities across goroutines.

THINK OF IT LIKE

select is a receptionist with several ringing lines. If one caller never hangs up, everyone else waits — unless you have a rule for when to pick up which phone.

WHY IT MATTERS IN PRODUCTION

You will write select in health checks, graceful shutdown and any custom controller machinery outside controller-runtime. In interviews the follow-up is almost always context. Context integration: select on ctx.Done() alongside your work channel so SIGTERM during a rolling update actually stops the goroutine.

STEP BY STEP

- 1 Define select as multiplexing channel operations with random tie-breaking among ready cases.
- 2 Show the three common cases: work, cancellation, timeout via time.After or context.
- 3 Explain default — non-blocking receive/send — and why busy loops are a smell.
- 4 Describe starvation and the fix: no default on hot paths, dedicated shutdown goroutine, or priority via separate selects.

EVIDENCE YOU WOULD QUOTE

```
grep -R 'select {' internal/ | head # locate shutdown and worker loops

go test -timeout 30s -run TestGracefulShutdown ./...

kill -TERM $(pidof manager) ; sleep 2 ; ps aux | grep manager # verify clean exit
```

RED FLAG - DO NOT SAY THIS

Do not put heavy work inside the select body. select chooses; goroutines should do the work.

EXPECT THIS FOLLOW-UP

How does context cancellation replace a manual done channel in a worker loop?

Q55

INTERMEDIATE

How should platform code use context.Context?

SAY THIS

context carries deadlines, cancellation signals and request-scoped values down a call chain. The rule for libraries and controllers is: accept Context as the first parameter, never store it in a struct. context.WithCancel, WithTimeout and WithDeadline create child contexts; cancelling the parent cancels all children. In Kubernetes code you pass ctx into client-go calls so watches and API requests stop promptly when reconcile ends or the process shuts down. Values in context are for request metadata — trace IDs, user identity — not for passing optional function parameters you were too lazy to thread.

THINK OF IT LIKE

Context is the fire alarm for a building wing. Pull one lever and every room on that branch knows to stop and leave — but only if people actually listened when they moved in.

WHY IT MATTERS IN PRODUCTION

This is non-negotiable in operator interviews. A Reconcile that ignores context will keep calling the API during leader loss, prolong shutdown past the kubelet's grace period, and fight the next leader's work. OpenShift and upstream controllers propagate context from manager.SetupWithManager through to client calls — breaking that chain is a production bug.

STEP BY STEP

- 1 State the contract: first parameter, never stored, propagates cancellation and deadlines.
- 2 Name WithCancel, WithTimeout, WithDeadline and what cancelling a parent does.
- 3 Map it to reconcile: ctx from controller-runtime, pass to client.Get/List/Patch/Delete.
- 4 Warn against context.Value abuse — only cross-cutting metadata, not business config.

EVIDENCE YOU WOULD QUOTE

```
grep -n 'context.Background()' controllers/ # Background in hot paths is a review flag

oc logs -n <ns> deploy/<operator> | grep 'context canceled'

dlv break reconcile.go:42 # inspect ctx.Err() on shutdown path
```

RED FLAG - DO NOT SAY THIS

Do not say "I use context.Background() everywhere in the controller". That tells the interviewer your operator ignores SIGTERM semantics.

EXPECT THIS FOLLOW-UP

Reconcile runs past the 30-second timeout every time. Where do you enforce the deadline?

Q56

INTERMEDIATE

When do you use sync.Mutex, and what is wrong with a global lock in a controller?

SAY THIS

A Mutex serialises access to shared mutable state — a cache map, a metrics counter, an in-memory index the controller maintains alongside the API. Lock before read/write, unlock with defer in the same function, and keep the critical section small. sync.RWMutex helps when reads dominate. In controllers the smell is a global Mutex around reconcile: it defeats parallelism, hides races you should fix with the API server's optimistic concurrency, and becomes a bottleneck under load. Prefer the apiserver as source of truth and patch with resourceVersion; use Mutex only for process-local bookkeeping.

THINK OF IT LIKE

A Mutex is a toilet key for one stall. Fine for that stall; ridiculous if the whole building shares one key.

WHY IT MATTERS IN PRODUCTION

client-go's informer cache is already thread-safe. Interviewers ask this to see if you reach for locks before understanding what the framework gives you. Real bugs show up when operators cache derived state — last applied checksum, external API tokens — without sync, or when tests pass because -race was never run in CI.

STEP BY STEP

- 1 Define Mutex as mutual exclusion for shared memory, with defer unlock as the idiomatic pattern.
- 2 Mention RWMutex when many readers and rare writers justify the complexity.
- 3 Contrast with channels — Mutex for protecting state, channels for orchestration.
- 4 Call out the controller anti-pattern: global lock serialising every reconcile.

EVIDENCE YOU WOULD QUOTE

```
go test -race ./controllers/...

grep -R 'sync.Mutex' internal/ controllers/

go tool trace trace.out # look for lock contention in load tests
```

RED FLAG - DO NOT SAY THIS

Do not say "I add a Mutex whenever I have concurrency". That usually means the design needs channels, the API server, or fewer shared variables.

EXPECT THIS FOLLOW-UP

How does resourceVersion give you lock-free concurrency against the API server?

Q57

INTERMEDIATE

What does the race detector do, and when must you run it?

SAY THIS

Go's race detector instruments memory accesses at runtime and reports when two goroutines access the same variable and at least one is a write without synchronisation. Enable it with go test -race or go run -race. It slows execution and increases memory use — typically five to ten times — so you run it on unit and integration tests in CI, not on every local edit. It catches bugs that pass review: map writes from reconcile goroutines, lazy-init singletons, closed-over loop variables before Go 1.22. It does not catch every logic bug and it does not help with distributed races across the cluster — only your process.

THINK OF IT LIKE

It is CCTV for shared desks. It will not stop someone taking your stapler, but it will prove two people reached for it at once.

WHY IT MATTERS IN PRODUCTION

Platform teams shipping operators should treat -race in CI as seriously as go vet. A data race in a controller can corrupt an in-memory cache and write garbage back to the API server under load — intermittent, evil, and exactly the class of bug senior interviews ask you to prevent. **OpenShift CI runs race-enabled tests on core components for this reason.**

STEP BY STEP

- 1 Explain what it detects: unsynchronised concurrent access, at least one write.
- 2 Show how to enable it: -race on test and selective integration runs.
- 3 State the cost: slower, more RAM — CI gate, not default dev loop.
- 4 Note limits: in-process only; does not replace proper API-level concurrency design.

EVIDENCE YOU WOULD QUOTE

```
go test -race ./...

grep -i race .github/workflows/*.yaml .gitlab-ci.yaml

go test -race -count=50 ./controllers/ # flake hunt on suspicious tests
```

RED FLAG - DO NOT SAY THIS

Do not say "we will catch races in production". Production is the wrong place to learn your operator has a data race.

EXPECT THIS FOLLOW-UP

Tests pass without -race but fail with it. What is your triage order?

Q58**INTERMEDIATE****What is idiomatic error handling in Go for controllers and CLI tools?****SAY THIS**

Go uses explicit error returns, not exceptions. Check err immediately; wrap with `fmt.Errorf "...: %w", err` to preserve the chain; inspect with `errors.Is` and `errors.As`. In controllers return `(ctrl.Result{}, err)` to requeue with exponential backoff, or `(ctrl.Result{RequeueAfter: t}, nil)` for known transient conditions. User-facing CLI messages belong at the top of main; libraries return errors without printing. Sentinel errors like `ErrNotFound` are fine for control flow if documented. `panic` is for programmer bugs, not failed API calls.

THINK OF IT LIKE

Errors are receipts, not alarms. You pass the receipt up the chain until someone who can refund the customer — log, metric, requeue — is holding it.

WHY IT MATTERS IN PRODUCTION

Operators that swallow errors or always return nil hide outages until CR status stops updating. Interviewers listen for wrap vs print, and for knowing when `client.IgnoreNotFound` is correct. **Go 1.20+ errors.Join helps aggregate parallel failures in migration tools.**

STEP BY STEP

- 1 State the rule: check every error, wrap with context at boundaries.
- 2 Show `errors.Is/As` for typed inspection — `NotFound`, `Conflict`, `TooManyRequests`.
- 3 Map to reconcile returns: error for retry, `RequeueAfter` for timed retry, nil for done.

4 Separate CLI: cobra RunE returns error to main; controller logs with log.FromContext(ctx).**EVIDENCE YOU WOULD QUOTE**

```
grep -R 'fmt.Errorf.*%w' controllers/

oc get <crd> <name> -o jsonpath='{.status.conditions}' | python3 -m json.tool

controller-runtime metrics: workqueue_retries_total
```

RED FLAG - DO NOT SAY THIS

Do not log an error and return nil in Reconcile. You have hidden the failure from the workqueue and the object will never retry.

EXPECT THIS FOLLOW-UP

When do you surface a terminal error in status.conditions instead of retrying forever?

Q59**INTERMEDIATE****What are the moving parts of client-go for someone writing an operator?****SAY THIS**

client-go is the Go client for the Kubernetes API. The pieces you actually touch are: rest.Config from kubeconfig or in-cluster service account; typed clientsets for built-in resources; the dynamic client for arbitrary GVRs; the informer/reflector pair that lists and watches, maintaining a local cache; the workqueue that deduplicates events into reconcile keys; and listers for cache reads without hitting the API. controller-runtime wraps these with a cleaner manager, shared informers, and a delegating client that reads from cache and writes through the API. You still need to know what happens when the cache is stale or when a direct API read is required.

THINK OF IT LIKE

client-go is the postal system: trucks on schedules (informers), a local sorting office (cache), and registered mail (writes) when the copy on your desk might be out of date.

WHY IT MATTERS IN PRODUCTION

Senior platform roles expect you to debug without controller-runtime magic. If watches disconnect, the reflector relists; if relist is too heavy, you see APF throttling. Knowing client-go explains operator memory use — one cache entry per watched object — and why cluster-scoped watches on ConfigMaps are a bad idea.

STEP BY STEP

- 1 Name rest.Config and auth — in-cluster vs kubeconfig, QPS/Burst tuning.
- 2 Explain informer: ListWatch, DeltaFIFO, indexer, resync period.
- 3 Describe workqueue: Add, rate limiting, forget after success.
- 4 State what controller-runtime adds: Manager, cached client, scheme registration.

EVIDENCE YOU WOULD QUOTE

```
oc whoami ; oc auth can-i list pods --all-namespaces

grep -R 'NewForConfig' vendor/sigs.k8s.io | head

curl -k -H "Authorization: Bearer $(cat /var/run/secrets/.../token)"
https://kubernetes.default.svc/api/v1/namespaces
```

RED FLAG - DO NOT SAY THIS

Do not say "controller-runtime replaces client-go so I do not need to know it". When the cache lies, you will need the raw client.

EXPECT THIS FOLLOW-UP

When must you use the uncached API reader instead of the default client?

Q60**SENIOR****How does `golang.org/x/sync/errgroup` help in platform tooling?****SAY THIS**

`errgroup.Group` wraps a set of goroutines and returns the first error from any of them, cancelling sibling work when tied to a context. `g, ctx := errgroup.WithContext(parent)` then `g.Go(func() error { ... })` for each task; `g.Wait()` blocks for completion or failure. It is the right tool for parallel cluster checks — hit every node API, pull metrics from several endpoints, migrate objects in shards — where you want all-or-nothing semantics and clean cancellation. It replaces ad-hoc `WaitGroups` plus error channels plus manual cancel wiring that everyone gets slightly wrong.

THINK OF IT LIKE

`errgroup` is a project manager who stops the whole job when one subcontractor walks off — and tells everyone else to down tools the same day.

WHY IT MATTERS IN PRODUCTION

CLI tools and admission webhooks use `errgroup` heavily; controllers less so because `reconcile` is already serialised per object. Interviewers pair this with worker pools: `errgroup` for a fixed fan-out job, bounded channel workers for sustained throughput. Knowing both keeps you from using `errgroup` to launch ten thousand goroutines against the API server during a migration script.

STEP BY STEP

- 1 Define `errgroup`: `WaitGroup` plus first-error propagation, optional context cancellation.
- 2 Show `WithContext` pattern: one failed goroutine cancels `ctx` for the rest.
- 3 Contrast with `sync.WaitGroup` — no built-in error return or cancel propagation.
- 4 Give a platform example: parallel node validation with a concurrency cap via semaphore.

EVIDENCE YOU WOULD QUOTE

```
grep -R 'errgroup' cmd/ hack/ | head
```

```
go doc golang.org/x/sync/errgroup
```

```
time ./migrate --workers=20 # compare with and without cancel on first 500 error
```

RED FLAG - DO NOT SAY THIS

Do not use `errgroup` to fire unbounded API calls. Pair it with a semaphore or a worker count you can defend.

EXPECT THIS FOLLOW-UP

One shard returns a retryable error. Do you fail the whole `errgroup` or wrap retry logic?

Q61

SENIOR

How would you design a worker pool in Go for sustained API work?

SAY THIS

A worker pool fixes the number of goroutines consuming jobs from a channel. Producers send work items — object keys, node names, file paths — into a jobs channel; N worker goroutines range over it and call a handler; a WaitGroup or done channel signals completion. Size the pool from downstream limits: API server QPS, etcd latency, registry rate limits. Use context cancellation so workers exit when the parent shuts down. For error handling, either send results on a separate channel, use errgroup with a bounded semaphore, or collect errors in a sync-safe slice. The anti-pattern is one goroutine per item on a million-object backlog.

THINK OF IT LIKE

It is a bank with four counters, not four thousand. Customers queue; the building does not hire a new teller for every person who walks in.

WHY IT MATTERS IN PRODUCTION

Bulk adoption tools, custom pruners and migration controllers hit this in production. client-go's own rate limiting is per-client; your pool is the second line of defence. Interviewers want to hear you connect pool size to APF priority levels and 429 responses, not just recite channel syntax.

STEP BY STEP

- 1 Sketch the components: jobs channel, fixed worker count, handler func, shutdown via context.
- 2 Explain sizing: start from API limits and p99 latency, load-test, tune.
- 3 Cover error aggregation — fail fast vs best-effort with a summary report.
- 4 Mention alternatives: workqueue rate limiting in controllers, errgroup for bounded batches.

EVIDENCE YOU WOULD QUOTE

```
grep -R 'worker' cmd/ | grep -E 'chan|pool'
```

```
oc adm top api-resources # watch request rates during a bulk job
```

```
curl -s localhost:9090/metrics | grep apiserver_request_total
```

RED FLAG - DO NOT SAY THIS

Do not hard-code ten workers because it is a round number. Tie the number to a limit you measured or the API server will throttle you anyway.

EXPECT THIS FOLLOW-UP

Workers start getting 429 Too Many Requests. What do you change first — pool size or client QPS?

Q62

SENIOR

Walk through the controller-runtime reconciliation loop.

SAY THIS

You register a Reconciler with `SetupWithManager: For(&YourCR{}), Owns(&Deployment{}), WithOptions`. The manager starts shared informers; when an object changes, the controller enqueues a Request — `namespace/name`, not the event object. `Reconcile(ctx, req)` runs; you fetch the latest object, compare spec to reality, create/update/delete dependents, update status, return `Result`. On error the item requeues with backoff; on success it is forgotten until the next watch event. Resync period periodically requeues everything for drift detection. Leader election ensures only one manager instance mutates at a time. The loop is level-triggered — state, not edge — so missed events do not matter if the next resync or update catches up.

THINK OF IT LIKE

Reconciliation is a hotel housekeeper with a room list, not a guest who only reacts when someone shouts. They enter, compare the room to the standard, fix what is wrong, leave.

WHY IT MATTERS IN PRODUCTION

This is the core operator interview question. They want idempotency, status subresource updates, owner references, and knowing that deleting the CR should trigger cleanup via finalizers — not orphan Deployments. OpenShift platform operators all follow this shape; debugging is `log.FromContext`, events, and `workqueue depth metrics`.

STEP BY STEP

- 1 Register: For primary CR, Owns/ Watches secondary types, predicates if needed.
- 2 Describe enqueue: name-keyed, deduplicated, not one reconcile per field change storm.
- 3 Inside Reconcile: Get latest, handle `NotFound`, apply spec, patch status, return `Result`.
- 4 Close with level-triggered semantics, leader election, and resync as a safety net.

EVIDENCE YOU WOULD QUOTE

```
oc logs -n <ns> deploy/<operator> --tail=100 | grep reconcile

curl localhost:8080/metrics | grep workqueue_depth

oc describe <crd> <name> | sed -n '/Events/, $p'
```

RED FLAG - DO NOT SAY THIS

Do not describe reconciliation as "watch events and react". Events are hints; desired state in etcd is the source of truth.

EXPECT THIS FOLLOW-UP

Spec matches reality but status is wrong. Where do you look first?

Q63

SENIOR

What does the Operator SDK give you on top of controller-runtime?

SAY THIS

Operator SDK is scaffolding and tooling: init/generate for project layout, CSV and bundle generation for OLM, scorecard integration tests, and helpers for Ansible/Helm hybrid operators. For Go operators the core is still controller-runtime and kubebuilder-style markers — +kubebuilder:rbac, webhook, printcolumn. SDK CLI creates Dockerfile, Makefile, and bundle manifests so you can run `operator-sdk run bundle` and test OLM install locally on kind or OpenShift Local. It does not replace understanding reconciliation; it removes boilerplate and standardises packaging for Red Hat catalogues and community operators.

THINK OF IT LIKE

controller-runtime is the engine; Operator SDK is the factory tooling and the compliance paperwork to ship the car.

WHY IT MATTERS IN PRODUCTION

Platform teams evaluating build vs buy ask whether SDK's OLM integration is worth the CRD lifecycle opinion. In 2025/2026 interviews expect scorecard, bundle validation, and multitenancy annotations — run `operator-sdk bundle validate` and know what warn vs error means before publishing to a catalog.

STEP BY STEP

- 1 Separate layers: client-go → controller-runtime → kubebuilder markers → Operator SDK CLI.
- 2 Name deliverables: bundle/, CSV, scorecard, kustomize overlays, run bundle workflow.
- 3 Explain when to use Helm/Ansys plugins vs pure Go — team skills and CR complexity.
- 4 Mention CI: make docker-build docker-push bundle-build bundle-validate bundle-push.

EVIDENCE YOU WOULD QUOTE

```
operator-sdk init --domain example.com --repo github.com/example/operator

operator-sdk bundle validate ./bundle

oc get csv -n <ns> -o jsonpath='{.items[0].status.phase}'
```

RED FLAG - DO NOT SAY THIS

Do not claim Operator SDK "runs the operator in the cluster". It builds and packages; OLM and your Deployment run it.

EXPECT THIS FOLLOW-UP

Bundle validation fails on untested bundle upgrade paths. What do you fix?

Q64

SENIOR

How do finalizers and the workqueue interact during deletion?

SAY THIS

A finalizer is a string in `metadata.finalizers` that blocks object removal until cleared. When a user deletes a CR with a finalizer, the object stays with `deletionTimestamp` set and phase `Terminating`. Your reconcile sees the delete, runs cleanup — tear down cloud resources, revoke certificates, remove external DNS — then removes the finalizer with a patch and the garbage collector deletes the object. The workqueue receives a delete event like any other change; reconcile must handle `deletionTimestamp != nil` before create logic. If cleanup fails, return error and the workqueue retries with backoff; the object stays `Terminating` until you succeed or an admin removes the finalizer. Never orphan external state by stripping finalizers without finishing cleanup.

THINK OF IT LIKE

Finalizers are the checklist on a hotel room door during checkout. The guest cannot leave until housekeeping signs off — unless someone breaks the process and eats the minibar charge.

WHY IT MATTERS IN PRODUCTION

Stuck `Terminating` namespaces and CRs are daily platform tickets. The fix is almost always find the controller that owns `example.com/cleanup`, read its logs, fix the external dependency, or acknowledge manual intervention.

Interviewers test whether you know forced removal is a break-glass action with audit consequences.

STEP BY STEP

- 1 Explain `deletionTimestamp`: object visible, finalizers block removal from `etcd`.
- 2 Reconcile branch: if deleting, run cleanup, patch finalizers minus yours, return.
- 3 Connect workqueue: delete triggers reconcile same as update; errors mean retry.
- 4 Describe failure modes: external API down, RBAC missing on status/finalizer patch.

EVIDENCE YOU WOULD QUOTE

```
oc get <crd> <name> -o jsonpath='{.metadata.deletionTimestamp}{"\n"}{.metadata.finalizers}'

oc patch <crd> <name> --type=merge -p '{"metadata":{"finalizers":[]}}' # break-glass only

grep -R 'Finalizer' controllers/ ; oc logs deploy/<operator> | grep -i finalizer
```

RED FLAG - DO NOT SAY THIS

Do not teach "remove the finalizer with a patch" as the normal fix. That is how orphaned load balancers and leaked volumes happen.

EXPECT THIS FOLLOW-UP

Namespace stuck `Terminating` with dozens of resources. What is your ordered triage?

PART 6 · Q65-Q76 · 12 QUESTIONS

Podman, Buildah, Skopeo and container images

Building and moving images without Docker — the toolchain RHEL and OpenShift CI actually standardise on

Red Hat's container stack is daemonless: Podman runs containers, Buildah builds them, Skopeo copies and inspects them, and UBI gives you a supportable base. Platform engineers meet this toolchain in CI pipelines, disconnected registries, rootless build hosts and the shift from Docker-in-Docker to Buildah-in-Kubernetes. Interviewers want practical answers — how subuid mapping works, why multi-arch manifests matter on heterogeneous clusters, and what you would check before trusting an image in production beyond the tag name.

Level mix - Foundation: 4 · Intermediate: 4 · Senior: 3 · Architect: 1

A container is a Linux process wearing five costumes

There is no "container" object in the kernel. Every isolation feature you see is a separate Linux mechanism, which is exactly why container problems are usually Linux problems.

Namespaces	What the process can see: PID, network, mount, UTS, IPC, user
cgroups v2	What the process can use: CPU shares and quota, memory limit, IO and PID caps
Union filesystem	What the process reads: stacked read-only image layers plus a thin writable layer
SELinux + seccomp	What the process may do: MCS labels on files, syscall filtering, capability set
Image + registry	What the process is: a manifest of layers addressed by an immutable digest
Pod (Kubernetes)	The shared unit: one network namespace and shared volumes for one or more containers

Q65

FOUNDATION

Why is Podman daemonless, and what changes for platform engineers?

SAY THIS

Podman talks directly to the OCI runtime — crun or runc — and the storage library containers/storage. There is no long-lived root daemon holding sockets and state. Each podman command is a process that forks the runtime, which means no single point of compromise, simpler systemd integration via podman generate systemd and Quadlet, and straightforward rootless operation mapped to the calling UID. For platform engineers the shift is operational: CI no longer needs a privileged Docker socket mounted into the builder; OpenShift builds can use Buildah strategies; and node troubleshooting uses crictl and podman interchangeably on RHCOS for debugging, not docker ps.

THINK OF IT LIKE

Docker with a daemon is a restaurant with one head chef who must approve every order. Podman is a food truck — you order, it cooks, nobody waits on a central kitchen phone.

WHY IT MATTERS IN PRODUCTION

This is the default on RHEL 9 and OpenShift 4.x nodes run CRI-O, which shares the same storage and runtime ecosystem. Teams still saying "install Docker on the build server" raise eyebrows in 2025/2026 interviews because it reintroduces a root daemon and socket permissions problem Podman was designed to remove.

STEP BY STEP

- 1 Contrast architecture: Podman CLI → conmon + runtime + graph driver, no dockerd.
- 2 List platform wins: rootless, pod semantics compatible with kube, systemd units, no socket.
- 3 Mention compatibility: docker-compatible CLI flags, podman-docker package for alias.
- 4 Note boundaries: Podman builds images via Buildah under the hood; it is not a registry.

EVIDENCE YOU WOULD QUOTE

```
podman info | head -30
```

```
systemctl --user status podman.socket # rootless API socket if enabled
```

```
oc debug node/<node> -- chroot /host crictl ps ; chroot /host podman ps
```

RED FLAG - DO NOT SAY THIS

Do not say "Podman is just Docker with a different name". The missing daemon changes security, CI design and how you run builds on shared infrastructure.

EXPECT THIS FOLLOW-UP

How would you migrate a Jenkins pipeline from Docker socket to Buildah?

Q66

FOUNDATION

What is Buildah, and when do you use it instead of Podman build?

SAY THIS

Buildah constructs OCI images by committing container layers — from scratch, from an existing image, or via a Containerfile — without requiring a running container engine daemon. buildah bud builds from a Containerfile; buildah from starts an empty or base working container you mutate with run, copy, config, then commit. Use Buildah in CI when you want fine-grained scripting, multi-stage builds without extra privileges, or to build inside a locked-down namespace where podman build is too heavy. Podman build is Buildah under a convenience wrapper; in pipelines that only need docker build behaviour, either works, but Buildah is the lower-level tool operators embed in OpenShift S2I and custom build strategies.

THINK OF IT LIKE

Buildah is a pottery wheel — you shape each layer by hand. Podman build is the same wheel with a recipe card clipped to it.

WHY IT MATTERS IN PRODUCTION

OpenShift's built-in builds and many Tekton tasks use Buildah with chroot isolation and vfs or overlay mounts.

Interviewers ask this to see if you know why Dockerfile RUN instructions become buildah run plus commit, and why --isolation chroot matters in unprivileged CI.

STEP BY STEP

- 1 Define Buildah as OCI image construction without dockerd.
- 2 Contrast bud (Containerfile) vs from/run/commit (scripted layer assembly).
- 3 Explain typical CI invocation: buildah bud --layers --isolation chroot -t ...
- 4 State relationship to Podman and OpenShift BuildConfig strategies.

EVIDENCE YOU WOULD QUOTE

```
buildah --version ; buildah info
```

```
buildah bud -t localhost/test:latest .
```

```
oc get buildconfig -n <ns> -o yaml | grep -A5 strategy
```

RED FLAG - DO NOT SAY THIS

Do not assume Buildah requires root. Rootless builds are supported with user namespace mapping — that is usually the whole point in CI.

EXPECT THIS FOLLOW-UP

Build fails with permission denied on RUN yum install. What isolation settings do you check?

Q67**FOUNDATION****What is Red Hat UBI, and why do platform teams standardise on it?****SAY THIS**

Universal Base Image is Red Hat's freely redistributable container base built from RHEL content. UBI images — standard, minimal, micro, init — can be pulled, rebuilt and republished without extra licensing for your applications, while still receiving updates through registry.redhat.io or mirrored registries. Platform teams standardise on UBI because support boundaries are clear, SELinux and OpenShift compatibility are tested, and security scanners have consistent CVE data. Micro and minimal reduce attack surface for Go operators and static binaries; standard suits apps needing yum and common tools. The interview point is contractual: UBI is not "CentOS renamed"; it is the supported path for customer workloads on OpenShift.

THINK OF IT LIKE

UBI is a certified foundation slab. You can build any house on it and ship the house anywhere; the slab supplier still stands behind the concrete formula.

WHY IT MATTERS IN PRODUCTION

Enterprises mandate UBI in Dockerfile/Containerfile policy to avoid unlicensed RHEL layers or unmaintained alpine variants in regulated environments. Pair with image mirroring in disconnected sites and RHSA notifications tied to image streams.

STEP BY STEP

- 1 Define UBI tiers: ubi9/ubi, ubi-minimal, ubi-micro, init helpers.
- 2 State redistribution rights and update path via registry.redhat.io or mirror.
- 3 Contrast with alpine/busybox — glibc, RPM ecosystem, OpenShift support matrix.
- 4 Mention tooling: podman pull registry.redhat.io/ubi9/ubi-minimal, rpm -q in builds.

EVIDENCE YOU WOULD QUOTE

```
podman pull registry.redhat.io/ubi9/ubi-minimal:latest

skopeo inspect docker://registry.redhat.io/ubi9/ubi-minimal:latest | grep -i arch

oc get istag -n openshift | grep ubi
```

RED FLAG - DO NOT SAY THIS

Do not ship arbitrary yum repos into UBI in production images without a CVE review process. "Latest packages" and "reproducible builds" pull in opposite directions.

EXPECT THIS FOLLOW-UP

When would you choose ubi-micro over ubi-minimal for an operator image?

Q68

FOUNDATION

Containerfile versus Dockerfile — does the difference matter?

SAY THIS

Syntax is identical for practical purposes: FROM, RUN, COPY, ENTRYPOINT, multi-stage AS blocks all work in a Containerfile. Red Hat documentation prefers Containerfile to signal OCI-native tooling without implying a Docker daemon dependency. Buildah and Podman accept either filename; -f points to whichever you use. Platform standards often mandate Containerfile in repos for clarity and lint rules. Interview nuance: some Docker-specific directives — HEALTHCHECK with certain forms, legacy builder-only flags — may differ; test with buildah bud. For OpenShift builds, the strategy dockerfile path in BuildConfig accepts Containerfile when you set contextDir and dockerfile keys accordingly.

THINK OF IT LIKE

Containerfile is the same recipe card with the word "kitchen" crossed out and "food prep area" written in — the dish does not change.

WHY IT MATTERS IN PRODUCTION

Policy-as-code in CI sometimes fails builds named Dockerfile to push teams toward the Red Hat toolchain. The substantive interview topic is not the filename but whether the instructions are compatible with rootless Buildah and produce arbitrary-UID-safe images.

STEP BY STEP

- 1 State syntax parity and tooling acceptance: bud -f Containerfile .
- 2 Explain naming convention: Red Hat docs, OCI emphasis, enterprise policy.
- 3 Note BuildConfig/Tekton: reference path explicitly in task parameters.
- 4 Mention validation: buildah bud --layers, dive or syft for review, not the extension.

EVIDENCE YOU WOULD QUOTE

```
buildah bud -f Containerfile -t localhost/app:test .

ls -la Containerfile Dockerfile 2>/dev/null ; head Containerfile

oc set build-hook --help ; grep -i dockerfile oc/new-build --help
```

RED FLAG - DO NOT SAY THIS

Do not treat renaming Dockerfile to Containerfile as "migration to Podman". You still need rootless-safe instructions and a non-Docker CI executor.

EXPECT THIS FOLLOW-UP

Which Dockerfile patterns break under rootless Buildah?

Q69

INTERMEDIATE

What is Skopeo used for in a platform pipeline?

SAY THIS

Skopeo performs image and repository operations without building or running containers. skopeo copy moves images between registries, directories and docker-archive tarballs, preserving or converting formats. skopeo inspect reads manifest metadata without a full pull. skopeo sync bulk-replicates repositories — essential for disconnected OpenShift mirrors. skopeo list-tags and signverify support promotion gates. Platform teams use it in mirroring jobs, air-gapped installs, and pre-flight checks before deploying a digest to production. It shares containers/image and containers/storage with Podman and Buildah so transport and auth configuration is consistent.

THINK OF IT LIKE

If Buildah manufactures the product and Podman uses it, Skopeo is the shipping department — it moves boxes between warehouses without opening them.

WHY IT MATTERS IN PRODUCTION

Disconnected OpenShift installs depend on oc mirror plus skopeo copy to populate registry.redhat.io content on a local registry. Interviewers connect Skopeo to supply chain: copy by digest, verify signatures on inspect, and never rely on a tag that moved between staging and production.

STEP BY STEP

- 1 List core commands: copy, inspect, sync, delete, signverify.
- 2 Explain transports: docker://, containers-storage:, dir:, oci-archive:.
- 3 Describe disconnected mirroring: sync catalog → internal registry, preserve digests.
- 4 Mention auth: registries.conf, auth.json, pull secrets same as Podman.

EVIDENCE YOU WOULD QUOTE

```
skopeo inspect docker://registry.redhat.io/ubi9/ubi-minimal:latest | head -20

skopeo copy docker://src/image@sha256:... docker://dest/image@sha256:...

oc adm release mirror --help | head -5
```

RED FLAG - DO NOT SAY THIS

Do not use skopeo copy with a floating tag in production promotion. Copy the digest you tested or you have not promoted anything.

EXPECT THIS FOLLOW-UP

How do you mirror a multi-arch catalog for a disconnected cluster?

Q70

INTERMEDIATE

How do rootless container builds work on RHEL?

SAY THIS

Rootless Podman and Buildah run as an unprivileged user mapped into a user namespace where your UID appears as 0 inside the build container. Image layers land in ~/.local/share/containers/storage instead of /var/lib/containers. Network uses slirp4netns or pasta unless you have delegated capabilities. Builds that need package installs work because the inner root is fake; binds to host paths you cannot read still fail, which is the point. Platform teams run rootless builders on shared CI workers so a compromised pipeline cannot own the host. Limitations to know: some storage drivers behave poorly rootless, privileged RUN instructions are impossible, and subuid range size caps the number of nested IDs available.

THINK OF IT LIKE

Rootless build is a simulator cockpit. The controls feel real inside, but you are not actually flying the airline's plane.

WHY IT MATTERS IN PRODUCTION

OpenShift namespaces default to restricted-v2 SCC; builders there are non-root. Interviewers ask how you install RPMs in a Containerfile without docker build --privileged and whether you trust rootless images to run on OpenShift without random UID assignment.

STEP BY STEP

- 1 Explain user namespaces: outer UID mapped to inner 0 for build steps.
- 2 Point to storage location and graph root under the user's home.
- 3 List constraints: no true privileged ops, network namespace overhead, fuse-overlays on some setups.
- 4 Describe CI pattern: run buildah bud as CI user, push with skopeo copy and registry auth.

EVIDENCE YOU WOULD QUOTE

```
podman info --format '{{.Store.Root}}' ; id ; cat /etc/subuid

buildah bud --isolation chroot -t localhost/app:rootless .

grep user /etc/containers/containers.conf
```

RED FLAG - DO NOT SAY THIS

Do not disable user namespaces cluster-wide to "make builds work". Fix subuid allocation and storage driver choice instead.

EXPECT THIS FOLLOW-UP

Rootless build cannot bind-mount /var/run/docker.sock. Why is that a feature?

Q71

INTERMEDIATE

What are subuid and subgid, and what breaks when they are missing?

SAY THIS

/etc/subuid and /etc/subgid allocate ranges of subordinate IDs to a user. Rootless Podman maps UIDs inside the container to this range on the host so file ownership in layers is consistent and multiple containers do not collide. A typical entry is build:100000:65536 — start at 100000, length 65536. Without it, rootless podman fails at setup with cannot find UID/GID for user. Platform teams provisioning build hosts must automate subuid assignment in Ansible or cloud-init alongside disk quotas for ~/.local/share/containers. Nested ID exhaustion shows up as cannot allocate user namespace when ranges are too small or shared across too many users.

THINK OF IT LIKE

subuid is a block of parking spaces reserved for your badge. Without a reserved block you cannot park — you certainly cannot park fifty guest cars.

WHY IT MATTERS IN PRODUCTION

Shared CI runners are where this breaks in production. One golden image with wrong numeric ownership often traces back to building as root on a laptop and running rootless on OpenShift with arbitrary UID 1000680000. Interviewers want you to connect subuid to OpenShift's runAsUser strategy and fsGroup.

STEP BY STEP

- 1 Define subuid/subgid files and the three-field format: name, start, count.
- 2 Explain mapping: container UID 0 → host start+offset, keeping layer tar ownership sane.
- 3 List failure symptoms: podman info warnings, permission denied on volume mounts.
- 4 Describe provisioning: usermod --add-subuids, loginctl enable-linger for long-running rootless services.

EVIDENCE YOU WOULD QUOTE

```
cat /etc/subuid /etc/subgid

podman unshare cat /proc/self/uid_map

grep subid /etc/nsswitch.conf
```

RED FLAG - DO NOT SAY THIS

Do not hand-edit subuid ranges without checking for overlaps. Duplicate ranges corrupt ownership silently across users.

EXPECT THIS FOLLOW-UP

An image built rootless shows wrong ownership on OpenShift volumes. How do you fix the Dockerfile?

Q72

INTERMEDIATE

How do you run Buildah safely in CI and OpenShift pipelines?

SAY THIS

Avoid Docker-in-Docker and privileged daemon sockets. Preferred pattern: an unprivileged Tekton task or GitLab/Konflux job running buildah bud with `--isolation chroot` or `oci`, storage driver `vfs` or `overlay` in a writable `emptyDir`, and push via `buildah push` or `skopeo copy` using workspace registry credentials. Mount `/tmp` and `HOME` for layer cache if allowed. In OpenShift, use non-root SCC-compatible builders — custom pipeline namespace with `anyuid` only if policy demands and you understand the risk. Cache images with `--layers` and registry mirror. Set `STORAGE_DRIVER` explicitly when nodes lack fuse-overlays for rootless. Resource limits must cover peak layer extraction — OOM during yum is common.

THINK OF IT LIKE

Safe CI build is cooking in your own booth at the fair, with your own gas line — not borrowing the central kitchen's master key.

WHY IT MATTERS IN PRODUCTION

Many breaches traced to privileged CI containers with `docker.sock` mounted. Interviewers want Tekton/Konflux specifics: buildah task from Red Hat catalogs, workload identity for registry push, and separating build from deploy signing stages.

STEP BY STEP

- 1 Reject privileged DinD; choose rootless buildah in isolated namespace.
- 2 Configure storage: `emptyDir` graph root, `vfs` if overlay unavailable, layer cache policy.
- 3 Wire auth: `dockercfg` or `imagePullSecret` mounted for push.
- 4 Split pipeline: build → scan → sign → promote, each stage pinned by digest.

EVIDENCE YOU WOULD QUOTE

```
grep -R buildah .tekton/ .gitlab-ci.yml

oc get task buildah -n openshift-pipelines -o yaml | head -40

buildah bud --isolation chroot --storage-driver vfs -t $IMAGE .
```

RED FLAG - DO NOT SAY THIS

Do not mount `/var/run/docker.sock` "because it is easier". You have given the pipeline root on the host.

EXPECT THIS FOLLOW-UP

Buildah task OOMKills on `RUN dnf install`. What do you tune?

Q73

SENIOR

How do you build and publish multi-architecture images in 2025?

SAY THIS

Multi-arch images are manifest lists: one tag points to per-architecture manifests — amd64, arm64, s390x, ppc64le — each with its own digest. buildah manifest create starts a list; buildah manifest add builds or adds each arch; buildah manifest push publishes to the registry. Podman manifest commands mirror this. In CI, either native builders per arch fan-in with skopeo or buildah manifest, or cross-build with QEMU binfmt_misc registration — slower but one pipeline. OpenShift heterogeneous clusters — IBM Z, ARM edge, mixed worker pools — require manifest lists or pods fail ImagePullBackOff with no matching manifest. Always push the list, not individual arch tags, to production tags.

THINK OF IT LIKE

A multi-arch manifest is a menu in three languages. The dish name is the same; the kitchen prepares the version each guest can actually eat.

WHY IT MATTERS IN PRODUCTION

Apple Silicon laptops building amd64-only images that work locally but fail in CI is a classic gotcha. Platform interviews in 2025/2026 expect cosign sign on the manifest list index digest so signature covers all architectures.

STEP BY STEP

- 1 Explain manifest list vs single-arch manifest and registry resolution.
- 2 Walk buildah manifest workflow: create, add per arch, annotate, push --all.
- 3 Compare native multi-node CI vs QEMU cross-build trade-offs.
- 4 Verify with skopeo inspect --raw and oc adm release info for platform support.

EVIDENCE YOU WOULD QUOTE

```
buildah manifest create localhost/app:multi ; buildah manifest add ... ; buildah manifest push --all
```

```
skopeo inspect docker://quay.io/<org>/<image>:latest | jq '.Manifests'
```

```
podman run --rm mplatform/mquery quay.io/<org>/<image>:latest
```

RED FLAG - DO NOT SAY THIS

Do not docker push from a single-arch laptop and call it multi-arch. You have built one flavour and mislabelled the menu.

EXPECT THIS FOLLOW-UP

How do you sign a manifest list so all architectures are covered?

Q74

SENIOR

How should platform teams implement container image signing?

SAY THIS

Signing binds a cryptographic identity to an image digest. Cosign from Sigstore is the common choice: `cosign generate-key-pair` or `keyless` with OIDC in CI, `cosign sign registry.example.com/app@sha256:...`, `cosign verify` with public key or Fulcio certificate chain. Red Hat and Quay support signing integration; OpenShift can enforce signature policy via `ClusterImagePolicy` or admission configuration referencing sigstore policy documents. Platform pattern: CI builds image, pushes by digest, signs the digest, promotion copies only signed digests to production registry. Tags are signed only if you accept race conditions; digest signing is the durable approach. Rotate keys, audit verify failures, and mirror signatures with skopeo copy when air-gapped.

THINK OF IT LIKE

Signing is a wax seal on a specific letter, not on the word "letter" in general. Anyone can reuse the word; they cannot reuse the seal on different paper.

WHY IT MATTERS IN PRODUCTION

Supply chain interviews post-SLSA and EO 14028 expect digest-pinning plus verify before deploy. OpenShift 4.14+ image policy APIs connect to this; know the difference between Red Hat signed catalog images and your own app images signed in Tekton.

STEP BY STEP

- 1 State object of signing: digest, not mutable tag.
- 2 Outline cosign sign/verify flow in CI and at admission.
- 3 Mention keyless OIDC for short-lived certificates vs long-lived KMS keys.
- 4 Describe air-gap: copy signatures with skopeo, distribute public keys or CA roots.

EVIDENCE YOU WOULD QUOTE

```
cosign sign --key cosign.key registry.example.com/app@sha256:abc...
cosign verify --key cosign.pub registry.example.com/app@sha256:abc...
oc get clusterimagepolicy ; oc adm policy images --help
```

RED FLAG - DO NOT SAY THIS

Do not verify tags in production policy. Tags move; signatures on tags lie the moment someone retags.

EXPECT THIS FOLLOW-UP

Admission rejects a Red Hat catalog image that worked yesterday. What changed?

Q75

SENIOR

What role do SBOMs play in container supply chain security?

SAY THIS

A Software Bill of Materials lists packages and libraries inside an image — RPMs, pip modules, Go modules — tied to a specific digest. Tools like syft generate SPDX or CycloneDX SBOMs; gype or Trivy scan them for CVEs. Platform teams attach SBOMs as build artifacts, store them in OCI referencers or alongside releases, and feed VEX statements when vulnerabilities are not exploitable. Red Hat provides SBOM metadata for UBI and certified operators; your application images need their own. In interviews connect SBOM to policy: block critical CVEs at deploy, waivers with expiry, and reproducible builds so SBOM matches what ran.

THINK OF IT LIKE

An SBOM is the ingredients list on packaged food. The health inspection does not eat the meal — it checks whether a known allergen is listed.

WHY IT MATTERS IN PRODUCTION

Enterprise customers request SBOM exports for every release. OpenShift compliance operator and Konflux supply chain pipelines automate generation in Red Hat's ecosystem. **Know the limit: an SBOM is only as honest as the build that produced it — hermetic builds matter.**

STEP BY STEP

- 1 Define SBOM purpose: inventory for vulnerability and license management.
- 2 Name generators and formats: syft → SPDX/CycloneDX; scan with gype/trivy.
- 3 Explain storage: CI artifact, OCI attachment/ referencer, release portal.
- 4 Link to policy: CVE gates, VEX, UBI RHSA correlation, not checkbox compliance.

EVIDENCE YOU WOULD QUOTE

```
syft registry.example.com/app@sha256:abc... -o spdx-json
```

```
gype sbom:./app.spdx.json
```

```
skopeo inspect docker://registry.example.com/app@sha256:abc... | jq '.Labels'
```

RED FLAG - DO NOT SAY THIS

Do not treat SBOM generation as security by itself. Without scanning, policy enforcement and signed digests, it is paperwork.

EXPECT THIS FOLLOW-UP

A CVE hits a base image package. Walk through patch, rebuild and redeploy with SBOM update.

Q76

ARCHITECT

What storage drivers matter for Podman and Buildah, and how do crun and runc differ on RHEL?

SAY THIS

Storage drivers control how image layers map to disk: overlay or overlayfs is default on RHEL with kernel support; vfs is slow but works everywhere including rootless without fuse-overlayfs; zfs and btrfs exist but are rare in enterprise. containers-storage picks driver via storage.conf — graphroot, runroot, mountopt for metacopy and force_mask. Wrong driver choice causes "cannot mount overlay" in CI or excessive inode use with vfs. For runtime, crun is the default low-level OCI runtime on RHEL 9 and RHCOS: written in C, fast startup, cgroup v2 aware, better for dense pods. runc remains available and is the reference implementation. CRI-O on OpenShift selects crun; podman --runtime crun matches node behaviour. Differences matter for seccomp, systemd cgroup delegation, and KPIs like pod start latency — not day-to-day Dockerfile authoring.

THINK OF IT LIKE

Storage drivers are filing systems for stacked transparencies; crun versus runc is choosing which motor spins the projector — same film, different startup time and fuel use.

WHY IT MATTERS IN PRODUCTION

Node incidents with "no space left on device" often trace to overlay whiteouts and abandoned images in /var/lib/containers, not Kubernetes itself. Architects compare crun vs runc when vendors ask for runc-specific workarounds or when debugging systemd unit pods on RHEL 9.

STEP BY STEP

- 1 Name drivers: overlay preferred, vfs fallback, read storage.conf graphroot.
- 2 Explain rootless overlay needs fuse-overlayfs or native overlay mount options.
- 3 Contrast crun vs runc: performance, cgroup v2, default on RHEL 9/RHCOS/CRI-O.
- 4 Tie to operations: podman system prune, crictl pull failures, monitor /var/lib/containers.

EVIDENCE YOU WOULD QUOTE

```
grep -E 'driver|graphroot' /etc/containers/storage.conf

crun --version ; runc --version

oc debug node/<node> -- chroot /host crictl info | grep runtime
```

RED FLAG - DO NOT SAY THIS

Do not switch an entire CI fleet to vfs without sizing disk and IOPS. It fixes mounts and melts performance.

EXPECT THIS FOLLOW-UP

Nodes fill /var/lib/containers despite image garbage collection. What is your cleanup order?

PART 7 · Q77-Q88 · 12 QUESTIONS

Bare metal provisioning

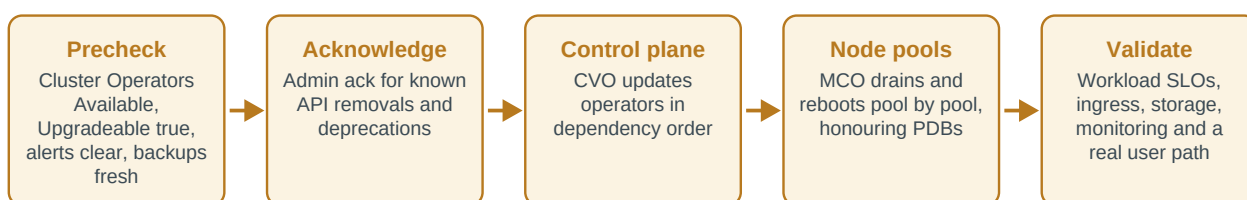
From BMC power-on to a node joining the cluster — Satellite, PXE, Kickstart and Ironi

Bare metal OpenShift clusters do not appear from an API call. Someone must power the machine on, deliver a boot image over the network, install RHEL with the right Ignition payload, and only then let the installer or Bare Metal Operator hand the node to the cluster. Interviewers probe this area because it separates people who have run OpenShift from people who have run the platform underneath it. The answers that land well connect Satellite, Capsules, DHCP and TFTP to a node that actually appears in `oc get nodes` — and explain what breaks when any hop in that chain is wrong.

Level mix - Foundation: 4 · Intermediate: 5 · Senior: 2 · Architect: 1

A supported OpenShift upgrade, in order

Upgrades are the highest-risk routine change you own. The order is not negotiable and "rollback" is not a step that exists.



Q77

FOUNDATION

What role does Red Hat Satellite play in bare metal provisioning?

SAY THIS

Satellite is the content and lifecycle hub for RHEL in the datacentre. It mirrors Red Hat repositories, publishes custom content views, manages subscriptions and activation keys, and hosts Kickstart templates and discovery images that turn a blank server into an installed host. For OpenShift bare metal you use it to standardise the OS build — same packages, same kernel, same hardening — before Ignition or the installer takes over.

THINK OF IT LIKE

It is the central kitchen that prepares identical meal kits. Every server gets the same recipe and ingredients; the local restaurant only plates and serves.

WHY IT MATTERS IN PRODUCTION

Satellite rarely installs OpenShift itself, but it owns everything up to that point. Teams that skip it often end up with hand-built Kickstarts, drifting package sets and no audit trail of which build a node received. For interview purposes, Satellite is the answer to 'how do we make a thousand blades identical before they join the cluster?'

STEP BY STEP

- 1 Explain content mirroring, subscriptions and activation keys.
- 2 Describe Kickstart and discovery image hosting as the provisioning entry point.
- 3 Connect a finished RHEL host to OpenShift via Ignition or the Bare Metal Operator.
- 4 Mention Capsules when provisioning happens in remote sites.

EVIDENCE YOU WOULD QUOTE

```
hammer content-view list ; hammer activation-key list
```

```
hammer host list --search 'name-worker'
```

```
On the node: cat /etc/redhat-release ; rpm -qa | wc -l
```

RED FLAG - DO NOT SAY THIS

Do not describe Satellite as optional once you are at scale. Ad-hoc installs are how production clusters acquire silent drift.

EXPECT THIS FOLLOW-UP

How would you promote a tested Kickstart from dev to production in Satellite?

Q78**FOUNDATION****How does PXE network boot work with PXELinux and PXEGrub2?****SAY THIS**

Preboot eXecution Environment lets a server with no local OS fetch a boot loader over the network. The firmware requests DHCP option 66/67 or equivalent, learns the TFTP server and boot file name, downloads the loader via TFTP, and that loader fetches a kernel and initramfs. PXELinux is the legacy BIOS chain loader; PXEGrub2 is the UEFI-aware Grub2 image used on modern hardware. Both end at the same place — a Linux kernel running an installer or discovery image.

THINK OF IT LIKE

PXE is a courier who delivers the key to the front door. PXELinux or PXEGrub2 is which locksmith cut the key — different shape, same door.

WHY IT MATTERS IN PRODUCTION

Interviewers care because the boot chain is where provisioning fails most often, and the failure looks like 'machine hangs at DHCP' rather than an OpenShift error. BIOS versus UEFI determines which loader you publish, and mixing them up produces a silent hang after TFTP starts. Capsule servers in remote sites must serve the same boot artifacts as the main Satellite or nodes there will never match the standard build.

STEP BY STEP

- 1 State the sequence: DHCP assignment, TFTP fetch of loader, loader fetches kernel/initramfs.
- 2 Distinguish PXELinux for legacy BIOS from PXEGrub2 for UEFI.
- 3 Point to the Kickstart or discovery image as the payload the kernel runs.
- 4 Verify firmware boot mode matches the published loader before debugging OpenShift.

EVIDENCE YOU WOULD QUOTE

```
On the provisioning host: ls /var/lib/tftpboot/pxelinux.cfg/ ; ls /var/lib/tftpboot/grub2/

tcpdump -i <iface> port 67 or port 69 during a boot attempt

Journal on Capsule: journalctl -u foreman-proxy-dns -u dhcpd -u tftpd
```

RED FLAG - DO NOT SAY THIS

Do not publish a PXELinux layout to a UEFI-only server. It will never chain-load.

EXPECT THIS FOLLOW-UP

The server gets an IP but stops after TFTP. What do you check next?

Q79

FOUNDATION

What do DHCP, TFTP and DNS each contribute to network boot?

SAY THIS

DHCP assigns the machine an IP address and, critically, tells it where to boot from — the next-server address and boot file name via options 66 and 67 or the UEFI DHCP options. TFTP is the trivial file transfer service that actually delivers the small boot loader and initial files because nothing else is running yet. DNS provides the names the installed host and the OpenShift cluster will need — api, api-int, apps wildcard, and forward and reverse records for each node — so the machine can resolve endpoints after the OS is up.

THINK OF IT LIKE

DHCP is the reception desk that assigns a desk number and directions. TFTP is the lift that carries your bags to the floor. DNS is the building directory you need once you are inside.

WHY IT MATTERS IN PRODUCTION

These three services are owned by different teams in most enterprises, which is why provisioning runbooks spend pages on hand-offs. A working DHCP scope with no TFTP server gives you an IP and a hang. TFTP with no DNS leaves you with an installed host that cannot join OpenShift because api-int does not resolve. Senior answers name the owner of each service and the record each one needs.

STEP BY STEP

- 1 DHCP: IP lease plus next-server and bootfile options matched to BIOS or UEFI.
- 2 TFTP: boot loader, kernel and initramfs reachable from the provisioning VLAN.
- 3 DNS: api, api-int, apps wildcard and per-node forward and reverse before install completes.
- 4 Validate all three from a test NIC on the provisioning network before the first production boot.

EVIDENCE YOU WOULD QUOTE

```
dig api.<cluster>.<domain> ; dig api-int.<cluster>.<domain>

dhcpd -t -cf /etc/dhcp/dhcpd.conf # syntax check before reload

tftp <tftp-server> -c get grub/grubx64.efi /tmp/test.efi
```

RED FLAG - DO NOT SAY THIS

Do not assume DNS can wait until after install. The OpenShift bootstrap and node join process need working records immediately.

EXPECT THIS FOLLOW-UP

Which DHCP options differ between legacy BIOS PXE and UEFI HTTP boot?

Q80

FOUNDATION

What is Kickstart and how does it automate RHEL installation?

SAY THIS

Kickstart is an unattended installation answer file. It declares partitioning, packages, network settings, users, firewall posture and post-install scripts so Anaconda installs RHEL without human input. Satellite hosts Kickstart templates parameterised with host-specific values — hostname, IP, activation key — and passes them to the booting kernel via the ks= URL. For OpenShift the Kickstart usually ends with a first-boot script or Ignition fetch that prepares the node for the installer or Bare Metal Operator.

THINK OF IT LIKE

It is a detailed shopping list and assembly instructions left on the counter. The installer follows it exactly, every time, without asking questions.

WHY IT MATTERS IN PRODUCTION

Kickstart is where OS standardisation lives. Two teams with different %packages sections produce two different clusters that both claim to run the same OpenShift version. Interviewers want to hear about idempotency — the same Kickstart with different host variables — and about %post scripts that hand off to cluster join rather than doing OpenShift installation inside Kickstart itself.

STEP BY STEP

- 1 Define sections: lang, network, bootloader, partitioning, %packages, %post.
- 2 Bind the template to an activation key and content view in Satellite.
- 3 Pass ks= at boot time so Anaconda pulls the rendered file.
- 4 Use %post to fetch Ignition, register subscriptions or trigger discovery.

EVIDENCE YOU WOULD QUOTE

```
hammer host list --search 'build_status=failed'
```

```
On installing host: tail -f /tmp/anaconda.log
```

```
Rendered Kickstart: curl -k https://<satellite>/unattended/provision?token=<token>
```

RED FLAG - DO NOT SAY THIS

Do not embed secrets in plain-text Kickstarts. Use Satellite parameters or vault integration.

EXPECT THIS FOLLOW-UP

A Kickstart succeeds but the node has the wrong OpenShift kernel. Where do you look?

Q81**INTERMEDIATE****What is a BMC and how do IPMI and Redfish differ?****SAY THIS**

The Baseboard Management Controller is an out-of-band management chip on the server that controls power, reads sensors and exposes virtual media and serial consoles independently of the main CPU. IPMI is the older binary protocol, usually on a dedicated management NIC, used by tools such as ipmitool and by Ironic for power and boot-order control. Redfish is the modern RESTful API over HTTPS with JSON payloads, better suited to automation, firmware inventory and virtual media uploads on current hardware.

THINK OF IT LIKE

The BMC is the building manager with keys to every switch. IPMI is the old phone line to the manager; Redfish is the manager's web portal.

WHY IT MATTERS IN PRODUCTION

Bare metal OpenShift depends on reliable out-of-band control. The Bare Metal Operator talks to Ironic, which talks to the BMC to power on a server, set one-time PXE boot and read hardware inventory. Interviewers probe whether you know that management network isolation is non-negotiable — a BMC on the production LAN is a critical vulnerability — and that Redfish support varies by vendor even when the logo is on the HCL.

STEP BY STEP

- 1 Define BMC role: power, boot order, sensors, virtual media, serial over LAN.
- 2 Contrast IPMI CLI tooling with Redfish REST endpoints.
- 3 Explain Ironic's use of the BMC for provisioning and deprovisioning cycles.

4 Isolate management traffic on a dedicated VLAN with strict ACLs.**EVIDENCE YOU WOULD QUOTE**

```
ipmitool -I lanplus -H <bmc-ip> -U admin power status

curl -k -u admin:pass https://<bmc-ip>/redfish/v1/Systems/1

oc get baremetalhost -n openshift-machine-api -o wide
```

RED FLAG - DO NOT SAY THIS

Do not put BMC interfaces on an untrusted network. They have full power over the server below the OS.

EXPECT THIS FOLLOW-UP

Ironic cannot power on a registered host. How do you isolate BMC versus Ironic?

Q82**INTERMEDIATE****How does the OpenShift bare metal discovery service work?****SAY THIS**

Discovery is an assisted onboarding path for user-provisioned bare metal. You boot unknown hardware with a discovery ISO — often built from Satellite or the OpenShift assisted installer image — which collects inventory, tests connectivity and reports hardware details back to a discovery server or hub. Once validated, that inventory becomes a BareMetalHost definition Ironic can manage, with MAC addresses, BMC credentials and firmware notes captured before anyone writes a custom YAML by hand.

THINK OF IT LIKE

It is a job interview before the first day. You learn what the candidate actually is before assigning them a desk.

WHY IT MATTERS IN PRODUCTION

Discovery saves weeks when you have heterogeneous hardware or remote hands who cannot run oc commands. The failure mode is booting the wrong ISO on the wrong VLAN — discovery registrations appear in the hub but BMC addresses are unreachable from the provisioning network. Interviewers reward candidates who connect discovery to inventory accuracy for Ironic rather than treating it as a separate installer gimmick.

STEP BY STEP

- 1 Build or download the discovery ISO aligned to the OpenShift version.
- 2 Boot target servers from ISO or PXE; agents report inventory to the discovery service.
- 3 Review CPU, NIC, disk and firmware data; fix BMC or network gaps.
- 4 Approve hosts for provisioning; Ironic registers them as BareMetalHost objects.

EVIDENCE YOU WOULD QUOTE

```
Discovery hub UI or API: list discovered hosts and validation status

oc get baremetalhost -n openshift-machine-api -o yaml | grep -A5 'hardwareDetails'

On discovery agent host: journalctl -u agent -- or equivalent agent log
```

RED FLAG - DO NOT SAY THIS

Do not skip validating BMC reachability from the provisioning network. Discovery inventory without working out-of-band control cannot provision.

EXPECT THIS FOLLOW-UP

Discovery shows the wrong NIC for provisioning. How do you fix it before approval?

Q83**INTERMEDIATE****What are Capsule servers in Satellite and when do you need them?****SAY THIS**

A Capsule is a Satellite proxy deployed close to the servers it serves — a remote datacentre, edge site or high-latency network. It runs local DNS, DHCP, TFTP, Puppet or Ansible callbacks and content caching so machines do not pull boot files or RPMs across a WAN link. The main Satellite remains the control plane; Capsules sync content views and report back. You need them when provisioning VLANs cannot reach the central Satellite directly or when local PXE latency and bandwidth would make boot unreliable.

THINK OF IT LIKE

Satellite is headquarters. A Capsule is the regional warehouse — same catalogue, shorter delivery route.

WHY IT MATTERS IN PRODUCTION

OpenShift bare metal at multiple sites almost always implies Capsules. The interview trap is forgetting that DHCP and TFTP must be authoritative locally — pointing remote servers at a central TFTP host over a congested link produces intermittent PXE failures that look like bad hardware. Capsule sizing also matters: hundreds of simultaneous builds need adequate disk for synced repos and TFTP roots.

STEP BY STEP

- 1 Identify sites where servers cannot reach central Satellite services reliably.
- 2 Deploy a Capsule with synced content views and local DHCP, DNS and TFTP roles.
- 3 Register the Capsule with the main Satellite and promote the same content views.
- 4 Point provisioning subnets at the Capsule's next-server and TFTP paths.

EVIDENCE YOU WOULD QUOTE

```
hammer capsule list ; hammer capsule content list --id <id>
```

```
On Capsule: foreman-proxy status ; ls /var/lib/tftpboot/
```

```
From remote site: time tftp <capsule-ip> -c get pxelinux.0 /tmp/test
```

RED FLAG - DO NOT SAY THIS

Do not sync a Capsule manually and forget content view promotion. Stale boot images are how remote sites run the wrong OpenShift version.

EXPECT THIS FOLLOW-UP

Central Satellite is up but remote PXE fails. Walk me through Capsule troubleshooting.

Q84

INTERMEDIATE

How does OpenShift integrate with vSphere for machine provisioning?

SAY THIS

On vSphere, OpenShift uses the Machine API with a vSphere provider: MachineSets define template, folder, resource pool, network and disk layout, and the machine controller clones VMs from an OVA or template with Ignition injected via guestinfo. It is not bare metal, but it fills the same platform role — automated node lifecycle with the installer or IPI creating load balancers, DNS records and machines. UPI on vSphere means you create the VMs yourself but still use the same Ignition and RHCOS/RHEL boot process.

THINK OF IT LIKE

Bare metal provisioning is hiring craftsmen on site. vSphere integration is ordering prefabricated modules from a factory — still your cluster, different supply chain.

WHY IT MATTERS IN PRODUCTION

Interviewers ask this in bare metal conversations because many 'on-prem OpenShift' roles are hybrid: control plane on VMs, workers on metal, or the reverse. The integration points — templates with the right Ignition, DHCP and DNS for API endpoints, storage classes backed by vSAN — are the same services bare metal needs, just delivered by the hypervisor instead of PXE. Knowing when to choose full metal versus vSphere workers is an architect signal.

STEP BY STEP

- 1 Prepare a VM template with guestinfo Ignition support and compatible hardware version.
- 2 Configure install-config or MachineSet with vCenter credentials, datacentre and network.
- 3 Ensure DNS and load balancers for api and apps resolve before install.
- 4 For hybrid clusters, label and taint metal workers separately from VM workers.

EVIDENCE YOU WOULD QUOTE

```
oc get machineset -n openshift-machine-api -o yaml | grep -i vsphere

oc get machine -A -o wide

vCenter: monitor clone tasks tied to machine names
```

RED FLAG - DO NOT SAY THIS

Do not treat vSphere templates like generic Linux images. Missing Ignition guestinfo support produces nodes that boot but never join.

EXPECT THIS FOLLOW-UP

When would you choose bare metal workers over vSphere VMs for OpenShift?

Q85

INTERMEDIATE

What does the Bare Metal Operator do with Ironic?

SAY THIS

The Bare Metal Operator is the Kubernetes controller that manages BareMetalHost custom resources — registering hardware, validating BMC credentials, driving provisioning states and handing enrolled nodes to the Machine API. Ironic is the OpenStack bare metal service that actually talks to BMCs, builds deploy ramdisks, writes disk images and toggles boot devices. In OpenShift, Ironic runs on the bootstrap or control plane; the operator translates BareMetalHost specs into Ironic API calls and surfaces status back to oc.

THINK OF IT LIKE

BareMetalHost is the work order on the clipboard. Ironic is the technician in the datacentre with the cart of tools.

WHY IT MATTERS IN PRODUCTION

This is the centre of native bare metal IPI on OpenShift 4. Candidates who only know Satellite Kickstart miss the day-two story: deprovisioning, reprovisioning, firmware RAID and the inspection phase before a host is trusted. Ironic's provisioning network must reach the BMC and the host NIC used for PXE, which is a common design review topic.

STEP BY STEP

- 1 Define BareMetalHost with BMC address, credentials, MAC and provisioning flags.
- 2 Operator registers the host; Ironic inspects hardware if enabled.
- 3 Provisioning state: Ironic powers on, PXE boots deploy ramdisk, writes RHCOS image.
- 4 Host becomes a Machine; cluster-operator completes node configuration via MachineConfig.

EVIDENCE YOU WOULD QUOTE

```
oc get baremetalhost,bmh -n openshift-machine-api
```

```
oc describe baremetalhost <name> -n openshift-machine-api | grep -i state
```

```
On provisioning node: podman logs -f ironic-conductor # or equivalent Ironic log
```

RED FLAG - DO NOT SAY THIS

Do not register BMC credentials in Git. Use secrets referenced by the BareMetalHost.

EXPECT THIS FOLLOW-UP

A BareMetalHost is stuck in provisioning. Which states do you walk through?

Q86

SENIOR

What is UEFI HTTP boot and why is it replacing traditional PXE?

SAY THIS

UEFI HTTP boot lets firmware download boot loaders and kernels over HTTP or HTTPS instead of TFTP. DHCP still provides the initial network configuration, but options now point to a URI such as `http://capsule/boot/grubx64.efi` rather than a TFTP path. HTTPS adds integrity and works better across routed networks and firewalls than TFTP's small-packet UDP model. Large initramfs images and modern security expectations make HTTP boot the default on current servers; Satellite and Capsules publish both TFTP and HTTP endpoints during the transition.

THINK OF IT LIKE

TFTP is a bicycle courier with a small bag. HTTP boot is a van that can carry the whole stage set and confirm delivery with a signature.

WHY IT MATTERS IN PRODUCTION

Enterprises refreshing hardware hit this during OpenShift upgrades: new UEFI-only boxes that never successfully TFTP a full RHCOS ramdisk. Interviewers want you to mention certificate trust in firmware, proxy settings and the need to update DHCP snippets — not just 'turn on HTTP boot'. **Mixed fleets often run HTTP for UEFI and TFTP for legacy BIOS until the last old server retires.**

STEP BY STEP

- 1 Enable HTTP boot in firmware; confirm firmware trusts the serving certificate or use HTTP on an isolated provisioning VLAN.
- 2 Publish Grub2 and kernel artifacts on Capsule or Satellite HTTP endpoints.
- 3 Update DHCP to supply the HTTP URI via vendor-specific UEFI options.
- 4 Validate boot alongside existing TFTP paths during a hardware refresh migration.

EVIDENCE YOU WOULD QUOTE

```
curl -I http://<capsule>/pub/grubx64.efi # artifact reachable
```

```
tcpdump -i <vlan> port 67 or port 80 during boot
```

UEFI firmware boot log — screenshot or IPMI serial capture of HTTP fetch status

RED FLAG - DO NOT SAY THIS

Do not disable TFTP on day one when legacy BIOS servers still exist. Run both chains until inventory confirms otherwise.

EXPECT THIS FOLLOW-UP

HTTP boot fails with certificate errors in firmware. What are your options?

Q87**SENIOR****A bare metal node fails to join the cluster after provisioning. How do you troubleshoot?****SAY THIS**

I work backwards through the chain. First confirm the OS is actually the expected build — correct kernel, ignition applied, subscription valid. Then DNS: api-int must resolve from the node. Then network: can the node reach the API on 6443 and the machine config server? Then certificates: pending CSRs for kubelet and node client. Only after those pass do I look at Ironi/BareMetalHost state, Machine object conditions and finally kubelet logs on the node itself.

THINK OF IT LIKE

The patient is in the building but not at the meeting. Check they have the right badge, know the room number and the lift works — before questioning their presentation skills.

WHY IT MATTERS IN PRODUCTION

The classic mistake is jumping to OpenShift logs when the node still has the wrong IP or no reverse DNS. Another is provisioning success in Ironi while Ignition never ran, leaving ssh keys and hostname wrong. **Senior candidates narrate a ordered checklist and say which team owns each failure — DNS team, network team, platform team — because bare metal incidents are almost always cross-functional.**

STEP BY STEP

- 1 Verify OS build, Ignition completion and subscription on the node console or SSH.

- 2 From the node: `dig api-int.<cluster>.<domain> ; curl -k https://api-int:6443/healthz`
- 3 Check `oc get csr` and `oc get machine <name> -o yaml` for failure conditions.
- 4 Review BareMetalHost provisioning state and Ironic logs if the node never reached OS.

EVIDENCE YOU WOULD QUOTE

```
oc get nodes ; oc get csr | grep Pending

On node: journalctl -u kubelet --no-pager | tail -50

oc describe machine <name> -n openshift-machine-api
```

RED FLAG - DO NOT SAY THIS

Do not re-provision immediately. You may destroy evidence and repeat the same misconfigured Kickstart or DNS gap.

EXPECT THIS FOLLOW-UP

Ironic says provisioned but `oc get nodes` never shows the host. Where is it stuck?

Q88**ARCHITECT****How would you design bare metal provisioning for OpenShift at enterprise scale?****SAY THIS**

I separate concerns into layers: Satellite and Capsules for OS standardisation and content promotion; isolated provisioning and BMC networks with local DHCP, TFTP and HTTP; discovery for inventory; Ironic and the Bare Metal Operator for cluster-native lifecycle; and GitOps for BareMetalHost and MachinePool definitions. Each site gets a Capsule, shared activation keys and content views, and a tested promotion path from lab to production. Rollouts are canaried — one rack, one row, one site — with automated validation that DNS, pull secret and Ignition match before the next wave.

THINK OF IT LIKE

It is an airport hub system: central timetable, regional gates, identical boarding process, local ground crew — not one overwhelmed check-in desk for every flight worldwide.

WHY IT MATTERS IN PRODUCTION

Architect answers acknowledge organisational reality: networking owns DHCP, security owns BMC VLANs, Linux owns Kickstart, platform owns OpenShift. The design artefact interviewers want is a reference architecture diagram with trust boundaries, not a list of tools. Scale also means decommissioning and firmware lifecycle — provisioning is not a one-way door.

STEP BY STEP

- 1 Map sites, Capsule placement and network VLANs for provisioning, BMC and production.
- 2 Standardise content views, Kickstart and discovery ISO promotion across environments.
- 3 Automate BareMetalHost registration from discovery with sealed BMC secrets.
- 4 Define validation gates — DNS, API reachability, node Ready — before fleet expansion.

EVIDENCE YOU WOULD QUOTE

```
Reference architecture doc: VLANs, Capsule map, promotion workflow

hammer content-view version list --content-view Production-RHEL

Metrics: provisioning success rate, time-to-Ready, failed BMC percentage by site
```

RED FLAG - DO NOT SAY THIS

Do not centralise all TFTP and DHCP on one Satellite without redundancy. A single outage freezes every rack.

EXPECT THIS FOLLOW-UP

How do you handle firmware upgrades across thousands of BMCs without breaking Ironic?

PART 8 · Q89-Q100 · 12 QUESTIONS

HPC and AI on OpenShift

GPUs, RDMA, MPI and the operators that turn a Kubernetes cluster into an AI factory

Running AI on OpenShift is not as simple as scheduling a pod with `nvidia.com/gpu`. The NVIDIA GPU Operator installs drivers and the container toolkit cluster-wide; Node Feature Discovery labels the hardware; ClusterPolicy is the single knob that must match your OpenShift version; and training jobs need RDMA, NUMA alignment and MPI launcher patterns that general workloads never touch. Interviewers use this part to see whether you have operated a GPU cluster or only read the documentation — the difference shows up in how you talk about placement, monitoring and what breaks when someone upgrades the operator without a rollback plan.

Level mix - Foundation: 3 · Intermediate: 5 · Senior: 3 · Architect: 1

Why an HPC pod is fast or slow: alignment, not tuning

Latency-sensitive workloads do not need more resources, they need the same NUMA node. Every layer here has to agree or you pay for a cross-socket hop on every packet.

Guaranteed QoS	Integer CPU requests equal to limits - the entry ticket for exclusive CPUs
CPU Manager	Static policy pins the container to dedicated cores; no noisy-neighbour throttling
Memory + huge pages	Pre-allocated huge pages on the same NUMA node reduce TLB misses
Device alignment	SR-IOV VF or GPU must sit on the same NUMA node as the CPUs and memory
Topology Manager	single-numa-node policy refuses the pod rather than silently misaligning it
PerformanceProfile	Isolated and reserved core split, kernel args and tuned profile, applied via MCO

Q89

FOUNDATION

What does the NVIDIA GPU Operator install and manage on OpenShift?

SAY THIS

The GPU Operator is a bundle of controllers and DaemonSets that automate everything required to run CUDA workloads on OpenShift without hand-installing drivers on each node. It deploys the NVIDIA driver container, the device plugin that advertises `nvidia.com/gpu` resources, the container toolkit so runtimes inject GPU libraries, optional DCGM for monitoring, MIG manager when partitioning is enabled, and supporting validators that confirm the stack is healthy before marking nodes ready for GPU pods.

THINK OF IT LIKE

It is a turnkey AV installer — amplifier, speakers, wiring and calibration — instead of buying each component and hoping they are compatible.

WHY IT MATTERS IN PRODUCTION

Before the operator, platform teams SSHed to nodes, installed drivers, rebooted, and fought drift every upgrade. The operator makes GPU nodes cattle, but it also centralises risk: a bad ClusterPolicy upgrade can drain every GPU node at once. Interviewers want you to name the moving parts, not just say 'install the operator from OperatorHub'.

STEP BY STEP

- 1 Install the NVIDIA GPU Operator from OperatorHub or a mirrored catalog.
- 2 Define a ClusterPolicy matched to your OpenShift version and driver branch.

- 3 Wait for driver daemonsets, device plugin and toolkit pods to reach Ready on GPU nodes.
- 4 Validate with an CUDA sample pod requesting nvidia.com/gpu.

EVIDENCE YOU WOULD QUOTE

```
oc get clusterpolicy -n nvidia-gpu-operator -o yaml | head -40

oc get pods -n nvidia-gpu-operator -o wide

oc describe node <gpu-node> | grep -A3 'Allocatable'
```

RED FLAG - DO NOT SAY THIS

Do not install GPU drivers manually on operator-managed nodes. Two driver stacks on one node is a support nightmare.

EXPECT THIS FOLLOW-UP

GPU operator pods are Ready but workloads fail with 'unknown device'. What next?

Q90**FOUNDATION****What is Node Feature Discovery and why does the GPU Operator depend on it?****SAY THIS**

Node Feature Discovery runs a worker on each node that detects hardware and kernel features — CPU flags, PCI devices, kernel modules, memory — and publishes them as node labels. The GPU Operator uses those labels to decide which daemonsets to schedule: only nodes with NVIDIA PCI IDs get driver pods; only nodes with the right CPU features get certain validators. Without NFD, the operator cannot distinguish a GPU server from a generic worker and will either skip hardware or attempt installs on the wrong machines.

THINK OF IT LIKE

NFD is the inventory scanner in a warehouse. Without labels on the shelves, the forklift driver does not know which aisle has GPUs.

WHY IT MATTERS IN PRODUCTION

NFD is easy to overlook because it installs quietly alongside the GPU Operator, but it is the scheduling glue for every accelerated workload on OpenShift. Custom rules can expose InfiniBand HCAs, SR-IOV VFs or storage accelerators using the same mechanism. When labels are missing after a BIOS change, every downstream operator looks broken even though the hardware is fine.

STEP BY STEP

- 1 Deploy NFD operator and NodeFeatureRules for GPU PCI vendor IDs and related features.
- 2 Confirm nfd-worker pods label nodes with feature.node.kubernetes.io/pci-* labels.
- 3 Install GPU Operator with node affinity keyed on those labels.
- 4 Extend rules for InfiniBand, NVMe or custom accelerators as needed.

EVIDENCE YOU WOULD QUOTE

```
oc get nodefeaturediscovery -A

oc get node <gpu-node> --show-labels | grep feature.node.kubernetes.io

oc get nodefeaturerules -n openshift-nfd -o yaml | head -30
```

RED FLAG - DO NOT SAY THIS

Do not assume all GPU nodes auto-label correctly after a firmware upgrade. Re-run discovery or verify rules after hardware changes.

EXPECT THIS FOLLOW-UP

A new GPU node never gets driver pods. Which labels do you check first?

Q91**FOUNDATION****What makes a container CUDA-capable on OpenShift?****SAY THIS**

Three layers must align. The node runs an NVIDIA driver installed by the GPU Operator. The container runtime uses the NVIDIA container toolkit to inject driver libraries and device nodes when a pod requests nvidia.com/gpu. The image carries the CUDA user-space libraries and application binaries compatible with that driver version — typically from nvidia/cuda or a derived training framework image. Missing any layer produces errors ranging from 'cannot load libcuda.so' to silent CPU fallback.

THINK OF IT LIKE

CUDA in a container is a rental car: the road is the driver, the key is the device plugin, and the car itself is your image. You need all three to drive.

WHY IT MATTERS IN PRODUCTION

Platform teams own the first two layers; data science owns the image — until something breaks and both sides blame each other. Interview answers that mention driver/CUDA compatibility matrices and the difference between CUDA forward compatibility and exact version match sound experienced. OpenShift's restricted SCC also means images must not assume root-only GPU paths without testing.

STEP BY STEP

- 1 Confirm GPU Operator driver daemonset is Ready on the target node.
- 2 Request nvidia.com/gpu in pod resources limits; use runtimeClassName if configured.
- 3 Pull an image whose CUDA tag matches the deployed driver branch.
- 4 Run nvidia-smi inside the pod as a smoke test before the real workload.

EVIDENCE YOU WOULD QUOTE

```
oc run cuda-test --image=nvidia/cuda:12.2.0-base-ubuntu22.04 --restart=Never --overrides='{ "spec": { "containers": [ { "name": "cuda-test", "image": "nvidia/cuda:12.2.0-base-ubuntu22.04", "command": [ "nvidia-smi"], "resources": { "limits": { "nvidia.com/gpu": "1" } } } ] } }'
```

```
oc exec <pod> -- ls -l /usr/local/nvidia/lib64/libcuda.so*
```

```
oc describe pod <pod> | grep -i nvidia
```

RED FLAG - DO NOT SAY THIS

Do not pin ancient CUDA images against a new driver without checking compatibility. Framework wheels often lag driver releases.

EXPECT THIS FOLLOW-UP

Pod shows GPU allocated but nvidia-smi fails with driver/library version mismatch. Cause?

Q92

INTERMEDIATE

What is a ClusterPolicy and what does it configure?

SAY THIS

ClusterPolicy is the GPU Operator's cluster-wide configuration custom resource. One object declares the driver version, whether MIG or GPUDirect RDMA is enabled, daemonset placement tolerations, container registry mirrors, toolkit settings, DCGM exporter options, and validator behaviour. Changing it triggers a rolling reconcile across GPU nodes — driver pods restart, device plugins re-register — so it is the equivalent of a MachineConfig for the entire GPU software stack.

THINK OF IT LIKE

It is the single thermostat for the whole building's heating system. Turn it without telling anyone and every room changes temperature at once.

WHY IT MATTERS IN PRODUCTION

Version skew between ClusterPolicy and OpenShift is the most common production incident in GPU clusters. Teams upgrade OpenShift, leave the policy on an old driver branch, and wonder why pods hang in ContainerCreating. Senior answers treat ClusterPolicy changes like kernel upgrades: canary node, workload test, documented rollback.

STEP BY STEP

- 1 Match driver and operator versions to the OpenShift release compatibility matrix.
- 2 Set daemonset tolerations and node selectors for dedicated GPU pools.
- 3 Enable optional components — DCGM, MIG manager, sandbox workloads — explicitly.
- 4 Apply to one node pool first; run a CUDA validation job before fleet-wide rollout.

EVIDENCE YOU WOULD QUOTE

```
oc get clusterpolicy cluster-policy -n nvidia-gpu-operator -o yaml
oc get pods -n nvidia-gpu-operator -l app=nvidia-driver-daemonset
GPU operator validator pod logs after a policy change
```

RED FLAG - DO NOT SAY THIS

Do not edit ClusterPolicy during active training jobs without maintenance windows. Driver pod restarts evict running GPU workloads.

EXPECT THIS FOLLOW-UP

OpenShift upgraded but GPU pods fail with driver version mismatch. Where do you look?

Q93

INTERMEDIATE

What is GPUDirect RDMA and when does it matter for AI?

SAY THIS

GPUDirect RDMA lets a GPU read and write memory on a remote machine — or on a local NIC — without copying through host CPU buffers. For distributed training, that means all-reduce and parameter exchange use the NIC and GPU DMA engines directly, cutting latency and CPU overhead compared with staging tensors in pinned host memory first. It matters when training spans multiple GPUs on multiple nodes and the network — usually InfiniBand or RoCE — is the bottleneck.

THINK OF IT LIKE

Normal I/O is handing a package to a courier who drives it across town. GPUDirect RDMA is a pneumatic tube straight into the other office.

WHY IT MATTERS IN PRODUCTION

Enabling GPUDirect RDMA in ClusterPolicy is only half the story. You still need compatible NIC firmware, correct PCI topology, often ACS settings in BIOS, and NCCL or MPI configured to use the right transport. Interviewers probe whether you understand it as a system requirement — driver, NIC, topology, library — not a checkbox in the operator.

STEP BY STEP

- 1 Enable GPUDirect RDMA in ClusterPolicy when hardware supports it.
- 2 Verify NIC and GPU share favourable PCI topology; disable ACS if vendor guidance requires.
- 3 Install and configure the Network Operator for SR-IOV or macvlan as your CNI needs.
- 4 Validate with NCCL tests or vendor micro-benchmarks across nodes.

EVIDENCE YOU WOULD QUOTE

```
oc get clusterpolicy -o jsonpath='{.items[0].spec.gdrcopy/gpudirectRdma}{"\n"}'
```

```
nvidia-smi topo -m # run on GPU node via oc debug
```

NCCL all_reduce_perf or equivalent multi-node benchmark output

RED FLAG - DO NOT SAY THIS

Do not enable GPUDirect RDMA without measuring baseline training throughput first. Misconfigured RDMA can be slower than TCP.

EXPECT THIS FOLLOW-UP

NCCL hangs at init despite GPUDirect enabled. What topology issues do you suspect?

Q94**INTERMEDIATE****How do InfiniBand and SR-IOV support AI networking on OpenShift?****SAY THIS**

InfiniBand provides high-bandwidth, low-latency RDMA-capable networking ideal for multi-node training collectives. SR-IOV exposes virtual functions from a physical HCA or NIC directly into pods, bypassing the OVS kernel path for data-plane traffic. On OpenShift the SR-IOV Network Operator configures NodePolicy objects, creates SrioNetwork definitions and attaches VF interfaces to pods that need bare-metal-like performance. Together they give MPI and NCCL a direct path from GPU memory to the fabric.

THINK OF IT LIKE

InfiniBand is the dedicated race track. SR-IOV is giving each driver their own lane instead of sharing the public motorway.

WHY IT MATTERS IN PRODUCTION

AI networking on OpenShift trades Kubernetes convenience for performance. SR-IOV pods lose standard ClusterIP Services for that interface, need their own IPAM and sit outside default NetworkPolicy enforcement. Interviewers want you to state those trade-offs and tie them back to NUMA — the VF must sit on the same socket as the GPU.

STEP BY STEP

- 1 Install SR-IOV Network Operator; define SrioNetworkNodePolicy for HCAs or NICs.
- 2 Create SrioNetwork objects for the training VLAN or partition keys.

- 3 Schedule training pods with resource requests for VF devices and GPU resources.
- 4 Align CPU, GPU and HCA on one NUMA node; verify with topology tools.

EVIDENCE YOU WOULD QUOTE

```
oc get sriovnetworknodepolicy,sriovnetwork -n openshift-sriov-network-operator

oc get sriovnetworknodestate -n openshift-sriov-network-operator -o yaml | head -40

Inside training pod: ibstat ; ip -br link
```

RED FLAG - DO NOT SAY THIS

Do not deploy SR-IOV for AI without a network security model. Default NetworkPolicy does not protect VF traffic.

EXPECT THIS FOLLOW-UP

Training pods need both GPU and IB VF. How do you prevent them landing on the wrong NUMA node?

Q95

INTERMEDIATE

How do you run MPI workloads on OpenShift?

SAY THIS

MPI on OpenShift uses a launcher pattern: a Kubernetes Job or Operator-managed MPIJob creates a launcher pod that invokes mpirun or horovodrun, which spawns worker pods — one per rank — with shared SSH keys or the PMI2/PMIx interface over the cluster network. Workers need predictable DNS, often a headless Service, compatible RDMA interfaces if using InfiniBand, and the same container image on every rank. The OpenShift MPI Operator or Kubeflow Training Operator wraps this lifecycle so you submit a CR instead of hand-writing mpirun commands.

THINK OF IT LIKE

MPI is an orchestra conductor with musicians in separate rooms. The launcher is the conductor; headless Services are the intercom; every musician needs the same sheet music — the container image.

WHY IT MATTERS IN PRODUCTION

The failure modes are familiar to anyone who has run HPC on Kubernetes: DNS latency during scale-up, ranks starting before workers are ready, SELinux blocking intra-pod communication, and NCCL picking the wrong interface. Interviewers reward mentioning gang scheduling — all ranks or none — and resource quotas on GPU namespaces.

STEP BY STEP

- 1 Build or pull an MPI-enabled CUDA image with OpenMPI or Intel MPI and NCCL.
- 2 Install an MPIJob-capable operator; define worker and launcher resource requests.
- 3 Expose a headless Service for rank discovery; mount SSH secrets or use PMI.
- 4 Test with a small ring benchmark before scaling to full node count.

EVIDENCE YOU WOULD QUOTE

```
oc get mpijob -A # or training.kubeflow.org MPIJob CR

oc logs <launcher-pod> | grep -i 'rank\|connected'

mpi_info --param btl tcp # inside worker pod
```

RED FLAG - DO NOT SAY THIS

Do not run MPI at scale without testing DNS and NCCL socket selection at half scale first. Failures compound with rank count.

EXPECT THIS FOLLOW-UP

Rank 0 starts but workers never connect. What do you check in order?

Q96**INTERMEDIATE****What does the OpenShift Network Operator provide for GPU clusters?****SAY THIS**

The Network Operator manages cluster networking components as a coordinated stack: CNI plugin — OVN-Kubernetes by default — kube-multus for secondary interfaces, SR-IOV operator dependencies, IPAM controllers and sometimes Whereabouts or static IPAM for secondary networks. For GPU clusters it is the layer that lets training pods attach an SR-IOV or macvlan interface alongside the default pod network, which NCCL and MPI need when they cannot use the overlay.

THINK OF IT LIKE

The default CNI is the office LAN. Multus and SR-IOV are pulling a dedicated fibre cable straight to the machine room for workloads that outgrow Wi-Fi.

WHY IT MATTERS IN PRODUCTION

Candidates often conflate the GPU Operator with networking. The GPU Operator delivers drivers; the Network Operator delivers interfaces. A common incident is Multus configured but SrioNetworkNodePolicy missing, so pods declare a secondary net-attach-def and hang in ContainerCreating. Architecturally both operators must be upgraded and tested together.

STEP BY STEP

- 1 Confirm Network Operator is Healthy in ClusterOperator status.
- 2 Install SR-IOV or configure Multus net-attach-defs for the training network.
- 3 Annotate pods with k8s.v1.cni.cncf.io/networks for secondary interfaces.
- 4 Validate NCCL or MPI uses the intended interface name inside the pod.

EVIDENCE YOU WOULD QUOTE

```
oc get clusteroperator network -o yaml | grep -A5 conditions
```

```
oc get net-attach-def -A
```

```
oc describe pod <training-pod> | grep -i multus
```

RED FLAG - DO NOT SAY THIS

Do not attach SR-IOV networks without coordinating with the network team on VLANs and switch port modes. Kubernetes cannot fix a misconfigured top-of-rack.

EXPECT THIS FOLLOW-UP

Pod has Multus annotation but no secondary interface appears. Where do you debug?

Q97

SENIOR

What is MIG and when would you partition a GPU?

SAY THIS

Multi-Instance GPU is an NVIDIA A100-and-newer feature that splits one physical GPU into up to seven isolated instances, each with dedicated compute and memory. The GPU Operator's MIG manager applies a MIG partition profile on the node and advertises separate extended resources such as `nvidia.com/mig-1g.5gb`. You partition when many small inference or development workloads need GPU isolation but a full GPU each would leave expensive silicon idle — not when a single training job needs the entire device.

THINK OF IT LIKE

MIG is subdividing a house into flats with separate meters. Great for tenants who need privacy; wrong if one family wants the whole building for a party.

WHY IT MATTERS IN PRODUCTION

MIG profiles are fixed at node boot and require draining GPU workloads to change. Mixing MIG and full-GPU nodes in one pool demands clear taints and resource names so schedulers do not place a eight-GPU training job on MIG slices. Interviewers probe whether you know MIG is an operational commitment, not a runtime slider.

STEP BY STEP

- 1 Enable MIG in ClusterPolicy and deploy mig-manager on A100-or-newer nodes.
- 2 Choose a profile — e.g. 1g.5gb times seven — matching workload memory needs.
- 3 Taint MIG nodes; update pod specs to request `nvidia.com/mig-*` resources.
- 4 Plan profile changes as maintenance events requiring node drain.

EVIDENCE YOU WOULD QUOTE

```
oc get node <node> -o json | jq '.status.allocatable |
with_entries(select(.key|test("nvidia.com/mig"))).'
```

```
nvidia-smi -L # on node via oc debug
```

```
oc get clusterpolicy -o yaml | grep -i mig
```

RED FLAG - DO NOT SAY THIS

Do not enable MIG on nodes running large multi-GPU training. Those jobs need full devices and NCCL across NVLink.

EXPECT THIS FOLLOW-UP

Inference needs three profiles on one A100. How do you model that in Kubernetes resources?

Q98

SENIOR

How do training and inference workloads differ in placement requirements?

SAY THIS

Training is throughput-oriented and gang-scheduled: all GPUs across all nodes must start together, stay co-located for hours or days, and prefer NVLink within nodes and RDMA across nodes. Inference is latency- and density-oriented: many smaller pods, horizontal scaling, possibly MIG slices, strict pod anti-affinity for HA, and autoscaling based on queue depth or request rate. Training belongs on dedicated tainted pools with full GPUs; inference often shares nodes with careful quota and uses HPA or KEDA.

THINK OF IT LIKE

Training is a film crew on location for six weeks — everyone must arrive the same day with all the kit. Inference is a chain of coffee shops — open more counters when the queue grows.

WHY IT MATTERS IN PRODUCTION

Mixing them on one pool without taints is how production inference gets evicted by a researcher's multi-node Job. Interviewers want tiering: separate MachineConfigPools or node labels, different ClusterPolicy options — MIG on inference, full GPU on training — and different SLOs reflected in quotas and priority classes.

STEP BY STEP

- 1 Label and taint training nodes for exclusive multi-GPU Jobs with PriorityClass.
- 2 Place inference on MIG or fractional-GPU pools with HPA/KEDA and PodDisruptionBudgets.
- 3 Use topology spread for inference HA; use gang scheduling for training.
- 4 Monitor each tier separately — DCGM for utilisation, SLIs for inference latency.

EVIDENCE YOU WOULD QUOTE

```
oc get nodes -l nvidia.com/gpu.product -L node-role,cluster.ocs.io/openshift-storage
```

```
oc describe quota -n ai-training ; oc describe quota -n ai-inference
```

```
DCGM_FI_DEV_GPU_UTIL metrics split by namespace label
```

RED FLAG - DO NOT SAY THIS

Do not run bursty inference autoscaling on the same nodes as a week-long training job without isolation. Capacity math is not additive.

EXPECT THIS FOLLOW-UP

A researcher wants inference on training nodes 'just for a demo'. What do you say?

Q99**SENIOR****How do you monitor GPUs with DCGM on OpenShift?****SAY THIS**

DCGM — Data Center GPU Manager — exposes detailed GPU telemetry: utilisation, memory, temperature, power, ECC errors, NVLink counters and process-level stats. The GPU Operator deploys DCGM exporter as a daemonset that translates those metrics into Prometheus format on a scrape endpoint. You wire ServiceMonitor into the platform monitoring stack, build dashboards for utilisation and thermals, and alert on sustained thermal throttle, XID errors or zero utilisation on allocated GPUs — the classic sign of a stuck training job.

THINK OF IT LIKE

DCGM is the aircraft black box and instrument panel combined. Utilisation tells you the engines are running; XID errors tell you something is about to fall off.

WHY IT MATTERS IN PRODUCTION

Monitoring GPUs is not optional at scale — finance sees idle A100s, and silent ECC errors corrupt weeks of training. Interviewers want Prometheus metric names, integration with User Workload Monitoring for tenant namespaces, and the habit of correlating DCGM with node conditions and driver pod restarts after upgrades.

STEP BY STEP

- 1 Enable DCGM exporter in ClusterPolicy or the operator's monitoring section.
- 2 Create or enable ServiceMonitor so platform Prometheus scrapes GPU metrics.
- 3 Import NVIDIA or community Grafana dashboards for DCGM_FI_* metrics.

4 Alert on XID errors, thermal limits, and allocation-with-zero-utilisation.**EVIDENCE YOU WOULD QUOTE**

```
oc get servicemonitor -n nvidia-gpu-operator

curl -s http://<dcgm-exporter>:9400/metrics | grep DCGM_FI_DEV_GPU_UTIL

Prometheus: DCGM_FI_DEV_XID_ERRORS > 0
```

RED FLAG - DO NOT SAY THIS

Do not rely on nvidia-smi polling scripts in production. They miss ECC and NVLink counters DCGM exposes.

EXPECT THIS FOLLOW-UP

Metrics show 100% GPU utilisation but training loss is flat. What else do you check?

Q100**ARCHITECT****How would you design an AI node pool on OpenShift?****SAY THIS**

I split training and inference into separate MachineConfigPools with taints, labels and quotas. GPU Operator ClusterPolicy is pinned to tested driver branches per pool. NFD rules label GPU model and NIC type; SR-IOV policies expose HCAs on training nodes only. Network Operator delivers Multus definitions for RDMA VLANs. Monitoring via DCGM and User Workload Monitoring is mandatory before tenants arrive. Upgrade order is documented: OpenShift, then Network Operator, then GPU Operator, then validation Jobs — never the reverse.

THINK OF IT LIKE

It is building a hospital wing: separate theatres for surgery and clinics for outpatient, shared utilities, but different staffing rules and equipment in each room.

WHY IT MATTERS IN PRODUCTION

The architect answer ties hardware BOM to software policy: A100 nodes with NVSwitch for training, L40S or MIG-enabled nodes for inference, InfiniBand only where NCCL benchmarks justify it. It also covers tenancy — namespaces, quotas, PriorityClasses — and a golden path CI job that must pass after every platform change. That is what separates platform engineering from 'we installed the GPU operator once'.

STEP BY STEP

- 1 Define hardware SKUs per tier; document PCI topology and NIC placement per SKU.
- 2 Create dedicated MCPs with taints, PerformanceProfile if low-latency CPU matters.
- 3 Install NFD, Network Operator, GPU Operator with tier-specific ClusterPolicy.
- 4 Publish a tenant onboarding guide: quotas, MPIJob template, monitoring dashboards, upgrade windows.

EVIDENCE YOU WOULD QUOTE

```
Reference design: node pool diagram with taints, labels and network VLANs

oc get mcp,gpu,performanceprofile,sriovnetworknodepolicy -A

Golden path Job: CUDA + NCCL benchmark results after last platform change
```

RED FLAG - DO NOT SAY THIS

Do not buy GPUs before defining upgrade and rollback procedures. The most expensive component is downtime during a failed driver migration.

EXPECT THIS FOLLOW-UP

Finance wants one pool to cut cost. How do you explain why training and inference differ?

The last hour

You will not learn anything new in the last hour, so use it to protect what you already know. Say these six sentences out loud until they sound unrehearsed:

- 1 "Let me start with impact and scope, then the evidence I would collect."
- 2 "The owning controller here is X, so that is where I would read conditions first."
- 3 "The smallest safe action is Y, because it limits blast radius to one pool."
- 4 "I would validate against the customer-visible symptom, not just resource status."
- 5 "The trade-off is A versus B; I would choose A because of this constraint."
- 6 "To prevent recurrence I would add this alert and this guardrail."

Three habits that reliably lose offers

- Restarting things before reading anything. It sometimes works, and it always sounds junior.
- Answering a design question with a product name instead of a requirement and a trade-off.
- Claiming a backup, a policy or a failover works without ever having tested the restore.

Where to check current behaviour

Product behaviour moves. Before an interview, spot-check anything version-sensitive against the Red Hat OpenShift Container Platform documentation, the Red Hat Advanced Cluster Management documentation, the Red Hat Ansible Automation Platform documentation and the upstream Kubernetes concepts pages. If you are unsure in the room, say which release you are describing and offer to confirm - that is how senior engineers actually talk.